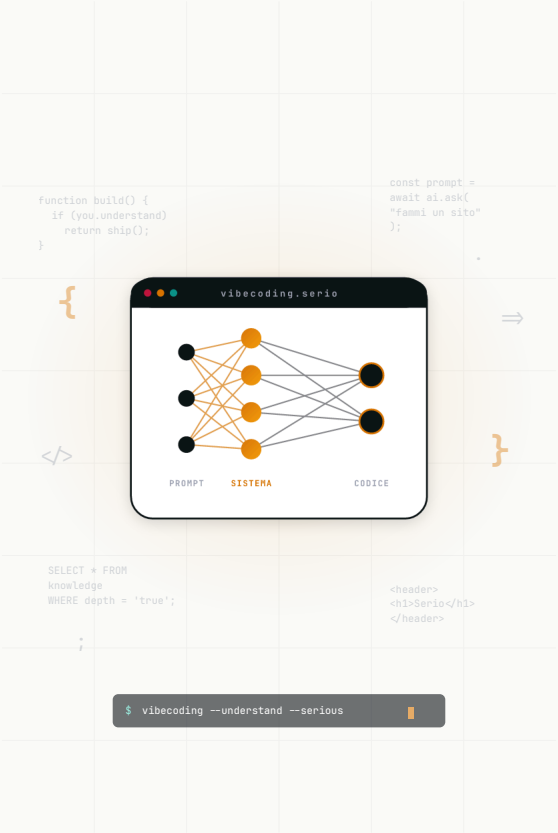
VIBECODING SERIO



Smetti di copiare prompt.

Inizia a capire cosa stai costruendo.

Il manuale per chi costruisce siti con l'AI
ma non capisce cosa sta costruendo.

Lupo Carro

Edizione 1.0 · 2026

{ vibecoding • serio }

Vibecoding Serio

Il manuale per chi costruisce siti con l'AI
ma non capisce cosa sta costruendo.

Lupo Carro

Edizione 1.0 — 2026

Vibecoding Serio
Il manuale per chi costruisce siti con l'AI ma non capisce cosa sta costruendo.

Edizione 1.0 — 2026
© 2026 Lupo Carro. Tutti i diritti riservati.

Composto in **Source Serif 4** per il testo, **Inter Tight** per i titoli, **JetBrains Mono** per il codice.
Impaginato con Typst.

vibecodingserio.vibecanyon.com
luca.cozza@gmail.com
github.com/cozzagit/vibecoding-serio

Indice

Modulo 0 — Le parole che ti servono per partire 12
Mettiti comodo 12
0.1 — Cosa significa «deploy», «produzione», «ambiente» 14
0.2 — Server, hosting, dominio, DNS: quattro cose diverse spesso confuse 17
0.3 — Cos'è un'API, una libreria, un framework, un pacchetto, un SDK 22
0.4 — FTP, SSH, Git: i tre modi per spostare codice 27
Chiusura del Modulo 0 31
🎯 Mini-quiz di autovalutazione 32
Modulo 1 — Cosa sta succedendo davvero quando premi «Deploy» 34
Mettiti comodo (di nuovo) 34
1.1 — Il vibecoder e il meccanico 35
1.2 — La mappa del territorio: dove vive un sito 37
1.3 — Cosa fa davvero Claude quando scrivi un prompt 40
1.4 — Il viaggio del click in 8 passi 44
Chiusura del Modulo 1 48
🎯 Mini-quiz di autovalutazione 49
Modulo 2 — Client e server: chi fa cosa 51
Mettiti comodo 51

2.1 — Il modello client-server in cinque minuti 52
2.2 — I linguaggi del client (frontend) 55
2.3 — I linguaggi del server (backend) 58
2.4 — Quando un linguaggio sta su entrambi (full-stack) 61
2.5 — Il database: il terzo pilastro 65
Chiusura del Modulo 2 68
🎯 Mini-quiz di autovalutazione 69
Modulo 3 — L'anatomia di una pagina web 71
Mettiti comodo (prendi anche carta e penna) 71
3.1 — I 5 macro-blocchi di ogni pagina 73
3.2 — Componenti di navigazione 77
3.3 — Componenti di contenuto 80
3.4 — Componenti di interazione 84
3.5 — Stati e feedback visivi 88
Chiusura del Modulo 3 91
🎯 Mini-quiz di autovalutazione 92
Modulo 4 — Il browser è stupido (ma fa tutto lui) 94
Mettiti comodo 94
4.1 — HTML in 30 minuti: i tag con significato 95
4.2 — CSS: layout responsive senza piangere 98
4.3 — JavaScript: la pagina si aggiorna o si ricarica? 102
4.4 — Il DOM e React: dove Claude mette le mani 103
4.5 — Server Components vs Client Components (Next.js 14+) . 106
Chiusura del Modulo 4 109
🎯 Mini-quiz di autovalutazione 109
Modulo 5 — Il server è il cameriere che nessuno vede 111
Mettiti comodo 111
5.1 — Frontend e backend: perché esistono separati 112
5.2 — API REST: la metafora del menu del ristorante 113
5.3 — I verbi HTTP: GET, POST, PUT, PATCH, DELETE 115
5.4 — I codici di stato HTTP: cosa sta gridando il server 118

5.5 — Headers, body, JSON: il formato di conversazione 120
Chiusura del Modulo 5 123
🎯 Mini-quiz di autovalutazione 123
Modulo 6 — Il database non è un foglio Excel (quasi) 125
Mettiti comodo 125
6.1 — Tabelle, righe, colonne 126
6.2 — Chiavi primarie e relazioni 127
6.3 — SQL in 10 minuti 130
6.4 — SQL vs NoSQL: Supabase, Firebase, Neon 133
6.5 — Migration e backup: non perdere mai i dati 135
Chiusura del Modulo 6 138
🎯 Mini-quiz di autovalutazione 138
Modulo 7 — Il viaggio del click in produzione 140
Mettiti comodo 140
7.1 — DNS in profondità 141
7.2 — HTTPS e certificati SSL 144
7.3 — Nginx: il buttafuori del server 145
7.4 — Variabili d'ambiente 148
7.5 — Cookie e sessioni: come il sito ricorda chi sei 150
7.6 — CORS in 3 paragrafi (l'errore più odiato spiegato) 153
Chiusura del Modulo 7 156
🎯 Mini-quiz di autovalutazione 156
Modulo 8 — Parlare a Claude come uno sviluppatore 158
Mettiti comodo 158
8.1 — Il problema del «fai una cosa bella» 159
8.2 — Descrivere il layout con nomi corretti 161
8.3 — Descrivere il comportamento con nomi corretti 164
8.4 — Spiegare un bug a Claude senza sapere il bug 167
8.5 — Il ciclo debug del vibecoder serio 170
Chiusura del Modulo 8 (e del libro) 172
🎯 Mini-quiz finale di tutto il libro 173

Perché il tuo sito è bellissimo in localhost e si rompe in produzione 176
Il momento esatto in cui ti rendi conto 176
Cosa pensa il vibecoder 177
Cosa succede davvero 178
I 5 motivi per cui il tuo sito si è rotto (in ordine di frequenza) . 178
Il framework «Diagnostica in 5 minuti» 181
Come parlarne a Claude per ottenerlo bene 182
La regola che ti porti a casa 184
Cosa è davvero un'API REST e perché la tua app si pianta a chiamarla 185
Il momento «boh» 185
Cosa pensa il vibecoder 186
Cosa succede davvero 186
Come parlarne a Claude per ottenerlo bene 188
La regola che ti porti a casa 190
Perché il tuo login non ricorda l'utente (e cosa sono i cookie) . . . 191
Il momento «ma come» 191
Cosa pensa il vibecoder 192
Cosa succede davvero 192
Come parlarne a Claude per ottenerlo bene 195
La regola che ti porti a casa 196
Il tuo database ha perso i dati. Perché e come non succede più . . 197
Il momento «no, no, no, no» 197
Cosa pensa il vibecoder 198
Cosa succede davvero: i 4 modi di perdere dati 198
Come non succede più: 4 strategie 199
Come parlarne a Claude per ottenerlo bene 202
La regola che ti porti a casa 203
CORS: l'errore più odiato spiegato in modo che tu possa fixarlo da solo 205

Il momento «questa cosa di nuovo» 205
Cosa pensa il vibecoder 206
Cosa succede davvero 206
Come si fixa 208
Diagnostica in 60 secondi 210
Come parlarne a Claude per ottenerlo bene 211
La regola che ti porti a casa 212
Glossario 1 — Programmazione & architettura 214
A. Ambienti e deploy (10 termini) 214
B. Infrastruttura web (14 termini) 215
C. Trasferimento codice (6 termini) 217
D. Linguaggi e framework (16 termini) 217
E. Concetti backend (12 termini) 218
F. Sicurezza e autenticazione (10 termini) 219
G. Database (12 termini) 220
Glossario 2 — Anatomia di una pagina web 222
A. Layout (5 termini) 222
B. Navigazione (8 termini) 223
C. Contenuto (12 termini) 224
D. Interazione (15 termini) 225
E. Feedback (8 termini) 226
F. Tipografia e elementi fini (5 termini) 227
✏️ Bonus: come trasformare in prompt prompt-ready 227
Appendice A — Cheat Sheet del Debug 230
Errori di rete / API 230
Errori JavaScript browser 232
Errori database 232
Errori build / deploy 233
Errori cookie / sessione 233
Pattern di prompt per ogni famiglia di errore 234
Appendice B — Checklist Pre-Deploy in 15 punti 235

Sicurezza & autenticazione 235
Database & dati 236
Performance & UX 236
SEO & analytics 237
⚠️ Bonus: stop-the-press 237
Appendice C — I 10 Prompt da Salvare 238
1. Setup nuovo progetto Next.js full-stack 238
2. Aggiungere autenticazione email/password 239
3. Debug API che ritorna 401 in produzione 239
4. Configurare CORS correttamente 240
5. Aggiungere upload di file 241
6. Query filtrata e paginata 241
7. Migration database con dati esistenti 242
8. Test end-to-end del flusso login 243
9. Documentazione automatica per API 243
10. Performance audit e ottimizzazione 244
✏️ Pattern generale del prompt informato 244
Appendice D — GDPR per Vibecoder Italiani 246
Cosa è GDPR (in 30 secondi) 246
Le 5 cose da fare assolutamente 247
Casi specifici per il vibecoder italiano 248
Checklist finale 249
Link utili 249

PREFAZIONE

Mettiti comodo

Hai aperto un libro che non avresti dovuto comprare. Sei arrivato qui perché qualcosa, in tutto questo costruire siti con l'AI, non ti torna. Il sito funziona, sì. Ma se ti chiedessero perché funziona, non sapresti rispondere. E quando si rompe — perché si rompe sempre — non sai nemmeno dove guardare.

Questo libro non ti renderà uno sviluppatore senior. Non è il suo scopo. Ti darà i fondamenti che separano un progetto fragile da uno che regge. I concetti, il lessico, le mappe mentali. Quelli che, una volta dentro, non ti escono più dalla testa.

Tre modi di leggerlo:

1. **Lineare** — dall'inizio alla fine, in 8-10 ore in 3-4 sessioni.
1. **Quick Wins** — i 5 capitoli «killer» che risolvono problemi che hai avuto la settimana scorsa.
1. **Riferimento** — glossari e appendici come dizionario quando ti servono.

Inizia come vuoi. Ma una raccomandazione: **non saltare il Modulo 0**. Sembra introduttivo, ma è il vocabolario su cui poggia tutto il resto.

L. C. — primavera 2026

Modulo 0 — Le parole che ti servono per partire

SUGGERIMENTO

4 capitoli · 14 pagine · 50 minuti di lettura
Pre-requisiti: nessuno. Hai aperto un browser almeno una volta.

Mettiti comodo

Prima di entrare nel dettaglio di come funziona un sito web, di come parlano un browser e un server, e di tutte quelle cose tecniche che ti spaventano — facciamo una cosa diversa.

Ti insegno **dieci parole**.

Dieci parole che probabilmente hai già sentito. Forse Claude le ha usate ieri in una sua risposta. Forse il tuo amico che si dice «developer» le ha buttate lì in chat. Tu hai annuito, perché annuire è gratis. Ma in realtà non sapevi cosa significassero davvero.

Le dieci parole sono queste:

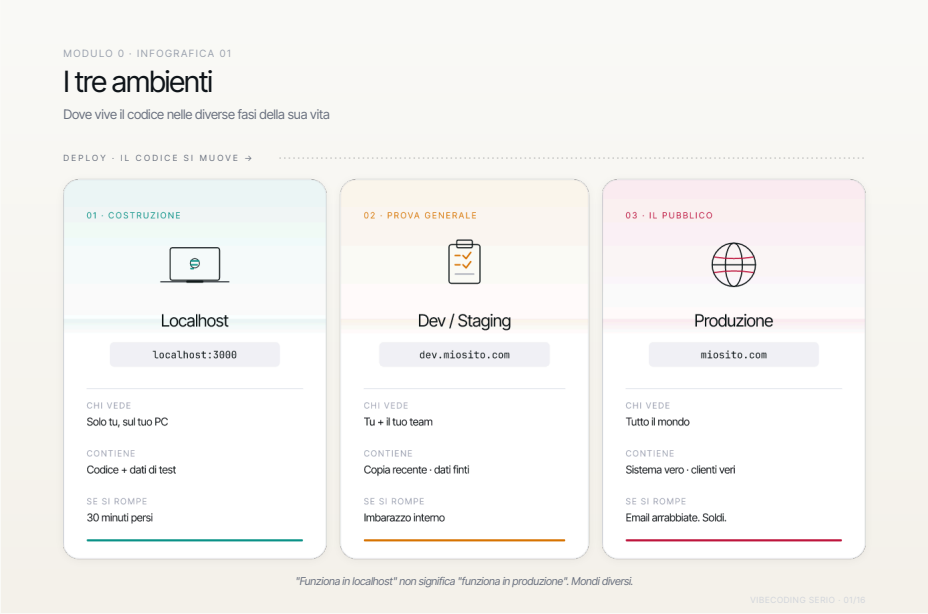
> **SUGGERIMENTO**
>
> deploy · ambiente · localhost · produzione · server · hosting · dominio · DNS · API · framework

Il resto del libro le useremo come fossero ovvie. Quindi sistemiamole adesso, in modo soft, senza fretta. Niente codice. Niente terminale. Solo le parole, le metafore giuste per ricordartele, e qualche esempio della vita di tutti i giorni.

Quando finisci questo modulo, non sarai diventato uno sviluppatore. Ma quando aprirai Claude e gli scriverai il tuo prossimo prompt, avrai cinque parole in più nel cassetto. E ogni parola in più è una iterazione in meno persa a chiarire.

Iniziamo dalla più importante: **dove vive il tuo sito**, e perché questa domanda ha più di una risposta.

0.1 — Cosa significa «deploy», «produzione», «ambiente»



Infografica 01 — I 3 ambienti — localhost, staging, produzione (a fianco di questo capitolo

Il momento «magia»

Hai costruito qualcosa con Bolt o Lovable o Cursor. Sul tuo PC, il sito si apre digitando un indirizzo strano nel browser, qualcosa tipo `http://localhost:3000`. Funziona. Lo vedi tu.

Poi clicchi un bottone — di solito si chiama «Deploy» o «Publish» — e dopo un minuto il sito è raggiungibile da un altro indirizzo. Adesso lo possono aprire i tuoi amici, tua madre, un cliente in Australia, chiunque conosca il link.

Quel «qualcosa è successo» che separa i due momenti ha un nome preciso: **deploy**.

Tre indirizzi, tre mondi

Il tuo sito può vivere in tre posti diversi nello stesso momento. Sembra strano, ma è normale. Si chiamano **ambienti**.

Ambiente 1 — Localhost (il tuo computer)

L'indirizzo è `localhost:3000` o `127.0.0.1:3000`. La parola «localhost» significa letteralmente «questo computer qui». Lo vedi solo tu, perché il sito sta girando sulla tua macchina. Se spegni il PC, il sito sparisce. Se chiudi il programma che lo fa girare, il sito sparisce.

È il tuo **laboratorio**. Qui puoi rompere quello che vuoi, sperimentare, cancellare il database per sbaglio, scrivere codice orribile. Nessuno lo vedrà mai. Quando hai finito di sperimentare, premi «deploy» e mandi solo la versione buona «fuori».

Ambiente 2 — Sviluppo (dev) o staging (server «di prova»)

Un indirizzo tipo `dev.miosito.com` o `staging-miosito.vercel.app`. È un server pubblico — quindi raggiungibile da chiunque conosca l'URL — ma di solito non è linkato da nessuna parte. È invisibile per Google, per il pubblico, per il tuo cliente. Lo usi per fare test «veri» prima di pubblicare la nuova versione.

Pensa al teatro: la prova generale. Tutto è in piedi (luci, scene, attori), ma il pubblico non c'è ancora.

Non tutti i progetti hanno questo ambiente. È utile quando lavori in team o quando il sito è critico (un e-commerce in saldi non vuole rischiare di esplodere durante il Black Friday).

Ambiente 3 — Produzione (prod) (il sito vero, live)

L'indirizzo è quello che dai al cliente. `miosito.it`. È quello che vedono **tutti**. Se si rompe, ricevi messaggi WhatsApp dei clienti alle 22:47.

Il termine «produzione» è preso in prestito dal mondo industriale. Una volta che il prodotto è «in produzione», non è più un prototipo: viene usato dai clienti veri.

«Deploy», spiegato in una riga

> **SUGGERIMENTO**
>
> **Deploy** = l'azione di spostare il codice da un ambiente al suo successivo.
>
> Tipicamente: dal tuo PC (localhost) → al server di sviluppo → al server di produzione.

Quando Vercel o Netlify ti dicono «deploy successful», stanno dicendo: «abbiamo preso il tuo codice, l'abbiamo messo sul server live, adesso il mondo lo vede».

Una tabella per non confonderti più

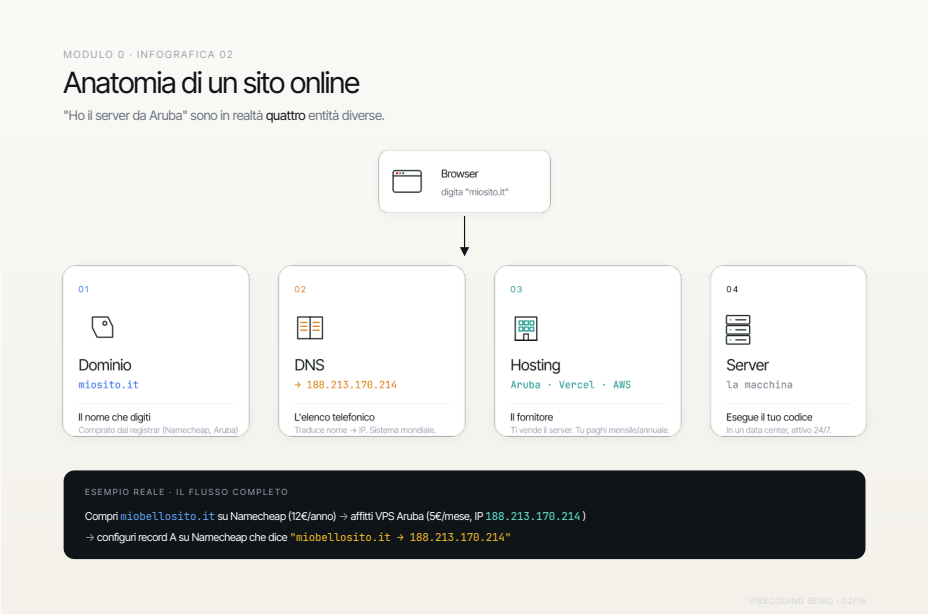
	Localhost	Dev / Staging	Produzione
URL tipico	`localhost:3000`	`dev.miosito.com`	`miosito.com`
Chi lo vede	Solo tu	Tu + team	Tutti
Cosa contiene	Codice + dati di test	Copia recente + dati finti	Sistema vero + dati clienti
Si rompe? Costa...	30 minuti di tempo	Imbarazzo interno	Email arrabbiate, soldi
Quanto puoi sperimentare	Tutto	Quasi tutto	Niente

Cosa ti porti via da questo capitolo

1. Il tuo sito può vivere in **tre ambienti**: localhost (PC tuo), dev/staging (server di prova), produzione (sito vero).
2. **Localhost mente**: funzionare lì non significa funzionare in produzione. Sono mondi diversi.
3. **Deploy** = spostare il codice da un ambiente all'altro. Il momento in cui il codice «esce di casa».
4. Quando qualcuno ti dice «in produzione non funziona», ti sta dicendo: «in localhost forse sì, ma sul sito vero c'è qualcosa che si comporta

diverso». È una **distinzione importantissima**, e il Modulo 7 ci tornerà a fondo.

0.2 — Server, hosting, dominio, DNS: quattro cose diverse spesso confuse



Infografica 02 — Anatomia di un sito online — quattro entità connesse

«Ho il server da Aruba»

Frase tipica del freelancer che vende siti. Suona competente. In realtà sta dicendo **quattro cose insieme** senza distinguerle. E quando una delle quattro va in crisi (e succede), non sa nemmeno dove guardare.

Smontiamole. Una alla volta.

Server — la macchina

Un **server** è un computer. Punto. Non è magia, non è «il cloud», non è una nuvola.

È un computer come il tuo, ma:

- Sta acceso 24/7
- Sta in una stanza enorme con altri mille computer (il «data center»)
- Non ha schermo, tastiera o mouse — ci si accede solo via rete
- Esegue **il tuo codice** invece che un browser o Word

Quando il tuo sito è «su Aruba», significa che il codice del tuo sito sta girando su uno di quei computer di Aruba.

> **SUGGERIMENTO**
>
> **Server fisico vs virtuale**: oggi quasi mai un server è una macchina fisica dedicata a te. Di solito è una macchina virtuale (VM) — un «computer simulato» che condivide la stessa macchina fisica con altri inquilini. Per te è uguale: lo controlli come fosse tutto tuo.

Hosting — il fornitore

L'**hosting** è chi ti **vende** quel server. Aruba, Vercel, Netlify, AWS, DigitalOcean, Hetzner. Sono tutti hosting provider.

Pensa alla differenza tra una **casa** e l'**agenzia immobiliare** che te l'affitta. La casa è il server. L'agenzia è l'hosting.

Diversi tipi di hosting per diversi bisogni:

- **Hosting condiviso** (Aruba «Hosting Linux», SiteGround): paghi 30€/anno, condividi la macchina con altri 200 siti, comodo per WordPress.
- **VPS — Virtual Private Server** (Aruba VPS, Hetzner): paghi 5-30€/mese, hai una macchina virtuale tutta per te. Più potente, più libertà, ma devi configurarla da solo.
- **PaaS — Platform as a Service** (Vercel, Netlify, Railway): paghi quanto consumi, non vedi mai il server, fai solo `git push` e tutto succede magicamente. Più caro per progetti grossi, comodissimo per cominciare.
- **IaaS — Infrastructure as a Service** (AWS, Google Cloud, Azure): roba enterprise, potentissima, complicata.

> **NOTA ITALIANA**
>
> **Per progetti italiani piccoli/medi**: Aruba VPS o Vercel coprono il 90% dei casi. Aruba se vuoi controllo totale e database/file pesanti. Vercel se vuoi zero pensieri e il sito è «front-end + qualche API».

Dominio — il nome

Il **dominio** è il nome che digiti nel browser. `miosito.it`, `vibecodingserio.vibecanyon.com`, `google.com`.

Si compra **separatamente** dall'hosting. Spesso da fornitori diversi:

- Lo compri da un **registrar** (Namecheap, Aruba domini, GoDaddy, Cloudflare Registrar)
- Lo paghi annualmente (5-50€/anno per i `.it` `.com` `.net` «normali»)
- Quando smetti di pagare, qualcun altro può prenderlo

Il dominio è solo un **nome**. Non contiene il sito. È come il nome sul citofono di casa: dice «qui abita Mario Rossi», ma Mario Rossi non è il citofono — è una persona che vive dentro l'appartamento.

> **SUGGERIMENTO**
>
> **Sottodominio**: quando vedi `dev.miosito.it` o `blog.miosito.it`, quei «dev» e «blog» sono **sottodomini**. Li configuri tu, gratis, una volta che hai il dominio principale. Non li paghi separatamente.

DNS — l'elenco telefonico

Qui arriviamo alla parte che a tutti i vibecoder sembra magia nera. In realtà è semplice.

Quando un utente digita `miosito.it` nel browser, il browser **non sa dove trovare il sito**. Il sito vive in un computer (server) da qualche parte nel mondo, identificato da un numero (l'**indirizzo IP**, tipo `188.213.170.214`). Ma l'utente ha digitato un nome, non un numero.

Come fa il browser a tradurre il nome in numero?

C'è un sistema mondiale che si chiama **DNS** (Domain Name System) che funziona come un **enorme elenco telefonico distribuito**. Tu chiedi «dov'è miosito.it?», il DNS risponde «all'indirizzo 188.213.170.214».

Configurare il DNS significa **scrivere nell'elenco telefonico**:

- «Quando qualcuno chiede `miosito.it`, mandalo a 188.213.170.214»
- «Quando qualcuno chiede `dev.miosito.it`, mandalo a 188.213.170.215»
- E così via.

Si fa nel pannello del registrar (Namecheap, Aruba, Cloudflare). Si chiamano «**record DNS**». I più comuni:

Record	Cosa fa	Esempio
A	Punta un dominio a un indirizzo IP	`miosito.it → 188.213.170.214`
CNAME	Punta un dominio a un altro dominio	`www.miosito.it → miosito.it`
MX	Dice «le email per questo dominio vanno qui»	`miosito.it → mail.aruba.it`
TXT	Note libere (per verifica proprietà, anti-spam)	`v=spf1 include:_spf.google.com`

> **SUGGERIMENTO**
>
> **Quando fai un cambio DNS**, ci vogliono da 5 minuti a 24 ore perché il cambiamento si propaghi in tutto il mondo. È il motivo per cui «ho cambiato il dominio ma il vecchio sito si vede ancora».

Mettiamo tutto insieme con un esempio reale

Vuoi mettere online un sito chiamato `miobellosito.it`. Ecco cosa succede in pratica:

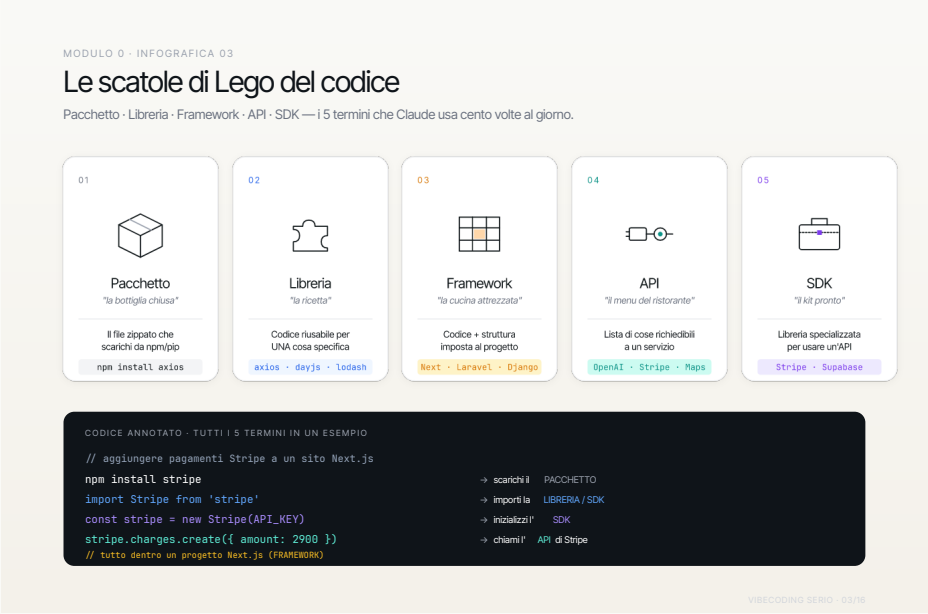
1. **Compri il dominio** `miobellosito.it` su Namecheap → 12€/anno
2. **Compri un VPS** su Aruba → 5€/mese, ti danno l'IP `188.213.170.214`
3. **Carichi il codice** del sito sul VPS (vedi capitolo 0.4)
4. **Configuri il DNS** su Namecheap: aggiungi un record A che dice `miobellosito.it → 188.213.170.214`
5. **Aspetti 30 minuti** per la propagazione DNS
6. Apri `https://miobellosito.it` e il sito è online

Hai usato **quattro entità diverse**: un dominio (Namecheap), un hosting+server (Aruba), e il DNS (configurato su Namecheap, ma è un sistema mondiale). Ognuna fa una cosa diversa.

Cosa ti porti via

1. **Server** = la macchina che esegue il tuo codice.
2. **Hosting** = il fornitore che ti vende il server.
3. **Dominio** = il nome (`miosito.it`). Si compra separatamente.
4. **DNS** = l'elenco telefonico mondiale che traduce il nome nel numero IP del server.
5. Spesso compri queste cose da fornitori **diversi** e devi farle parlare tra loro.
6. Quando un sito «non si vede», quasi sempre il problema è in **uno di questi quattro pezzi** — non nel codice.

0.3 — Cos'è un'API, una libreria, un framework, un pacchetto, un SDK



Infografica 03 — Le scatole di Lego del codice — API, libreria, framework, pacchetto, SDK

Le parole che Claude usa cento volte al giorno

Apri Claude. Gli chiedi di farti un sito. Lui ti risponde con un piano:

> **SUGGERIMENTO**
>
> «Userò il framework Next.js, con la libreria Tailwind per lo stile, il pacchetto next-auth per l'autenticazione, e chiamerò l'API di Stripe tramite il loro SDK ufficiale.»

Tu copi, incolli, premi invio. Funziona. Non hai capito niente di quello che ha detto.

Sistemiamo le cinque parole. È più facile di quello che sembra.

Una metafora: la cucina

Pensa a quando cucini.

- **Una ricetta** è un foglio con istruzioni: «Per fare la pasta al pomodoro, fai bollire l'acqua, aggiungi sale, butta la pasta...». Tu la usi, segui i passi, ottieni il piatto.
- Una **cucina già attrezzata** è un ambiente intero: hai i fornelli, le pentole, gli utensili, e una struttura imposta (devi muoverti tra fornello-piano-lavandino-frigo). Non parti da zero, parti da uno spazio che ha già delle regole.
- Una **bottiglia di salsa già fatta** è un prodotto pronto: la apri, la usi, non ti chiedi come l'hanno fatta. Risparmi tempo.
- Il **menu di un ristorante** è una lista di cose che puoi chiedere e ottenere senza cucinarle tu. Ordini, paghi, ricevi il piatto.
- Un **kit per fare i sushi** è una scatola che contiene tutto il necessario (riso, alga, salsa di soia, bacchette, tappetino) + istruzioni mirate per farti partire subito con un task specifico.

Ognuna di queste cinque cose esiste anche nel mondo del codice. Hanno nomi precisi.

Libreria — la ricetta

Una **libreria** è un pezzo di codice riusabile che fa **una cosa specifica**. Tu la usi per non scrivere quel codice da zero.

Esempi:

- `axios` → libreria per fare chiamate HTTP (parla con altri server)
- `dayjs` → libreria per gestire date e orari
- `lodash` → libreria con tante funzioni utility (raggruppare, ordinare, filtrare)
- `react-icons` → libreria con migliaia di icone già pronte

In codice, una libreria si «importa» e si «usa»:

```
import axios from 'axios';
const response = await axios.get('https://api.example.com');
```

Tu importi `axios`. Lui ti dà la funzione `get` già scritta. Tu la usi. Risparmi 200 righe di codice di rete che avresti dovuto scrivere a mano.

Framework — la cucina già attrezzata

Un **framework** è più grosso di una libreria. Non ti dà una funzione, ti dà **un ambiente intero con regole**.

Esempi:

- **Next.js** → framework per costruire siti e app web in React (ti impone struttura cartelle, routing, server-side rendering)
- **Laravel** → framework PHP per app web
- **Django** → framework Python per app web
- **Rails** → framework Ruby per app web

La differenza con la libreria è importante: con una **libreria** sei tu che scrivi il programma e chiami la libreria quando ti serve. Con un **framework** è il framework che esegue il programma e chiama il tuo codice quando gli serve.

In gergo si dice «un framework ti dice come deve essere fatta la tua app». Una libreria invece te la metti in tasca e la tiri fuori al momento giusto.

> **SUGGERIMENTO**
>
> **Regola pratica**: se la cosa che usi ti dice «metti i tuoi file in `pages/` e scrivi le tue pagine come componenti React», è un framework. Se ti dice «chiama questa funzione quando vuoi fare X», è una libreria.

Pacchetto (package) — la bottiglia di salsa

Un **pacchetto** è il **file zippato** in cui viene distribuita una libreria o un framework.

Quando dici «ho installato il pacchetto axios», stai dicendo: «ho scaricato il file zippato che contiene la libreria axios e l'ho messo nel mio progetto».

I pacchetti si scaricano da **registri di pacchetti**:

- **npm** (Node Package Manager) per JavaScript / TypeScript → migliaia di pacchetti
- **pip** per Python
- **composer** per PHP
- **gem** per Ruby
- **cargo** per Rust

Il comando tipico è `npm install nome-pacchetto` (o `pnpm`, o `yarn` — sono varianti dello stesso strumento).

> **SUGGERIMENTO**
>
> **Distinzione fine ma utile**: `axios` è la **libreria** (il codice). `axios-1.6.0.tgz` su npm è il **pacchetto** (il file). Nella pratica si usano come sinonimi: «ho aggiunto il pacchetto axios» = «ho aggiunto la libreria axios».

API — il menu del ristorante

Una **API** (Application Programming Interface) è una **lista di cose che puoi chiedere a un servizio**, e che il servizio è disposto a farti.

Esempi:

- L'**API di OpenAI**: puoi chiedere «rispondi a questa domanda», «genera questa immagine», «trascrivi questo audio»
- L'**API di Stripe**: puoi chiedere «addebita 29€ a questo cliente», «crea un abbonamento», «manda una fattura»
- L'**API di Google Maps**: puoi chiedere «dammi le coordinate di Via Manzoni 14, Como»

Per usare un'API:

1. Ti registri al servizio
2. Ottieni una **API key** (una password segreta)
3. Mandi richieste HTTP a un certo indirizzo (`https://api.openai.com/v1/...`)
4. Il servizio risponde con il risultato

L'API non è codice che gira sul tuo server. È un **contratto** tra te e qualcun altro: tu mandi una richiesta in un certo formato, loro rispondono in un certo formato. Tu non sai cosa succede in mezzo (e non ti interessa).

> **SUGGERIMENTO**
>
> **Anche tu puoi creare API**. Quando il tuo backend espone delle «rotte» (`/api/users`, `/api/login`), stai creando un'API. Il tuo frontend la chiama. Lo vedremo nel Modulo 5.

SDK — il kit per fare i sushi

Un **SDK** (Software Development Kit) è un **pacchetto che ti dà tutti gli strumenti per usare facilmente un'API specifica**.

Esempio: l'API di Stripe è complessa. Per usarla «a mano» devi scrivere richieste HTTP, gestire autenticazione, parsare risposte JSON. Stripe ha pubblicato un **SDK ufficiale** che ti dà funzioni già pronte:

```
import Stripe from 'stripe';
const stripe = new Stripe('sk_test_...');
const charge = await stripe.charges.create({ amount: 2900,
currency: 'eur' });
```

Tre righe. Tu chiami `stripe.charges.create(...)` e dietro le quinte l'SDK fa tutto il lavoro: chiama l'API di Stripe, gestisce errori, ritorna l'oggetto.

Praticamente, un **SDK è una libreria specializzata** per usare un servizio specifico. Stripe SDK, Supabase SDK, OpenAI SDK, AWS SDK.

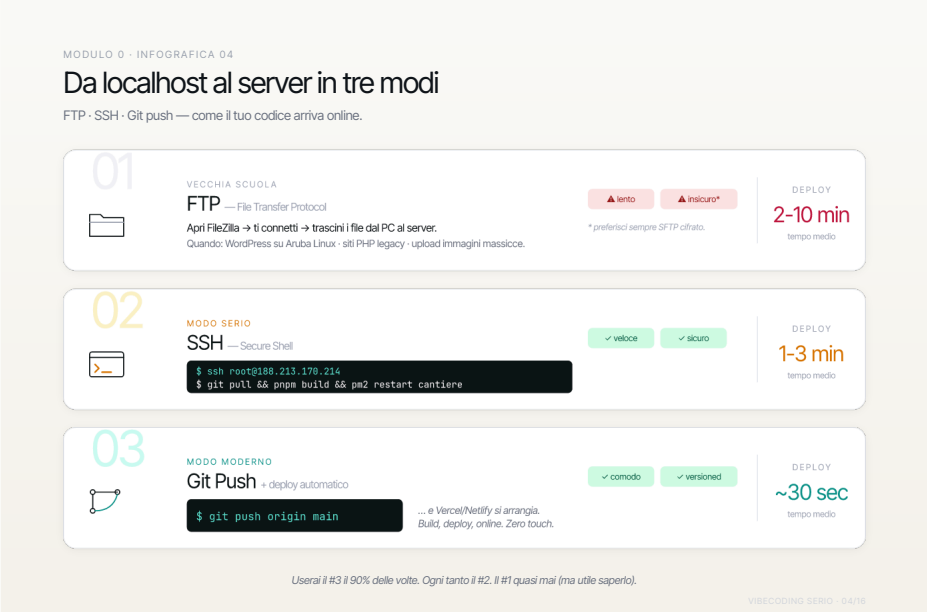
Tabella riassuntiva

Termine	Cos'è	Esempio	Lo «usi» così
Pacchetto	Il file zippato che si scarica	`axios-1.6.0.tgz`	`npm install axios`
Libreria	Codice riusabile per UNA cosa	`axios`, `dayjs`	`import axios from 'axios'`
Framework	Codice + struttura imposta	Next.js, Laravel	Costruisci tutto il progetto dentro
API	Lista di cose chiedibili a un servizio	API OpenAI, API Stripe	Mandi richieste HTTP
SDK	Libreria specializzata per usare un'API	Stripe SDK, Supabase SDK	Importi e chiami funzioni pronte

Cosa ti porti via

1. Quando Claude scrive «installa il pacchetto react-icons», sta dicendo «scarica questa libreria di icone».
2. Quando dice «useremo il framework Next.js», sta dicendo «tutto il progetto sarà costruito secondo le regole di Next.js».
3. Quando dice «chiamiamo l'API di OpenAI», sta dicendo «mandiamo una richiesta al loro server e aspettiamo una risposta».
4. Quando dice «usiamo l'SDK di Supabase», sta dicendo «usiamo le funzioni pronte di Supabase per non scrivere chiamate HTTP a mano».
5. Adesso queste parole non ti spaventano più.

0.4 — FTP, SSH, Git: i tre modi per spostare codice



Infografica 04 — Da localhost al server in 3 modi

Una domanda che nessuno ti ha mai spiegato

Hai scritto codice sul tuo PC. Funziona in localhost. Vuoi metterlo «sul server», così che il mondo lo veda. **Come ci arriva, fisicamente, da qui a là?**

Ci sono tre modi storici. Si sono susseguiti nel tempo, e oggi convivono. Ognuno è adatto a situazioni diverse.

FTP — il modo «vecchia scuola»

FTP sta per **File Transfer Protocol**. È nato negli anni "70 e si usa ancora oggi, soprattutto con hosting condivisi (Aruba «Hosting Linux», SiteGround).

Funziona così:

1. Apri un programma chiamato «client FTP» — il più popolare gratis è **FileZilla**
2. Inserisci tre cose: indirizzo del server (es. `ftp.miosito.it`), username, password
3. Vedi due pannelli affiancati: a sinistra i tuoi file locali, a destra i file sul server
4. **Trascini i file** da sinistra a destra. Vengono copiati sul server.

È il modo più «fisico» di mettere file online. Sembra di trascinare cartelle in Esplora Risorse.

Pro:

- Visivo, intuitivo per chi viene da Windows/Mac
- Funziona con qualsiasi hosting
- Gratis (FileZilla)

Contro:

- **Lento** se devi caricare un sito intero (centinaia di file = ore)
- **Insicuro** nella versione classica: la password viaggia in chiaro. Usa sempre **SFTP** (la versione cifrata) se disponibile
- Rischio errori umani: dimenticarti un file, sovrascrivere quello sbagliato
- Niente versioning: se rompi qualcosa, non puoi tornare indietro

Quando lo userai (raramente): WordPress su Aruba Hosting Linux. Caricamento di immagini massicce. Sito legacy in PHP che gira su hosting condiviso.

SSH — il modo «serio»

SSH sta per **Secure Shell**. È un modo per **aprire un terminale remoto** sul server.

Cioè: tu apri un terminale sul tuo PC, scrivi un comando, e quel comando viene eseguito **sul server**, non sul tuo PC. Come se fossi seduto fisicamente davanti al server.

Esempio reale di una sessione SSH:

```
$ ssh root@188.213.170.214        # mi connetto al server
$ cd /var/www/miosito              # entro nella cartella
del sito
$ git pull                         # scarico la versione
aggiornata
$ pnpm build                       # ricostruisco il sito
$ pm2 restart miosito              # riavvio l'applicazione
$ exit                             # esco dal server
```

Cinque comandi. Dieci secondi. Il sito è aggiornato.

Pro:
- **Velocissimo** una volta che ci hai preso la mano
- **Sicuro** (tutto cifrato, autenticazione con chiave invece di password)
- Puoi fare **qualsiasi cosa**: spostare file, modificare configurazioni, riavviare servizi, vedere i log
- Fondamentale per chi gestisce un VPS

Contro:
- **Solo riga di comando**: se hai paura del terminale, devi superarla
- Devi **configurare la chiave SSH** la prima volta (10 minuti di setup)
- Su un click sbagliato puoi rompere tutto (con grande potere viene grande responsabilità)

Quando lo userai (spesso, se hai un VPS): ogni volta che devi fare manutenzione, deploy manuale, troubleshooting di un sito su VPS Aruba, Hetzner, DigitalOcean.

Git push + deploy automatico — il modo «moderno»

Questo è il modo più **comodo** per chi non vuole pensare al server.

Funziona così:
1. Il tuo codice è in un **repository Git** (su GitHub, GitLab, Bitbucket)
2. Il tuo hosting (Vercel, Netlify, Railway, Render) è **collegato al repository**
3. Tu scrivi codice in locale, fai `git commit` e poi `git push`
4. Il servizio di hosting **si accorge automaticamente** del nuovo codice

5. Lo scarica, lo compila, lo mette online — tutto da solo
6. In 30 secondi il sito è aggiornato

Tu non tocchi mai il server. Non sai nemmeno dov'è. Lo gestisce il provider per te.

Pro:
- **Zero touch** dopo setup iniziale
- **Versioning gratis**: se rompi qualcosa, torni alla versione precedente con un click
- **Storia di tutto**: ogni modifica registrata, vedi chi ha cambiato cosa e quando
- **Anteprime gratis**: alcuni servizi (Vercel) ti danno un URL di prova per ogni branch del repo
- **Funziona da qualsiasi PC**: niente da configurare in locale a parte Git

Contro:
- Il **costo cresce** con l'uso (Vercel ti fa pagare per banda, build, ecc.)
- Sei **legato al provider**: se Vercel cambia prezzi o regole, ti adegui
- **Meno controllo**: se vuoi installare un programma custom sul server, non puoi
- Dietro le quinte ci sono ancora server, FTP, SSH — ma sono nascosti

Quando lo userai (probabilmente sempre): Next.js / Astro / SvelteKit su Vercel o Netlify. Progetti front-end. App piccole/medie. Tutto quello che fai con AI generative oggi.

Tabella decisionale: quale usare?

Situazione	Modo consigliato
Hai un sito Next.js / React / Astro statico	**Git push + Vercel/Netlify**
Gestisci un VPS Aruba con più siti, database, custom config	**SSH**
Hai WordPress su Aruba Hosting Linux	**SFTP**
Devi caricare un PDF in una cartella di un sito esistente	**SFTP** o **SSH**, dipende dall'host
Lavori in team su un progetto serio	**Git push + CI/CD**
Stai imparando, vuoi capire cosa succede	**SSH**, almeno una volta

Una nota sulla parola «Git»

Quando ho detto «Git push», ti ho usato un termine che merita un secondo.

Git è un sistema per **tracciare le modifiche al codice** nel tempo. Non è un modo per spostare file: è un modo per **versionare** il tuo progetto.

Ogni volta che fai un cambiamento e dici «questa è una modifica importante», Git la registra come un **commit**. Puoi tornare indietro a qualsiasi commit, vedere chi ha cambiato cosa, lavorare in parallelo con altre persone senza pestarsi i piedi.

GitHub (o GitLab, Bitbucket) è un **sito** che ospita repository Git online, in modo che tu possa condividerli con altri o usare i tuoi PC diversi.

Il «Git push» del modo moderno significa: «manda tutti i miei commit recenti al repository online». Da lì, il provider di hosting li raccoglie e fa il deploy.

> **SUGGERIMENTO**
>
> Approfondimento Git lo facciamo nel **Modulo 8**, quando parleremo di parlare a Claude. Per adesso ti basta sapere che `git push` = «manda il mio codice nel cloud, che poi ci pensa qualcun altro».

Cosa ti porti via

1. **FTP**: trascini file, vecchia scuola, lento. Si usa ancora con hosting condivisi.
2. **SSH**: terminale remoto sul server. Potente, veloce, richiede di non avere paura della riga di comando.
3. **Git push + deploy automatico**: il modo moderno. Tu fai `git push`, il provider fa il resto.
4. Probabilmente userai sempre il **terzo** modo, e ogni tanto il **secondo**.
5. Il primo modo è una conoscenza utile per non sentirti perso quando un cliente ti dice «ho un sito su Aruba con FTP».

Chiusura del Modulo 0

Hai letto il primo modulo. Adesso le **dieci parole** dell'inizio dovrebbero suonarti diverse:

> **SUGGERIMENTO**
>
> deploy · ambiente · localhost · produzione · server · hosting · dominio · DNS · API · framework

Non sei diventato uno sviluppatore. Sei diventato **uno che le parole le riconosce**. È un grosso primo passo: ogni libro tecnico, ogni risposta di Claude, ogni domanda che farai a uno sviluppatore vero adesso parte da una base condivisa.

Nei prossimi moduli costruiamo sopra:

- **Modulo 1** → cosa succede tecnicamente quando premi «deploy»
- **Modulo 2** → la grande divisione tra **client e server**, con tutti i linguaggi che vivono da una parte e dall'altra
- **Modulo 3** → l'**anatomia visuale** di una pagina web (quello che ti aiuta di più a parlare con Claude)

Prima di andare avanti, fai un piccolo test con te stesso.

🎯 Mini-quiz di autovalutazione

Rispondi a queste 5 domande. Se ne sbagli più di 2, rileggi il capitolo corrispondente prima di proseguire.

1. **Tuo amico ti dice: «il sito non funziona in produzione, in localhost sì». Cosa significa?**

> **SUGGERIMENTO**
>
> La risposta è alla fine.

2. **Hai comprato un dominio su Namecheap. Hai un VPS su Aruba. Ti manca un solo passaggio per far funzionare il dominio. Quale?**
3. **Differenza in una frase tra libreria e framework.**
4. **Quando Claude scrive «useremo il pacchetto stripe», cosa intende?**
5. **Hai un sito Next.js da deployare su Vercel. Quale dei tre modi userai (FTP, SSH, Git push)?**

Risposte

1. Significa che il sito è online ma c'è qualcosa che si comporta diversamente rispetto al suo PC. Quasi sempre è una variabile d'ambiente, un problema di CORS, o un database remoto diverso da quello locale. Lo vedremo a fondo nel Quick Win 1.
2. Configurare il **DNS**: aggiungere un record A su Namecheap che dice «manda chi cerca il mio dominio all'indirizzo IP del server Aruba».
3. Una **libreria** è codice che chiami quando ti serve (axios, dayjs). Un **framework** è codice che ti impone una struttura e che chiama il tuo codice (Next.js, Laravel).
4. «Scarica e installa la libreria di Stripe nel tuo progetto», facendo `npm install stripe`. Da lì potrai importarla nel tuo codice e chiamare le sue funzioni per parlare con l'API di Stripe.
5. **Git push**. Vercel è collegato al tuo repo GitHub: tu fai push, lui fa tutto il resto.

SUGGERIMENTO

Se sei arrivato fin qui senza saltare paragrafi, hai investito 50 minuti che ti faranno risparmiare ore di iterazioni con Claude nei prossimi mesi. Adesso prendi un caffè e poi vai al Modulo 1.

Modulo 0 redatto: 2026-04-25 · Versione 1.0 · 14 pagine · 3500 parole

Modulo 1 — Cosa sta succedendo davvero quando premi «Deploy»

SUGGERIMENTO

4 capitoli · 14 pagine · 50 minuti di lettura
Pre-requisiti: Modulo 0 letto. Conosci le parole deploy, ambiente, server, hosting, dominio, DNS, API, framework.

Mettiti comodo (di nuovo)

Adesso che hai le parole, possiamo guardare **come si mettono insieme**.

In questo modulo non ti insegno cose nuove — ti insegno a **vedere il sistema**. Quando un utente apre il tuo sito, succedono molte cose in pochi millisecondi. Tu le vedi solo come «ho premuto un link, è apparsa una pagina». Sotto, c'è una piccola coreografia tra browser, DNS, server, database e CDN.

Capirla cambia tutto. Quando il tuo sito si rompe, smetti di pensare «boh», e cominci a pensare «fammi vedere chi ha smesso di rispondere». È la

differenza tra il vibecoder che chiude il PC alle due di notte e il freelancer che fixa in dieci minuti.

Quattro tappe in questo modulo:
1. La metafora del meccanico (perché ti è utile capire e non solo usare)
2. La mappa del territorio (le 5 entità che compongono ogni sito web)
3. Cosa fa davvero Claude quando gli scrivi un prompt
4. Il viaggio del click in 8 passi (dal momento del click al risultato in pagina)

Andiamo.

1.1 — Il vibecoder e il meccanico

> **SUGGERIMENTO**
>
> Una metafora prima dei tecnicismi.

Due livelli di automobilisti

Ci sono due livelli di chi guida un'automobile.

Livello 1: chi sa guidare. Mette la chiave (o preme un pulsante), accelera, frena, parcheggia. Va dal punto A al punto B. È il 99% delle persone. È un livello onesto, valido. La macchina è uno strumento.

Livello 2: chi sa anche **cosa succede dentro**. Quando si accende la spia del motore, sa se è grave o se può aspettare il prossimo tagliando. Quando frena e sente un cigolio, sa che probabilmente sono le pastiglie consumate. Non è meccanico professionista. Ma quando qualcosa va storto, **non si sente nel panico**.

La differenza che conta

Il vibecoder oggi è al **livello 1** della programmazione web. Chiede a Claude di «fare un sito», riceve qualcosa che funziona, lo mette online. Va dal punto A al punto B.

Funziona finché tutto funziona. Quando si rompe (e si rompe sempre prima o poi), il vibecoder di livello 1 non sa neanche **dove guardare**. Apre Claude, scrive «non funziona», ottiene risposte generiche, prova fix a caso. Otto messaggi dopo, è peggio di prima.

Il **vibecoder serio** — quello che vogliamo diventare in questo libro — non è uno sviluppatore senior. Non scrive codice da zero. Continua a usare l'AI. Ma ha imparato il **livello 2**: quando il sito si rompe, sa **dove guardare** — e quindi sa **cosa chiedere a Claude**.

Cosa significa «capire il sistema»

Capire il sistema non significa saper costruire ogni pezzo da zero. Significa sapere:

- **Cosa** sta succedendo (browser → server → database)
- **Dove** può rompersi (DNS, certificato SSL, CORS, variabile d'ambiente mancante)
- **Quale entità** sta probabilmente avendo problemi quando vedi un certo errore
- **Chi chiamare**: a volte è il provider di hosting, a volte è il registrar del dominio, a volte è una variabile da settare nel pannello

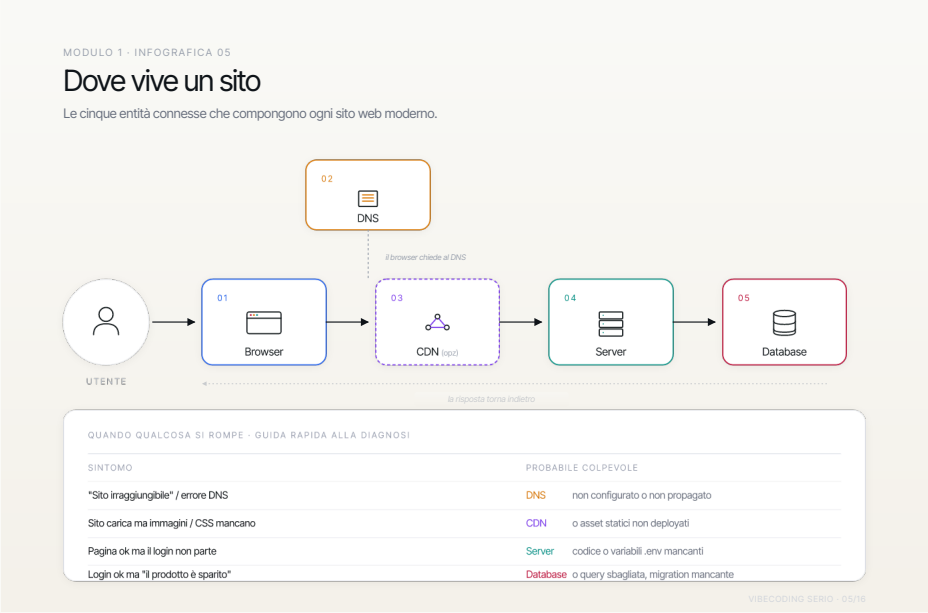
Una volta che hai questa mappa mentale, il «non funziona» smette di essere uno spavento. Diventa una **diagnosi**. E le diagnosi si fanno in dieci minuti, non in dieci ore.

I prossimi tre capitoli ti danno questa mappa.

Cosa ti porti via

1. Esistono due livelli di vibecoder: chi usa e basta, chi capisce abbastanza da diagnosticare.
2. Il livello 2 non è «diventa programmatore senior». È «smetti di andare nel panico quando qualcosa si rompe».
3. La mappa mentale del sistema è quello che separa i due livelli. È quello che costruiamo da qui in poi.

1.2 — La mappa del territorio: dove vive un sito



Infografica 05 — Dove vive un sito — utente, browser, DNS, CDN, server, database

Il sito non è «un posto»

Quando un utente visita `miosito.com`, il sito non vive in un solo posto. Vive in **almeno cinque entità diverse** che lavorano insieme.

Te le mostro tutte. Poi nei prossimi moduli le studieremo una per una.

Le cinque entità

1. **Il browser dell'utente**
(Chrome, Safari, Firefox, Edge)

È il programma che gira sul PC o sul telefono dell'utente. È la «porta d'ingresso» al tuo sito. Esegue il codice frontend (HTML, CSS, JavaScript) che il server gli ha mandato.

Caratteristica importante: il browser è **stupido per design**. Non si fida di niente, applica regole rigide di sicurezza (CORS, cookie httpOnly, HTTPS), e fa il minimo indispensabile da solo.

2. **Il sistema DNS**

L'elenco telefonico mondiale di cui abbiamo parlato nel Modulo 0. Quando l'utente digita `miosito.com`, il browser chiede al DNS dove vive il sito (qual è l'indirizzo IP del server). Il DNS risponde con un numero tipo `188.213.170.214`.

Ci vogliono 20-100 millisecondi. Se il DNS non risponde, il sito non parte nemmeno.

3. **La CDN**

(Content Delivery Network) — opzionale ma diffusissima

La CDN è una **rete di server distribuiti nel mondo**, ognuno con una copia dei file statici del tuo sito (immagini, CSS, JavaScript, font). Servizi come Cloudflare, Cloudfront (AWS), Fastly.

Quando un utente di Tokyo apre il sito, riceve i file dal server CDN più vicino a lui (Tokyo o Seoul) — invece che dal tuo server in Italia. Risparmia tempo: 200ms invece di 800ms.

Non tutti i siti usano una CDN. Vercel e Netlify ne hanno una integrata gratis. Su VPS Aruba la aggiungi tu (Cloudflare gratuito è perfetto).

4. **Il server (la tua applicazione)**

La macchina che esegue il **codice backend** del tuo sito. Quando il browser ha bisogno di dati dinamici (login, lista prodotti, salvataggio form), chiede al server.

Il server riceve la richiesta, **esegue il tuo codice**, magari interroga il database, e risponde.

Sui PaaS (Vercel, Netlify) il server è invisibile a te. Su VPS lo gestisci tu (con SSH, nginx, PM2).

5. **Il database**

Dove vivono i dati persistenti: utenti, ordini, prodotti, sessioni. Postgres, MySQL, MongoDB, Supabase, Firebase.

Il database **non parla mai direttamente col browser**. Solo il server può interrogare il database. Questa regola è fondamentale per la sicurezza: se il

browser potesse interrogare direttamente il DB, chiunque potrebbe rubare i dati di tutti.

Una mappa visuale

Queste cinque entità sono connesse da frecce. Il flusso è sempre lo stesso:

```
    Utente
      │ apre miosito.com
      ▼
   Browser
      │
      ▼
     DNS         (chi mi dà l'IP di miosito.com?)
      │
      ▼
    CDN         (file statici cachati vicino all'utente)
      │
      ▼
   Server       (codice backend, decide cosa fare)
      │
      ▼
  Database     (dati persistenti)
```

Le frecce vanno **a doppio senso** in realtà — il server risponde al browser, il database risponde al server. Ho semplificato per leggibilità.

Quando qualcosa si rompe, dove guardare

Questa mappa è oro. Quando il sito non funziona, percorrila in ordine:

Sintomo	Probabile colpevole
«Sito irraggiungibile» / errore DNS	DNS non configurato o non propagato
Sito si carica ma immagini/css mancano	CDN o asset non deployati
Pagina carica ma login non funziona	Server (codice o variabili d'ambiente)
Login funziona ma «il prodotto è sparito»	Database o query sbagliata

Sintomo	Probabile colpevole
Errore CORS in console	Server (regole CORS errate)
Lentezza solo in certe regioni del mondo	CDN mancante o configurata male

Vedi la differenza? Senza la mappa, dici «non funziona». Con la mappa, dici «il browser non riesce a parlare col server, probabilmente CORS o variabile mancante» — che è già un'ipotesi diagnostica.

Cosa ti porti via

1. Ogni sito web vive in **5 entità connesse**: browser, DNS, CDN (opzionale), server, database.
2. Il flusso va sempre **dall'utente al database e ritorno**.
3. Il **browser non parla mai col database**. Solo il server può interrogare il database. Questa regola ti salva la vita molte volte.
4. Quando qualcosa si rompe, **percorri la catena in ordine** invece di cercare a caso.

1.3 — Cosa fa davvero Claude quando scrivi un prompt

> **SUGGERIMENTO**
>
> Caso studio: il prompt più comune del vibecoder.

Il prompt che hai scritto cento volte

> **SUGGERIMENTO**
>
> «Crea un'app web con login Google e una dashboard utente.»

Lo hai scritto a Claude. Lui ti ha risposto con uno schema e con codice. Tu hai copiato, hai premuto qualche tasto, hai fatto deploy. Funziona.

Cosa ha fatto Claude in concreto? **Ha scritto sei pezzi di codice diversi, ognuno per una delle cinque entità che abbiamo appena visto** + una sesta entità (un servizio esterno). Te li mostro uno per uno.

I sei pezzi

Pezzo 1 — Pagina di login (frontend, gira nel browser)

Una pagina HTML con un bottone «Accedi con Google». Quando l'utente lo clicca, il browser viene reindirizzato a Google.

```
<button onClick={() => signIn('google')}>Accedi con Google</
button>
```

Banale. Ma è solo l'inizio.

Pezzo 2 — Configurazione OAuth (codice backend, gira sul server)

Il tuo backend deve dire a Google: «Ehi Google, sono il sito miosito.com, voglio che tu autentichi i miei utenti per me. Ecco le mie credenziali (clientid e clientsecret).»

Per fare questo, qualcuno deve essere andato sulla console di Google Cloud, aver creato un'applicazione OAuth, copiato due chiavi, e messe in un file `.env`. Spesso questo passaggio Claude lo dimentica di citarlo, e tu ti chiedi perché il login non parte.

Pezzo 3 — Endpoint di callback (codice backend, gira sul server)

Quando Google ha autenticato l'utente, lo manda di ritorno al tuo sito su un URL specifico, tipo `https://miosito.com/api/auth/callback/google`. Il tuo backend deve avere una rotta che riceve questa chiamata, verifica con Google che sia tutto ok, e crea una sessione per l'utente.

Pezzo 4 — Salvataggio sessione (cookie + database)

Una volta autenticato l'utente, devi **ricordare** che è autenticato. Altrimenti la prossima richiesta non lo riconosce.

Si fa con due strumenti:

- Un **cookie** nel browser (un piccolo testo che il browser re-invia ad ogni richiesta)
- Una **sessione** sul database (una nota che dice «questo cookie corrisponde a questo utente»)

Più tutto quello che Google ha mandato sull'utente: nome, email, foto profilo. Da salvare nel database.

Pezzo 5 — Pagina protetta (frontend + middleware backend)

La dashboard. Prima che il browser la renderizzi, il server deve verificare: «chi sta chiedendo questa pagina è autenticato?» Si guarda il cookie, si controlla la sessione, e se va bene si manda la dashboard. Se no, redirect al login.

Pezzo 6 — Servizio esterno (Google OAuth)

Tutto questo non funziona senza Google. Google è il **sesto pezzo**, ed è una delle cose più importanti da capire: **molti pezzi del tuo sito non sono «sul tuo server»**. Sono altrove, su servizi esterni che tu chiami via API.

Esempi di pezzi che spesso vivono altrove:

- Autenticazione → Auth0, Clerk, Supabase Auth, Google
- Pagamenti → Stripe, PayPal
- Email transazionali → Resend, Postmark, SendGrid
- File storage → Cloudflare R2, AWS S3, Supabase Storage
- AI → OpenAI, Anthropic, Replicate

Ognuno di questi è un pezzo «outsourcing» del tuo sistema. Tu gli mandi richieste API, loro fanno il lavoro complesso, ti rispondono.

Perché te li ho mostrati tutti

Quando il login Google «non funziona», **uno di questi sei pezzi è rotto**.

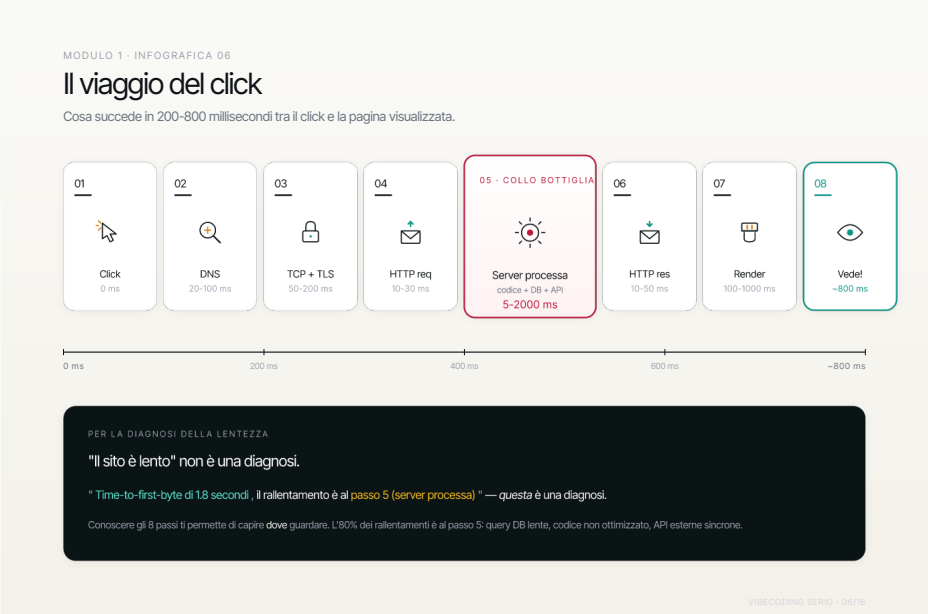
- Il bottone non parte → pezzo 1 (frontend)
- Google non riconosce il sito → pezzo 2 (configurazione OAuth)
- Dopo l'autenticazione redirect a una pagina di errore → pezzo 3 (callback URL non configurato in produzione)
- «L'utente è autenticato ma viene sloggato subito» → pezzo 4 (cookie non `Secure`, sessione non salvata)
- «La dashboard non protegge la pagina» → pezzo 5 (middleware mancante)
- «Errore 500 al click su Accedi» → variabili d'ambiente di Google mancanti in produzione

Senza questa mappa, scrivi a Claude «il login Google non funziona» e lui ti propone di provare 8 cose. Con questa mappa, scrivi «al click su Accedi vengo reindirizzato a Google ma poi sul callback ricevo un 401» e Claude ti porta dritto al pezzo 3.

Cosa ti porti via

1. Un prompt apparentemente semplice come «crea un login Google» nasconde **almeno 6 pezzi di codice diversi**.
2. I pezzi vivono in posti diversi: frontend, backend, database, e servizi esterni (Google, Stripe, ecc.).
3. **Buona parte del tuo sito non sta sul tuo server**: vive su servizi esterni che chiami via API.
4. Quando qualcosa «non funziona», il problema è quasi sempre in **uno specifico pezzo** — saperlo isolare riduce le iterazioni con Claude da 8 a 1.

1.4 — Il viaggio del click in 8 passi



Infografica 06 — Il viaggio del click — 8 passi numerati con icone e tempi

Una microstoria di 200 millisecondi

Un utente in un bar di Roma apre il telefono, digita `miosito.com`, preme invio. Dopo circa 200-800 millisecondi vede la pagina. In quei millisecondi succedono **otto cose distinte**.

Te le racconto come fossero scene di un film in slow motion. Le ho numerate.

Passo 1 — Il click

L'utente preme invio nel browser. Il browser vede `miosito.com` e si pone una domanda: «Dove diavolo è miosito.com?»

Lui non lo sa. È un nome, mica un indirizzo. Deve scoprirlo.

⏱️ Tempo: istantaneo

Passo 2 — DNS lookup

Il browser fa una richiesta al **resolver DNS** locale (di solito quello del provider di Internet, o il famoso `8.8.8.8` di Google).

«Hey, dov'è miosito.com?»

Il resolver, se non l'ha già in cache, fa una catena di chiamate ad altri server DNS (root, TLD, authoritative — tutto questo lo vediamo nell'infografica 16, Modulo 7) finché non trova la risposta: «miosito.com → 188.213.170.214».

Il browser adesso ha l'**indirizzo IP**. Sa dove andare.

⏱️ Tempo: 20-100 ms (10-30 ms se in cache)

Passo 3 — Connessione TCP + TLS

Il browser non può semplicemente «spedire» la richiesta. Deve **stabilire una connessione** col server. Si fa in due fasi:

TCP handshake (3 messaggi): «Ciao server, ci sei?» — «Sì, sono qui.» — «Bene, parliamo.» È il modo in cui Internet stabilisce ogni connessione.

TLS handshake (più messaggi): «Vorrei parlare in modo cifrato. Ecco la mia chiave.» — «Ecco il mio certificato firmato dall'autorità X.» — «OK, da adesso parliamo cifrato.» È quello che fa apparire il **lucchetto verde** nella barra del browser. Senza TLS, sarebbe HTTP «puro» e tutti potrebbero leggere i dati che viaggiano.

⏱️ Tempo: 50-200 ms

Passo 4 — HTTP request

Adesso che la connessione è aperta e cifrata, il browser manda finalmente la **richiesta vera**:

```
GET / HTTP/1.1
Host: miosito.com
User-Agent: Mozilla/5.0 ...
Accept: text/html, ...
Cookie: session_id=abc123...
```

Tradotto: «Dammi la pagina principale (`/`). Ah, e ti mando anche un cookie che dice chi sono.»

⏱️ Tempo: 10-30 ms (è solo l'invio dei dati)

Passo 5 — Server processa

Il server riceve la richiesta. Adesso esegue il tuo codice. A seconda di cosa fa la pagina, può:

- **Leggere file statici** (HTML, CSS, immagini) → veloce, pochi ms
- **Eseguire codice backend** (verificare il cookie, fare una query al database, chiamare un'API esterna) → da 10ms a parecchi secondi
- **Renderizzare HTML** dinamicamente con i dati appena letti dal database

Se il server deve fare 3 query al database e chiamare l'API di Stripe, qui ci vogliono 200-500 ms. Se è solo un file statico, 5 ms.

Questo è di solito il **collo di bottiglia** dei siti lenti. Quando un sito ci mette 4 secondi a caricare, il 95% di quelli sono qui dentro.

⏱️ Tempo: 5-2000 ms (estremamente variabile)

Passo 6 — HTTP response

Il server ha finito. Manda la **risposta** al browser:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Set-Cookie: session_id=xyz789; HttpOnly; Secure

<!DOCTYPE html>
<html>...
```

Tre cose:

- Lo **status code** (200, 404, 500...) — dice se è andata bene
- Gli **headers** (tipo di contenuto, cookie da settare, regole di cache...)
- Il **body** — il contenuto vero, in questo caso l'HTML della pagina

⏱️ Tempo: 10-50 ms

Passo 7 — Browser parsa e renderizza

Il browser riceve l'HTML. Inizia a «leggerlo» — in gergo si dice parsing. Mentre lo legge, scopre che la pagina ha bisogno di altri file:

- Un foglio di stile CSS (`<link rel="stylesheet" href="/style.css">`)
- Uno script JavaScript (`<script src="/app.js">`)
- Tre immagini (`<img src="...">`)

Per ognuno di questi file, il browser fa **un'altra richiesta** (passo 4 di nuovo, ma stavolta è più veloce perché la connessione è già aperta). Mentre li scarica, costruisce gradualmente la pagina:

1. **DOM**: la struttura della pagina (vedi Modulo 4)
2. **Stili applicati**: ogni elemento riceve i suoi colori, font, layout
3. **JavaScript eseguito**: aggiunge interattività (form, click, animazioni)
4. **Immagini caricate**: appaiono progressivamente

⏱️ Tempo: 100-1000 ms (dipende da quanto è «pesante» la pagina)

Passo 8 — L'utente vede

Finalmente, dopo tutto questo, **l'utente vede la pagina**. Probabilmente in 300-800 millisecondi totali, se tutto va bene.

Lui ha aspettato meno di un secondo. Non sa che dietro sono successe **otto cose distinte**. Tu, adesso, sì.

Riepilogo coi tempi

#	Cosa	Tempo tipico
1	Click	0 ms
2	DNS lookup	20-100 ms
3	TCP + TLS handshake	50-200 ms
4	HTTP request	10-30 ms
5	Server processa (codice + DB)	5-2000 ms
6	HTTP response	10-50 ms
7	Browser parsa + scarica risorse	100-1000 ms
8	Visualizzazione	—
	Totale tipico	**300-1500 ms**

Perché questa lista è oro

Quando uno strumento di analisi (Lighthouse, GTmetrix, Vercel Speed Insights) ti dice «il tuo sito è lento, score 47/100», ti sta dicendo: «uno di questi 8 passi sta richiedendo più tempo di quanto dovrebbe».

- Lentezza al passo 2 → DNS lento o non in cache
- Lentezza al passo 3 → TLS lento (raro), server geograficamente lontano
- Lentezza al passo 5 → query DB lente, codice mal scritto, API esterne lente — **questo è il 80% delle ottimizzazioni**
- Lentezza al passo 7 → file CSS/JS troppo grossi, troppi script bloccanti

Senza la lista, «il sito è lento» è un'affermazione vaga. Con la lista, è una **diagnosi**.

Cosa ti porti via

1. Quando un utente clicca un link, succedono **8 cose** in sequenza in poche centinaia di millisecondi.
2. Il **collo di bottiglia tipico** è il passo 5 (server + database).
3. Conoscere i passi ti aiuta a **diagnosticare la lentezza** invece di «boh il sito è lento».
4. È anche utile per la **sicurezza**: HTTPS al passo 3 protegge tutto quello che viaggia da lì in poi.

Chiusura del Modulo 1

Adesso hai due cose nuove in testa:

- **La mappa delle 5 entità** (browser, DNS, CDN, server, database) — il sistema in cui vive ogni sito web
- **Il viaggio del click in 8 passi** — la sequenza temporale di cosa succede quando l'utente apre una pagina

Più importante di tutto: hai capito che **il «sito» non è una cosa sola**. È un sistema di pezzi distribuiti che lavorano insieme. Ogni pezzo può rompersi indipendentemente. Sapere quali sono i pezzi è il primo passo per sapere dove guardare.

Nel **Modulo 2** entriamo nel cuore della distinzione più importante: **client e server**. Quali linguaggi vivono dove. Perché esistono separati. E perché JavaScript è speciale (gira in entrambi).

🎯 Mini-quiz di autovalutazione

1. **L'utente apre un sito. Il browser non riesce a contattare il server. Quale dei 5 pezzi del sistema è probabilmente il colpevole?**
2. **Il sito si carica ma il login non funziona («Accedi con Google» non parte). Quale pezzo del Modulo 1.3 cercheresti per primo?**
3. **Lighthouse ti dice che la pagina è lenta. Il «Time To First Byte» (TTFB) è di 1.8 secondi. In quale degli 8 passi del viaggio del click sta succedendo il rallentamento?**
4. **Perché il browser non può parlare direttamente col database?**
5. **Vero o falso: una CDN serve solo se il tuo sito ha utenti in tutto il mondo.**

Risposte

1. **Il DNS** (probabilmente). Se il browser non riesce a «trovare» il server, di solito il DNS non risponde, non è configurato, o non si è propagato dopo una modifica recente. Il server potrebbe essere giù, ma il sintomo «sito irraggiungibile» punta più spesso al DNS che al server.
2. **Il pezzo 1 (frontend del bottone)** o, più probabile in produzione, **il pezzo 2 (configurazione OAuth con Google)** — magari le credenziali Google sono nel `.env` locale ma non in quello di produzione.
3. **Passo 5 — il server processa**. TTFB elevato significa che il server impiega troppo tempo a generare la risposta. Spesso sono query lente al database, codice non ottimizzato, o chiamate ad API esterne sincrone.
4. **Per sicurezza**. Se il browser potesse parlare direttamente col database, qualunque utente con un po' di malizia potrebbe leggere/modificare i dati di tutti. Il server fa da «filtro autorizzato»: riceve la richiesta, verifica chi sei, decide cosa farti vedere.
5. **Falso**. Una CDN aiuta anche se i tuoi utenti sono tutti italiani — perché serve i file statici da nodi geograficamente vicini (Milano, Roma) molto più velocemente del tuo server centralizzato. Plus: scarica il tuo server dal traffico delle immagini/CSS, e ti protegge da picchi di traffico.

SUGGERIMENTO

Se hai sbagliato più di 2 risposte, rileggi i capitoli relativi prima del Modulo 2. Non c'è fretta. La mappa che stiamo costruendo qui è quella che userai per i prossimi 100 prompt a Claude.

Modulo 1 redatto: 2026-04-25 · Versione 1.0 · 14 pagine · 3700 parole

Modulo 2 — Client e server: chi fa cosa

SUGGERIMENTO

5 capitoli · 18 pagine · 1 ora 10 minuti
Pre-requisiti: Moduli 0 e 1 letti. Conosci le 5 entità (browser, DNS, CDN, server, database) e il viaggio del click.

Mettiti comodo

Adesso entriamo nel concetto **più importante** di tutto il libro.

Se assorbi solo questo modulo e lasci perdere il resto, hai già ottenuto il 60% del valore di «Vibecoding Serio». Tutto quello che farai con Claude da qui in poi sarà più chiaro, più diretto, con meno iterazioni perse.

Il concetto è: **client** e **server** sono due mondi separati, con regole diverse.

Sapere quale codice gira da una parte e quale dall'altra ti permette di:

- Capire perché certi errori succedono (e dove cercarli)
- Scrivere prompt più precisi a Claude
- Riconoscere subito cosa Claude sta scrivendo (o sbagliando)
- Non commettere errori di sicurezza ovvi

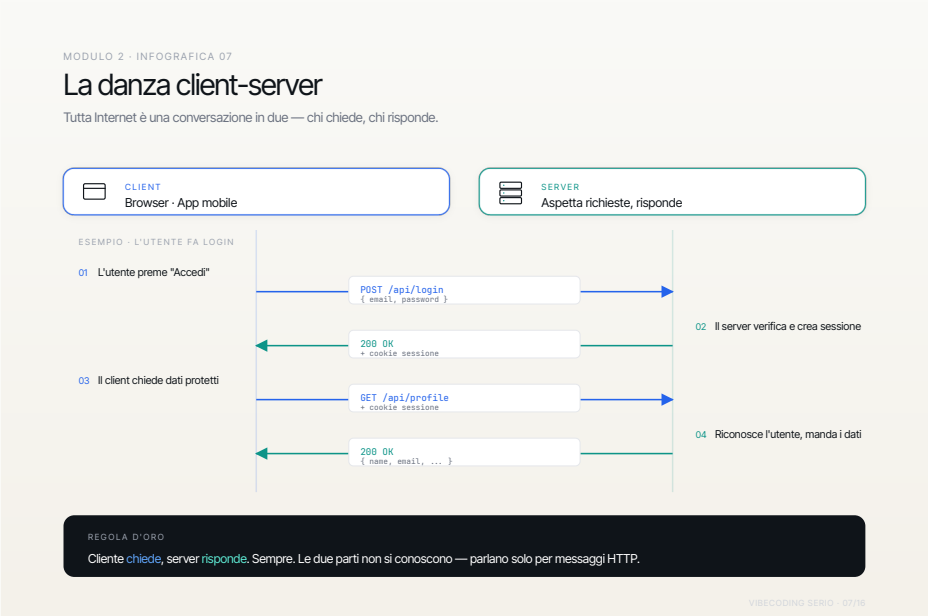
In questo modulo:

1. Il modello client-server in 5 minuti
2. I linguaggi del client (frontend) — cosa gira nel browser
3. I linguaggi del server (backend) — cosa gira sul server

4. Quando un linguaggio sta su entrambi (full-stack)
5. Il database: il terzo pilastro

Andiamo.

2.1 — Il modello client-server in cinque minuti



Infografica 07 — La danza client-server

Una conversazione in due

Tutta Internet, fondamentalmente, funziona così: **qualcuno chiede, qualcun altro risponde**.

- Tu apri un sito: il tuo browser **chiede**, il server del sito **risponde**.
- Apri una mail su Gmail: il browser **chiede**, il server di Google **risponde**.
- L'app di tua banca aggiorna il saldo: l'app **chiede**, il server della banca **risponde**.

Chi chiede si chiama **client**. Chi risponde si chiama **server**. Sempre.

Le due parti hanno ruoli diversi

Sono due metà della stessa moneta, ma fanno cose diverse.

Il client (di solito il browser dell'utente, o un'app mobile):
- Mostra cose all'utente (testi, immagini, bottoni)
- Riceve i suoi click, le sue digitazioni
- Manda **richieste** al server
- Riceve **risposte** e aggiorna lo schermo

Il server:
- Aspetta richieste
- Esegue codice (esempio: cerca nel database, calcola un prezzo, manda un'email)
- Risponde al client con il risultato
- Non vede mai l'utente. Sa solo che gli arrivano richieste.

Pensa al ristorante: il **cliente al tavolo** ordina dal menù. Il **cuoco in cucina** non vede il cliente. Riceve il foglietto dal cameriere (la richiesta), prepara il piatto (esegue codice), manda il piatto al cameriere (la risposta). Mai contatto diretto. Le due parti **non si conoscono**, parlano per messaggi.

Perché esistono separati

Una domanda legittima: «perché il browser non fa tutto da solo? Perché serve un server?»

Tre risposte, in ordine di importanza:

1. **Sicurezza**

Se il browser potesse fare tutto da solo, qualunque utente con un po' di malizia potrebbe leggere/modificare i dati di tutti. Un'app bancaria che gira «tutta nel browser» significherebbe che chiunque apre Chrome DevTools potrebbe vedere i saldi di tutti gli altri.

Il server agisce da **filtro autorizzato**: riceve la richiesta, verifica chi sei (autenticazione), decide cosa farti vedere (autorizzazione), e ti manda solo il pezzo che ti spetta.

2. **Dati condivisi**

Se hai un blog con 1000 utenti che leggono e commentano, quei dati (post, commenti) devono stare in un posto **centralizzato**. Non possono stare «nel browser di Mario», perché Lucia non vedrebbe mai i commenti di Mario.

Il server tiene i dati in un posto unico (il database), accessibile a tutti i client autorizzati.

3. **Cose che il browser non può fare**

Mandare un'email, processare un pagamento Stripe, integrare un servizio AI come OpenAI, salvare un file su un disco — queste cose il browser non le fa. O perché non ha gli strumenti, o perché sarebbe insicuro fargliele fare.

Il server le fa per conto del client.

Quando il confine si sposta (full-stack moderni)

Una nota importante. In progetti moderni come **Next.js**, frontend e backend possono **condividere lo stesso codice** e girare in entrambi i mondi.

Esempio: una funzione `validateEmail()` che controlla se una mail è ben formattata può girare:

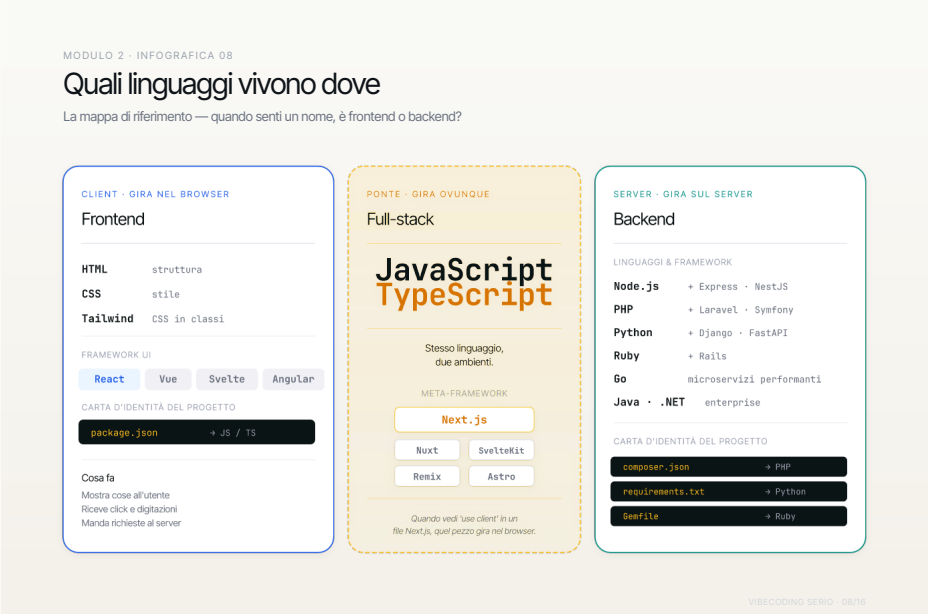
- **Lato client** per dare feedback istantaneo all'utente mentre digita
- **Lato server** per la sicurezza vera (perché lato client si può ingannare)

Stessa funzione, stesso file, due ambienti. Lo vedremo in 2.4.

Cosa ti porti via

1. Tutta Internet è una **conversazione client-server**: chi chiede, chi risponde.
2. **Il client** mostra e raccoglie input. **Il server** custodisce dati e fa cose pesanti.
3. Esistono separati per **sicurezza, dati condivisi, e cose che il browser non sa fare**.
4. La regola generale: **se richiede una password o un dato di altri utenti, deve passare dal server**. Senza eccezioni.

2.2 — I linguaggi del client (frontend)



Infografica 08 — Quali linguaggi vivono dove — la mappa di riferimento di tutto il libro

Cosa gira nel browser

Quando apri Chrome, Firefox o Safari, dentro al browser girano **solo tre tecnologie native**:

- **HTML** — la struttura della pagina
- **CSS** — l'aspetto visivo
- **JavaScript** — l'interattività

Punto. Tutto il resto che vedi su un sito (React, Vue, Tailwind, Svelte...) è **costruito sopra** queste tre tecnologie. Il browser non sa cosa è React. Sa solo HTML, CSS e JavaScript. I framework alla fine si «compilano» in questi tre.

Vediamoli uno per uno, senza mistificare.

HTML — lo scheletro

L'**HTML** (HyperText Markup Language) è una specie di Word per la struttura. Dici «questo è un titolo», «questo è un paragrafo», «questo è un bottone»,

«questa è un'immagine». Non dici **come si vede** (lo fa il CSS), né **come si comporta** (lo fa il JavaScript). Solo la struttura.

```
<header>...</header>
<main>
  <h1>Titolo della pagina</h1>
  <p>Un paragrafo di testo.</p>
  <button>Cliccami</button>
</main>
<footer>...</footer>
```

Quando Claude scrive una pagina, scrive HTML. È la prima cosa che genera.

CSS — la pelle

Il **CSS** (Cascading Style Sheets) descrive come si vede l'HTML. Colori, font, spaziature, layout, animazioni.

```
button {
  background: #2563EB;
  color: white;
  padding: 12px 24px;
  border-radius: 8px;
}
```

Tutto il «design» di un sito è CSS. È quello che separa un sito che sembra del 1998 da uno che sembra del 2026.

> **SUGGERIMENTO**
>
> **Tailwind CSS**: la libreria CSS più popolare oggi. Invece di scrivere `padding: 12px 24px`, scrivi `className="py-3 px-6"`. Stessa cosa, sintassi più compatta. Quando Claude scrive interfacce moderne, quasi sempre usa Tailwind.

JavaScript — i muscoli

Il **JavaScript** (JS) è il linguaggio che dà **vita** alla pagina. Quando clicchi un bottone, quando un menù si apre, quando un form si valida prima di inviarsi — tutto JavaScript.

```
button.addEventListener('click', () => {
  alert('Hai cliccato il bottone!');
});
```

Senza JavaScript, una pagina è una specie di brochure: la guardi, ma non interagisce.

TypeScript — JavaScript con i superpoteri

Il **TypeScript** (TS) è JavaScript con un controllo aggiuntivo dei «tipi». Tu dici «questa variabile è un numero», «questa funzione ritorna una stringa», e lo strumento ti avvisa se sbagli prima ancora di provare il codice.

```
function somma(a: number, b: number): number {
  return a + b;
}

somma('ciao', 5); // ❌ TypeScript ti dice "errore: 'ciao'
non è un numero"
```

In progetti seri si usa TypeScript invece di JavaScript «puro». Riduce i bug. Claude oggi quasi sempre scrive TypeScript per i progetti nuovi.

React, Vue, Svelte — i framework UI

Scrivere applicazioni complesse direttamente in JavaScript è faticoso. I **framework UI** ti danno strumenti per organizzare il codice in **componenti riusabili**.

- **React** (di Meta): il più diffuso. Creato nel 2013. Quello che Claude usa di default.
- **Vue**: alternativa più semplice da imparare. Popolare in Europa.
- **Svelte**: il più moderno (2019). Più snello, performance migliori.

- **Angular** (di Google): più enterprise, più rigido. Lo trovi in progetti aziendali grandi.

Sono tutti uguali in cosa fanno (UI dichiarativa con componenti) — diversi in come lo fanno. **React vince per popolarità e per quantità di librerie disponibili**, e per questo è quello su cui Claude ha la mano più allenata.

Next.js, Nuxt, SvelteKit — i meta-framework

Un livello sopra: i **meta-framework**. Prendono un framework UI (es. React) e ci aggiungono tutto quello che serve per costruire un sito serio:

- **Next.js** (basato su React) → il più popolare oggi
- **Nuxt** (basato su Vue)
- **SvelteKit** (basato su Svelte)
- **Remix** (basato su React)
- **Astro** (per siti statici/contenuto, agnostico)

Quando Claude scrive un'app web moderna, in 8 casi su 10 usa **Next.js**.

Cosa ti porti via

1. Il browser nativamente conosce solo **HTML + CSS + JavaScript**.
2. **Tailwind** è CSS scritto in modo più rapido.
3. **TypeScript** è JavaScript con controllo extra.
4. **React, Vue, Svelte** sono framework UI per organizzare codice JS in componenti. **React vince per popolarità**.
5. **Next.js** è il meta-framework più usato oggi (React + features). Quando vedi `pages/` o `app/`, è Next.js.

2.3 — I linguaggi del server (backend)

> **SUGGERIMENTO**
>
> Continua l'infografica 8.

Cosa gira sul server

Sul server può girare **qualunque linguaggio di programmazione**. Letteralmente. Il browser è limitato a JavaScript, il server no.

I più usati nel mondo web (in ordine di diffusione):

Node.js — JavaScript sul server

Node.js è una runtime che permette di eseguire JavaScript **fuori dal browser**. È la rivoluzione del 2009: per la prima volta, lo stesso linguaggio gira sia frontend che backend.

Framework backend Node.js comuni:
- **Express**: il più semplice, il «Flask» del mondo Node
- **Fastify**: alternativa moderna, performance migliori
- **NestJS**: più strutturato, stile Angular

Quando vedi su Vercel o Netlify un'app Next.js, sotto sta girando Node.js. Quando Claude scrive un endpoint `/api/...`, è codice Node.js.

PHP — il classico

Il **PHP** è il linguaggio dei siti web «tradizionali». WordPress, Joomla, Drupal — tutti PHP. Sembra antico, ma è ovunque: il **40% di Internet** gira ancora su PHP.

Framework PHP moderni:
- **Laravel**: il più popolare, elegante, sintassi pulita
- **Symfony**: più enterprise, modulare

Su hosting Aruba «Linux Hosting» il default è PHP. Se hai un sito legacy o un'agenzia te ne propone uno, è quasi sicuramente PHP+Laravel o WordPress.

Python — il linguaggio dell'AI

Il **Python** è il linguaggio scientifico per eccellenza. È diventato lo standard per:
- AI / Machine Learning (TensorFlow, PyTorch, OpenAI SDK)
- Data Science (pandas, numpy, jupyter)
- Web scraping
- Scripting / automation

Per app web esistono framework:
- **Django**: il «Laravel di Python», batterie incluse
- **Flask**: minimalista, fai tutto a mano
- **FastAPI**: il moderno, ottimo per le API

Se il tuo progetto coinvolge AI, processamento dati, modelli ML — quasi sicuramente Python. Cantiere ad esempio ha un servizio Python per l'OCR delle bolle, anche se il sito principale è Next.js.

Ruby — produttività estrema

Ruby + **Rails** è una coppia famosa. Usato in startup classiche (Airbnb, GitHub, Shopify, Basecamp). Ottimo per partire velocissimo. Meno popolare oggi rispetto a Node.js / Next.js, ma ancora forte.

Go — performance e semplicità

Go (o «Golang») è il linguaggio creato da Google. Più «low-level» rispetto a Node/Python, ma molto veloce. Perfetto per microservizi, sistemi che devono reggere tanto traffico, infrastrutture cloud.

Java, Kotlin, .NET (C#) — enterprise

Java (e **Kotlin**, suo cugino moderno) e **.NET** (Microsoft, linguaggio C#) sono i giganti del mondo enterprise. Banche, assicurazioni, grandi aziende. Robusti, super-tipizzati, lenti da scrivere ma stabili.

Se sviluppi un piccolo SaaS o un'app web standard, non li userai. Se finisci a lavorare con un'azienda da 1000 dipendenti, sì.

Tabella decisionale: cosa userà Claude in base al tuo progetto

Tipo di progetto	Stack tipico che Claude propone
App web moderna full-stack (login, dashboard, CRUD)	**Next.js** + Supabase / Postgres
Sito vetrina + form contatti	**Astro** o **Next.js** statico + form action
WordPress / blog tradizionale	**PHP + WordPress** (raramente Claude propone WordPress, ma se hai un legacy...)
Backend API per app mobile	**Node.js + Express** o **FastAPI**
Progetto AI / machine learning	**Python + FastAPI** + librerie ML
Microservizio ad alta performance	**Go**
App enterprise	**Java + Spring** o **.NET**

Quando vedi un linguaggio in un progetto, pensa subito al contesto

Quando apri un repository e vedi:

- `package.json` → JavaScript / TypeScript / Node
- `composer.json` → PHP
- `requirements.txt` o `pyproject.toml` → Python
- `Gemfile` → Ruby
- `go.mod` → Go
- `pom.xml` o `build.gradle` → Java
- `*.csproj` → .NET / C#

Sono come la **carta d'identità** del progetto. In un secondo capisci con che mondo hai a che fare.

Cosa ti porti via

1. Sul server può girare **qualsiasi linguaggio**. I più diffusi nel web: **Node.js, PHP, Python, Ruby, Go, Java, .NET**.
2. **Node.js** è il linguaggio del momento. È JavaScript che gira sul server. Vince per Next.js.
3. **PHP** è ancora ovunque (il 40% di Internet). WordPress lo tiene in vita.
4. **Python** è lo standard per AI / ML / data science. Se il tuo progetto tocca quei temi, sarà Python.
5. Quando apri un repo, **i file di dipendenze** ti dicono subito che linguaggio è.

2.4 — Quando un linguaggio sta su entrambi (full-stack)

La rivoluzione di JavaScript

JavaScript ha una caratteristica unica: **gira sia nel browser che sul server**. Tutti gli altri linguaggi che abbiamo visto girano solo da una parte (PHP solo server, Python quasi solo server, ecc.).

Questo crea un'opportunità potente per chi sviluppa: **scrivere lo stesso codice una volta, usarlo in due posti**.

Esempio concreto: validazione email

Vuoi che quando l'utente digita la sua email, il form gli dia subito un feedback («formato non valido») senza aspettare il submit. Allo stesso tempo, sul server vuoi essere sicuro che la mail sia valida prima di salvarla nel database (perché l'utente potrebbe ingannare la validazione client).

In un progetto **non full-stack** (frontend in React, backend in PHP), dovresti scrivere:

- Una funzione `validateEmail()` in JavaScript per il frontend
- Una funzione `validate_email()` in PHP per il backend
- **Due funzioni che fanno la stessa cosa**, in linguaggi diversi, da mantenere allineate

In un progetto **full-stack JavaScript** (Next.js, Remix, ecc.):

```
// validation.ts — UNA funzione
export function validateEmail(email: string): boolean {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}
```

E poi:

```
// app/components/Form.tsx — usata nel CLIENT
import { validateEmail } from '@/lib/validation';

const handleChange = (e) => {
  if (!validateEmail(e.target.value)) {
    setError('Email non valida');
  }
};

// app/api/users/route.ts — usata nel SERVER
import { validateEmail } from '@/lib/validation';

export async function POST(request) {
  const { email } = await request.json();
  if (!validateEmail(email)) {
    return Response.json({ error: 'Email non valida' },
```

```
{ status: 400 });
  }
  // ...salva nel DB
}
```

Stesso codice, due posti. Una sola fonte di verità. Bug fixato una volta, fixato in entrambi.

«use client» e «use server» in Next.js

Next.js (e simili) ha portato questo concetto al massimo. Nello stesso file di un componente puoi avere codice che gira:

- **Sul server** (componenti server, default)
- **Nel browser** (componenti client, segnati con `'use client'`)

```
// Questo gira sul server (default)
async function ProductPage({ id }) {
  const product = await fetchFromDatabase(id); // codice
server-only
  return <ProductDetail product={product} />;
}

'use client';
// Questo gira nel browser
function AddToCartButton({ productId }) {
  const [adding, setAdding] = useState(false); // codice
client-only
  return <button onClick={() => setAdding(true)}>Aggiungi</
button>;
}
```

È una distinzione importante. Tu come vibecoder devi sapere che esiste, anche se Claude la gestisce per te. Quando vedi `'use client'` in cima a un file, significa: «questo pezzo girerà nel browser». Senza, gira sul server.

Quando scegliere full-stack JS

Pro:

- Riusi codice tra client e server

- Un solo linguaggio da imparare
- Strumenti unificati (un solo `package.json`, una sola build)
- Deploy semplificato (es. Vercel)

Contro:

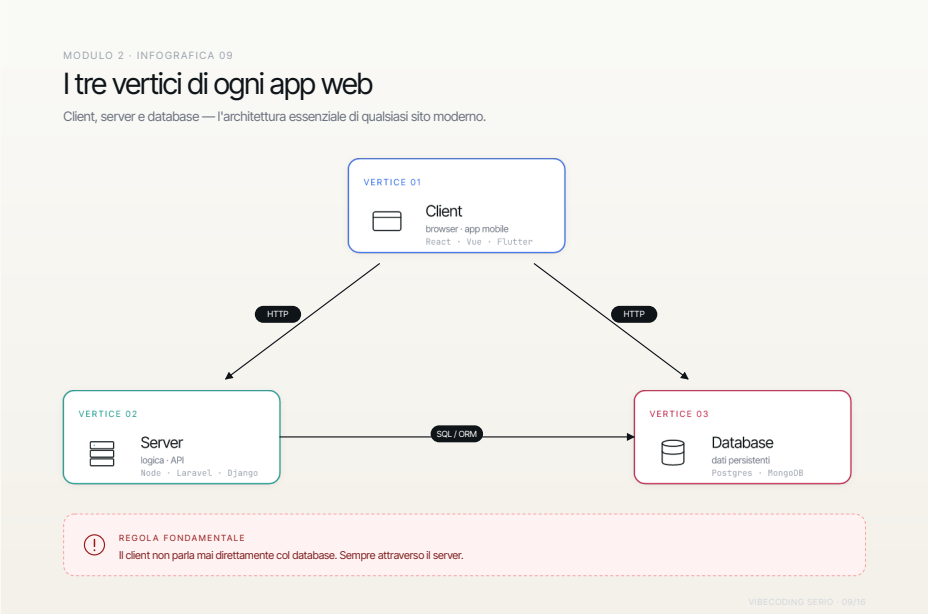
- Se hai bisogno di librerie scientifiche (ML, AI), Python è meglio
- Se hai team con expertise PHP, ha senso restare in PHP
- Se hai un sito enterprise legacy, non riscrivi tutto

In sintesi: **se parti da zero un progetto web moderno, Next.js full-stack è la scelta più probabile** che Claude ti proporrà — proprio perché unifica i due mondi.

Cosa ti porti via

1. **JavaScript è speciale**: gira in entrambi (client + server) grazie a Node.js.
2. Questo permette progetti **full-stack JavaScript** dove il codice è condiviso. Il framework principe è **Next.js**.
3. Quando vedi `'use client'` in cima a un file Next.js, significa «questo gira nel browser». Senza, gira sul server.
4. Per progetti web moderni partiti da zero, è la scelta più probabile di Claude.

2.5 — Il database: il terzo pilastro



Infografica 09 — Triangolo client-server-database

Né client, né server: una cosa a parte

Fin qui abbiamo parlato di due entità: client e server. Ma manca la terza, fondamentale: il **database**.

Il database è dove vivono i **dati persistenti** del tuo sito. «Persistenti» significa: che restano lì anche quando spegni il server, anche quando il browser si chiude. Gli utenti, i prodotti, gli ordini, le foto, i commenti — tutto ciò che deve durare nel tempo sta nel database.

Le tre entità in un triangolo

Ogni applicazione web ha **tre vertici**:

```
              CLIENT
              (browser, app mobile)
                /  \
               /    \
              /      \
        HTTP/         \HTTP
           /           \
          /             \
   SERVER ————————— DATABASE
   (logica, API)        SQL/ORM
```

- **Client** parla col **server** via HTTP (la conversazione che abbiamo visto in 2.1)
- **Server** parla col **database** via SQL o ORM
- **Client e database NON parlano direttamente**. Mai.

Perché client e database non parlano direttamente

Lo abbiamo già detto in 2.1: **sicurezza**. Se il browser potesse parlare col DB, qualunque utente con un po' di malizia potrebbe leggere/modificare i dati di tutti.

> **ATTENZIONE**
>
> **Eccezione importante**: Supabase e Firebase hanno un meccanismo chiamato **Row Level Security** (RLS) che permette al client di parlare col database **direttamente, ma con regole di sicurezza definite a livello DB**. È una soluzione moderna ma rischiosa per chi non sa cosa sta facendo. Quando Claude usa Supabase per un'app, devi sapere che le RLS sono spesso quello che separa «tutti vedono tutto» da «ognuno vede solo i propri dati».

Come parla il server col database: SQL

Il linguaggio universale dei database **relazionali** (i più diffusi: Postgres, MySQL, SQLite) è **SQL** (Structured Query Language). 4 verbi base:

```
SELECT * FROM users WHERE city = 'Como';     -- leggi
INSERT INTO users (name, city) VALUES ('Marco', 'Como'); --
crea
UPDATE users SET city = 'Milano' WHERE id = 1;          --
modifica
DELETE FROM users WHERE id = 1;                         --
cancella
```

Il libro non ti insegna SQL a fondo (è nel Modulo 6). Per ora ti basta sapere che il server, ogni volta che il client gli chiede dei dati, **traduce la richiesta in SQL** e parla col database.

ORM — il traduttore tra codice e SQL

Scrivere SQL a mano è scomodo e rischioso (rischio SQL injection). I moderni framework usano un **ORM** (Object-Relational Mapper): una libreria che ti permette di scrivere query in modo «naturale» nel tuo linguaggio, e l'ORM le traduce in SQL.

Esempio in TypeScript con **Drizzle** (l'ORM moderno più popolare per Node.js):

```
// Tu scrivi questo (TypeScript):
const userList = await db.select()
  .from(users)
  .where(eq(users.city, 'Como'));

// Drizzle traduce in SQL:
// SELECT * FROM users WHERE city = 'Como';
```

ORM popolari per ogni linguaggio:

- **Node.js / TypeScript** → Drizzle, Prisma, Sequelize
- **PHP** → Eloquent (Laravel)
- **Python** → SQLAlchemy (Flask), Django ORM
- **Ruby** → ActiveRecord (Rails)

Quando Claude scrive un endpoint backend, quasi sempre usa un ORM. Tu come vibecoder devi sapere che esiste e cosa fa: **traduce il tuo codice in query SQL al database**.

SQL vs NoSQL

Ci sono due famiglie di database:

Database SQL / relazionali (PostgreSQL, MySQL, SQLite, Supabase):
- Dati strutturati in tabelle, righe, colonne
- Schema rigido (sai esattamente che dati ci sono dentro)
- Eccellenti per dati relazionati (utente → ordini → prodotti)
- Sono la scelta default per il 90% dei progetti

Database NoSQL / documentali (MongoDB, Firestore, DynamoDB):
- Dati come «documenti» simili a JSON
- Schema flessibile (ogni documento può avere campi diversi)
- Buoni per dati molto eterogenei o per scaling massiccio
- Più rischiosi senza disciplina

> **SUGGERIMENTO**
>
> **Per il vibecoder che parte oggi**: punta su **SQL (Postgres su Supabase, Neon, o Aruba VPS)**. È la scelta che genera meno problemi futuri. NoSQL ha ancora senso in casi molto specifici — se Claude ti propone Firestore senza un motivo forte, valuta se non è meglio Supabase.

Cosa ti porti via

1. Il **database** è il terzo vertice. Ci vivono i dati persistenti (utenti, prodotti, ordini).
2. **Solo il server parla col database**. Il client mai (con eccezioni di Supabase/Firebase + RLS, ma è materia avanzata).
3. Si parla col DB in **SQL** (per i database relazionali). Il server usa un **ORM** per scrivere SQL in modo più sicuro e leggibile.
4. **SQL Postgres** è la scelta default moderna. NoSQL solo per casi specifici.

Chiusura del Modulo 2

Adesso hai in testa la **struttura mentale** che separa un vibecoder ingenuo da uno serio:
- **Client ⇄ Server ⇄ Database**: i tre vertici di ogni app web
- Quali **linguaggi** vivono dove (HTML/CSS/JS/React lato client; PHP/Python/Node/Go lato server)

- **JavaScript** è speciale: gira in entrambi (full-stack)
- Il **database** parla solo col server, mai col client

Quando Claude ti scrive 200 righe di codice, ora sai:

- Quale parte gira nel browser
- Quale parte gira sul server
- Dove vivono i dati
- Dove cercare quando qualcosa non funziona

Nel **Modulo 3** entriamo nel mondo **visivo**: l'anatomia di una pagina web. Tutti i componenti UI con il nome giusto. Quello che ti serve per dire a Claude «metti un drawer mobile per il menu» invece di «fai un menu che esce».

🎯 Mini-quiz di autovalutazione

1. **Hai un'app dove l'utente compila un form e i dati devono essere salvati. Il browser dell'utente può salvare direttamente nel database?**
2. **Quale linguaggio gira sia lato client che lato server, ed è il motivo per cui esiste Next.js?**
3. **Cosa significa il commento `'use client'` in cima a un file Next.js?**
4. **Apri un nuovo progetto su GitHub e vedi un file `composer.json`. Di che linguaggio è il backend?**
5. **Vero o falso: SQL e NoSQL sono due tipi di linguaggi di programmazione.**

Risposte

1. **No**. Per sicurezza, il client non parla mai direttamente col database. Il flusso è: client → server (via HTTP) → database (via SQL). Il server fa da filtro autorizzato. (Eccezione: Supabase con Row Level Security, ma è una scelta avanzata da fare con piena consapevolezza.)
2. **JavaScript**. Grazie a Node.js, lo stesso linguaggio gira in entrambi. Next.js sfrutta questa caratteristica per far condividere codice tra client e server.
3. «Questo file (componente) gira nel browser, non sul server». Senza la direttiva, in Next.js 14+ i componenti sono «server components» di default. Sapere quale è quale evita molti errori (es. usare API browser-only in un componente server).

4. **PHP**. Il file `composer.json` è il package manager di PHP (come `package.json` per Node.js). Probabilmente il framework è Laravel o Symfony.
5. **Falso**. SQL e NoSQL sono due tipi di **database**, non di linguaggi. SQL è anche un linguaggio (Structured Query Language) usato per parlare ai database relazionali, ma il termine «SQL/NoSQL» si riferisce alla famiglia di database, non al linguaggio del backend.

> **SUGGERIMENTO**
>
> Se hai sbagliato più di 2 risposte, rileggi prima del Modulo 3. Ma onestamente, se sei arrivato fin qui con attenzione, hai assorbito la mappa. Il resto è solo applicazione.

Modulo 2 redatto: 2026-04-25 · Versione 1.0 · 18 pagine · 4400 parole

Modulo 3 — L'anatomia di una pagina web

SUGGERIMENTO

5 capitoli · 22 pagine · 1 ora 30 minuti
Pre-requisiti: Moduli 0, 1, 2 letti. Sai cosa è un client, cosa è un server.

Mettiti comodo (prendi anche carta e penna)

Questo è il **modulo bandiera** del libro. Quello che, se applichi sul serio, cambia il modo in cui parli a Claude da domani mattina.

Il problema che risolve è semplice: oggi quando descrivi un'interfaccia a Claude, usi parole **vaghe**. «Fai un menu che esce». «Aggiungi un blocco con le icone». «Mettici un popup di conferma». Claude interpreta come può. A volte indovina, a volte no. Tu provi 3-4 modifiche, ti arrabbi, peggio.

La soluzione è **conoscere i nomi giusti** dei pezzi che compongono una pagina web. Quando tu dici «un drawer mobile sticky in basso con shortcut alle azioni primarie, dismissibile col gesto swipe down», Claude sa esattamente cosa generare. Una iterazione, non sette.

Questo modulo è in cinque parti. Ogni parte è un **insieme di componenti** raggruppati per funzione. Per ognuno: nome italiano + inglese, dove si usa,

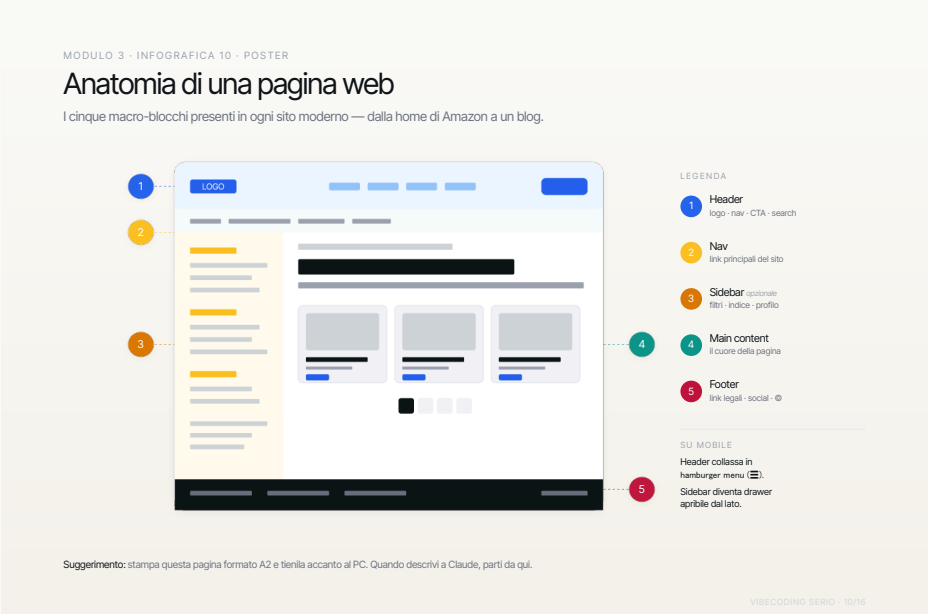
e la frase **«come dirlo a Claude»** — cioè il prompt-pattern già pronto da usare.

1. I 5 macro-blocchi della pagina (header, nav, main, sidebar, footer)
2. Componenti di navigazione (navbar, breadcrumb, tab, stepper, pagination, hamburger)
3. Componenti di contenuto (hero, card, banner, list, grid, carousel, accordion, table)
4. Componenti di interazione (modal, drawer, toast, tooltip, dropdown, form, button)
5. Stati e feedback visivi (empty state, loading, error, skeleton, spinner, hover, focus)

Alla fine avrai un **lessico** di circa 50 termini che ti accompagna per tutto il resto della tua carriera con AI. Ti consiglio di **stamparti** la pagina del glossario alla fine, e tenerla accanto al PC quando lavori.

Andiamo.

3.1 — I 5 macro-blocchi di ogni pagina



Infografica 10 — Anatomia di una pagina (FONDAMENTALE — la spina dorsale visiva del libro

La struttura che ogni pagina condivide

Apri un sito qualsiasi. Amazon, BBC, Wikipedia, il blog del tuo idraulico. Tutti hanno una struttura sotto che — anche se diversa nei dettagli — segue **lo stesso schema** di base.

Cinque blocchi, sempre quelli:

```
┌──────────────────────────────────────────────────┐
│  HEADER  (in cima)                               │
├──────────────────────────────────────────────────┤
│  NAVIGATION  (può essere nel header o separata)  │
├──────────────────────────────────────────────────┤
│          │                                       │
│ SIDEBAR  │   MAIN CONTENT (il cuore della pag.)  │
│ (opz.)   │                                       │
│          │                                       │
├──────────────────────────────────────────────────┤
│  FOOTER  (in fondo)                              │
└──────────────────────────────────────────────────┘
```

Vediamoli uno per uno.

Header — la cima della pagina

L'**header** è la fascia in alto. È quello che vedi prima di tutto. Tipicamente contiene:

- **Logo** (a sinistra)
- **Navigazione principale** (centro o destra)
- **CTA d'accesso** (login, registrati, carrello)
- A volte una **search bar**

L'header è quasi sempre **sticky**, cioè rimane visibile mentre l'utente scrolla. È la zona di «mai persi» della pagina.

> **PROMPT-READY**
>
> **Prompt-ready**: «Header sticky con logo a sinistra, navbar centrale con i 4 link principali, e bottone CTA primario a destra. Su mobile collassa in hamburger menu.»

Navigation (nav) — come ci si muove nel sito

La **navigation** può essere:

- **Dentro l'header** (è il caso più comune oggi)
- Una **sidebar laterale** (più frequente nelle dashboard / app)
- Una **bottom navigation bar** (tipica delle app mobile)

L'elemento HTML semantico è proprio `<nav>`. Le voci dentro sono i link principali del sito.

> **PROMPT-READY**
>
> **Prompt-ready**: «Sidebar nav fissa a sinistra con icone + label, sezioni raggruppate, sezione attiva evidenziata.»

Main content — il cuore della pagina

Il **main content** è il contenuto vero della pagina. L'articolo, la dashboard, il form di checkout. Tutto quello che cambia tra una pagina e l'altra.

L'elemento HTML è `<main>`. Per convenzione, ce n'è uno solo per pagina.

> **PROMPT-READY**
>
> **Prompt-ready**: «Main content centrato max-width 1200px, padding generoso, gerarchia chiara con h1 in cima.»

Sidebar — la colonna laterale (opzionale)

La **sidebar** è una colonna laterale (di solito a destra o sinistra). Contiene:

- **Filtri** (tipico negli e-commerce)
- **Indice del contenuto** (tipico nei blog tecnici)
- **Box pubblicitari** (siti vecchi)
- **Profilo utente / link rapidi** (dashboard)

Su mobile la sidebar **scompare** (o si trasforma in drawer). Concetto chiave: una sidebar non è mai obbligatoria, è un complemento.

> **PROMPT-READY**
>
> **Prompt-ready**: «Sidebar destra 280px sticky, con filtri categoria e prezzo. Su mobile diventa drawer apribile dal bottone filtri.»

Footer — la fine della pagina

Il **footer** è la fascia in fondo. Tipicamente contiene:

- **Link legali** (privacy, termini, cookie)

- **Contatti** e indirizzo
- **Social links**
- **Newsletter signup**
- **Copyright**

> **PROMPT-READY**
>
> **Prompt-ready**: «Footer con 3 colonne: contatti, link rapidi, newsletter. Riga sotto con copyright e link ai social.»

Esempio reale: Amazon

Per fissare i concetti, smontiamo Amazon:

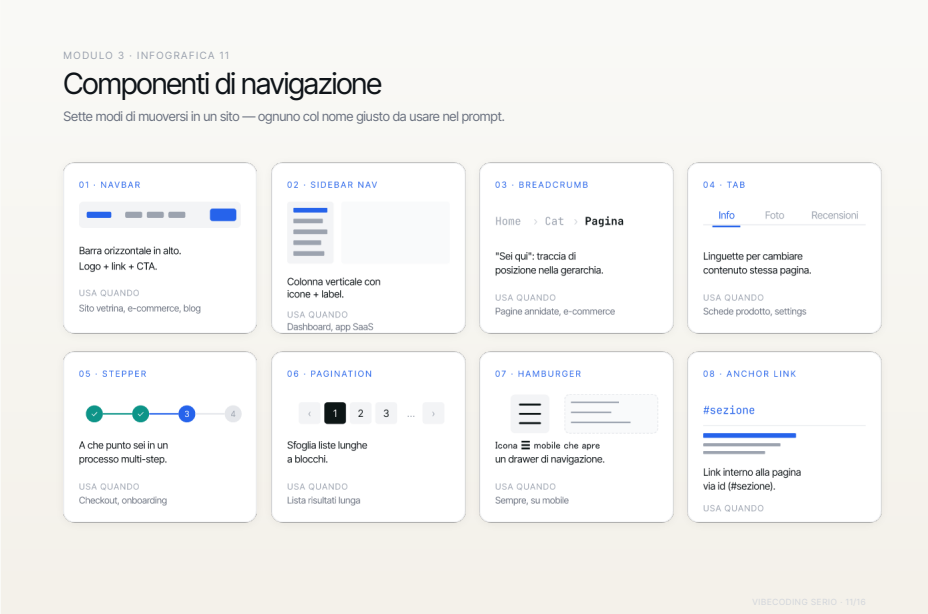
- **Header**: logo Amazon, search bar centrale, bandierina lingua, account, ordini, carrello → tutto sticky
- **Nav**: barra orizzontale sotto l'header con «Tutte le categorie», «Vendere», «Customer Service»
- **Sidebar** (in pagine prodotto): filtri (marca, prezzo, valutazione)
- **Main**: lista prodotti, scheda prodotto, recensioni
- **Footer**: link rapidi (Lavora con noi, Aiuto, Termini, Privacy) + paese/lingua

Ogni e-commerce, ogni dashboard, ogni blog: stessa struttura, dettagli diversi.

Cosa ti porti via

1. Ogni pagina web ha **5 macro-blocchi**: header, navigation, main, sidebar (opzionale), footer.
2. Sono presenti **quasi sempre**, anche se l'aspetto cambia.
3. Quando descrivi una pagina a Claude, parti da questi 5: dichiari ciascuno e specifichi cosa contiene.
4. **Suggerimento da stampa**: l'infografica 10 vale da sola lo sforzo. La trovi a fine modulo formato poster.

3.2 — Componenti di navigazione



Infografica 11 — Componenti di navigazione

Adesso scendiamo nei dettagli. Sette componenti per la navigazione. Vediamoli.

Navbar — la barra orizzontale in alto

La **navbar** è la barra di navigazione che vedi nell'header. Lista di link orizzontali. Su mobile collassa in un hamburger.

> **PROMPT-READY**
>
> **«Navbar orizzontale con 5 link, dropdown "Servizi" a tendina, CTA "Prova gratis" come bottone primario all'estrema destra.»**

Sidebar nav — barra laterale verticale

Una **sidebar di navigazione** è verticale. Tipica delle dashboard. Si trova nelle app come Notion, Slack, Linear, Vercel.

Caratteristiche:
- Icone + label per ogni voce
- Sezioni raggruppate (es. «Lavoro» / «Personale»)
- Voce attiva evidenziata
- Spesso **collassabile** (bottone per stringere e mostrare solo le icone)

> **PROMPT-READY**
>
> **«Sidebar nav 240px collassabile, sezioni raggruppate, icona + label per voce, voce attiva con sfondo accentato.»**

Breadcrumb — la traccia «sei qui»

Il **breadcrumb** è quella riga che vedi tipo: «Home > Categoria > Sottocategoria > Pagina attuale». Aiuta l'utente a capire dove si trova nella gerarchia del sito, e a tornare indietro velocemente.

L'origine del nome: «breadcrumb» in inglese sono le briciole di pane, come Hansel e Gretel.

> **PROMPT-READY**
>
> **«Breadcrumb in cima al main content, separatore ">" tra livelli, ultimo livello non cliccabile.»**

Tab — linguette per cambiare contenuto

I **tab** sono linguette che ti permettono di cambiare contenuto senza ricaricare la pagina. Ne vedi sempre nelle pagine prodotto (Descrizione / Caratteristiche / Recensioni).

> **PROMPT-READY**
>
> **«Tabs orizzontali con 3 sezioni — Info, Foto, Recensioni — bordo sottostante per il tab attivo.»**

Stepper — indicatore di avanzamento

Lo **stepper** mostra a che punto sei in un processo multi-step. Tipico dei checkout (carrello → spedizione → pagamento → conferma) o di onboarding lunghi.

> **PROMPT-READY**
>
> **«Stepper a 4 step orizzontale, step completati con check verde, step attivo con sfondo accentato, step futuri grigi.»**

Pagination — navigazione pagine

La **pagination** è la classica `« 1 2 3 ... 10 »` che vedi sotto le liste lunghe. Permette di sfogliare risultati a blocchi.

> **PROMPT-READY**
>
> **«Pagination sotto la lista prodotti, 12 per pagina, frecce avanti/indietro + numeri, pagina attiva con sfondo nero.»**

Hamburger menu — l'icona delle 3 linee mobile

L'**hamburger** è l'icona ☰ (tre linee orizzontali) che si vede nelle pagine mobile. Cliccandola, si apre un menu (di solito un **drawer** che entra dal lato).

Si chiama «hamburger» perché ricorda un panino visto di lato (le 3 linee = pane / hamburger / pane).

PROMPT-READY

«Hamburger menu icon a destra, al click apre un drawer dal lato sinistro con i link di navigazione.»

Cosa ti porti via

1. Sette componenti di navigazione: **navbar, sidebar nav, breadcrumb, tab, stepper, pagination, hamburger menu**.
2. Ognuno ha un suo **uso tipico**. Non sono intercambiabili.
3. Conoscerli ti permette di **descriverli con precisione** invece di «una cosa che permette di...».

3.3 — Componenti di contenuto

Infografica 12 — Componenti di contenuto

I **componenti di contenuto** sono i «blocchi» che riempiono il main content. Sono i mattoni della pagina.

Hero — la prima impressione

L'**hero** è il blocco grande in alto, sopra la «fold» (la prima schermata visibile senza scrollare). Tipicamente contiene:

- Un **titolo grosso** (h1)
- Un **sottotitolo** descrittivo
- Una o due **CTA** (call to action — bottoni)
- Un'**immagine di sfondo** o illustrazione

L'hero è la **prima impressione** del sito. Investirci tempo è giustificato.

> **PROMPT-READY**
>
> **«Hero a piena larghezza con titolo h1 di 4 righe, sottotitolo, due CTA (primaria + secondaria), immagine illustrativa a destra.»**

Card — riquadro autocontenuto

Una **card** è un riquadro con dentro un'unità di contenuto: un prodotto, un articolo, un profilo. Tipicamente ha:

- Un'**immagine** o icona in alto
- Un **titolo**
- Una **descrizione breve**
- Una **CTA** (bottone, link, prezzo)

> **PROMPT-READY**
>
> **«Card prodotto: immagine 4:3, titolo 1-2 righe, prezzo grosso, badge promo se in offerta, bottone "Aggiungi al carrello".»**

Banner — barra di avviso

Il **banner** è una barra orizzontale stretta che notifica qualcosa: una promozione, un avviso cookie, un messaggio importante. Si vede tipicamente in cima alla pagina o sopra un blocco.

> **PROMPT-READY**
>
> **«Banner di avviso giallo in cima alla pagina, dismissibile con X, testo "Spedizione gratuita per ordini sopra 50€".»**

List — lista verticale di elementi

Una **list** è la classica lista uno sotto l'altro. Email in Gmail, conversazioni in WhatsApp, todo in un to-do app.

> **PROMPT-READY**
>
> **«Lista email con avatar mittente, oggetto in bold, anteprima testo, timestamp a destra. Riga selezionata sfondo accentato.»**

Grid — griglia di card

Un **grid** è una griglia di elementi (di solito card). 2 colonne, 3, 4 — dipende dallo spazio. Su mobile collassa in single column.

> **PROMPT-READY**
>
> **«Grid prodotti: 4 colonne desktop, 2 tablet, 1 mobile. Gap di 24px tra le card.»**

Carousel — slider di immagini scorrevoli

Il **carousel** (o slider) è un blocco che mostra immagini/elementi scorrevoli con frecce o gesto swipe. Tipico delle homepage con «promozioni» o «best seller».

> **SUGGERIMENTO**
>
> Attenzione: i carousel sono **odiati dagli UX designer** (gli utenti vedono solo la prima slide). Se Claude te ne propone uno, valuta se non sia meglio una grid statica.

> **PROMPT-READY**
>
> **«Carousel con 5 slide, autoplay 4 secondi, frecce di navigazione + dot indicator, pause on hover.»**

Accordion — sezione espandibile

L'**accordion** è una sezione che si espande/chiude al click. Tipico delle pagine FAQ. Si chiama così perché ricorda la fisarmonica.

> **PROMPT-READY**
>
> **«Accordion FAQ con 6 domande. Click espande la risposta, chiude le altre. Icona + che ruota a × quando aperto.»**

Table — tabella di dati

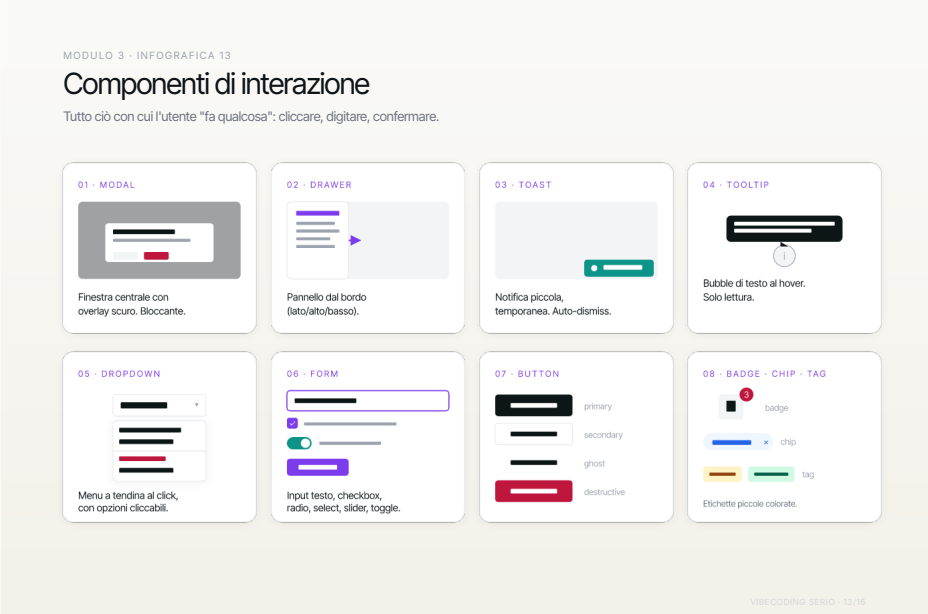
La **table** è la classica tabella con righe e colonne. Tipica delle dashboard, pagine dati, prezziari.

> **PROMPT-READY**
>
> **«Tabella ordini: colonne Data, Cliente, Importo, Stato, Azioni. Header sticky in scroll, righe alternate (zebra), filtro per colonna.»**

Cosa ti porti via

1. Otto componenti di contenuto: **hero, card, banner, list, grid, carousel, accordion, table**.
2. Sono i **mattoni** che riempiono il main content.
3. Sapere quale usare quando ti permette di chiedere a Claude la cosa giusta. «Una grid di card prodotto» è infinite volte meglio di «un blocco con dentro le cose».

3.4 — Componenti di interazione



Infografica 13 — Componenti di interazione

I **componenti di interazione** sono quelli con cui l'utente fa qualcosa. Cliccare, digitare, confermare, aprire e chiudere.

Modal — finestra popup centrale

Un **modal** (o «dialog») è una finestra che appare **sopra** la pagina, con uno sfondo scuro semitrasparente dietro (l'**overlay**). Bloccante: l'utente deve interagirci prima di tornare alla pagina.

Tipici: conferma di una cancellazione, login form, fotografia ingrandita.

> **PROMPT-READY**
>
> **«Modal di conferma per eliminare un cliente: testo "Sei sicuro? Questa azione è irreversibile", bottoni Annulla (ghost) e Elimina (rosso).»**

Drawer — pannello che esce dal lato

Un **drawer** è un pannello che entra **dal bordo** dello schermo (sinistra, destra, alto, basso). Tipico:

- Menu mobile (entra da sinistra)
- Filtri sidebar mobile (entra da destra)
- Carrello rapido (entra da destra)
- Action sheet iOS (entra dal basso)

> **PROMPT-READY**
>
> **«Drawer mobile dal basso con scorciatoie alle azioni primarie, dismissibile col swipe down.»**

Toast / Snackbar — notifica temporanea

Un **toast** (o snackbar — termine Material Design) è una **notifica piccola e temporanea** che appare in un angolo dello schermo, e sparisce dopo qualche secondo.

Tipico: feedback dopo un'azione («Salvato!», «Errore di connessione», «Profilo aggiornato»).

> **PROMPT-READY**
>
> **«Toast in basso a destra dopo il submit. Verde con icona check per success, rosso per error. Auto-dismiss dopo 4 secondi, hover pause.»**

Tooltip — bubble che appare al hover

Il **tooltip** è quel piccolo box di testo che appare quando passi il mouse sopra un elemento. Tipicamente spiega cosa fa un bottone con icona, o dà context aggiuntivo.

> **PROMPT-READY**
>
> **«Tooltip sull'icona info-circle nel form. Al hover mostra "I dati personali sono usati solo per la fatturazione, non vengono condivisi".»**

Dropdown — menu a tendina

Il **dropdown** è il menu che si apre **al click** su un elemento. Diverso dal tooltip (che è solo lettura) — il dropdown ha **opzioni cliccabili**.

Tipico: select del paese, menu utente in alto a destra, «Azioni» su una riga di tabella.

> **PROMPT-READY**
>
> **«Dropdown "Account" nell'header. Al click mostra Profilo, Impostazioni, divisore, Logout (rosso).»**

Form / Input / Checkbox / Radio / Select / Slider / Toggle / Datepicker

I **form** sono il modo principale con cui l'utente inserisce dati. Otto tipi di input principali:

- **Input text**: testo libero (nome, email, password)
- **Textarea**: testo multilinea (commento, descrizione lunga)
- **Checkbox**: opzioni multiple selezionabili (es. «Voglio ricevere offerte»)
- **Radio**: opzioni mutuamente esclusive (es. «Spedizione: standard / express»)
- **Select** (dropdown): scelta da una lista (es. «Paese: Italia, Germania...»)
- **Slider**: valore in un range (es. «Prezzo: 0€ — 200€»)
- **Toggle** (o switch): on/off (es. «Notifiche: attive»)
- **Datepicker**: scelta data dal calendario

> **PROMPT-READY**
>
> **«Form registrazione: input email, input password con toggle show/hide, checkbox "Accetto i termini" (obbligatorio), bottone primary "Crea account".»**

Button — il bottone, cuore dell'interazione

I **button** non sono tutti uguali. Esistono varianti:

- **Primary**: l'azione principale (es. «Acquista»). Sfondo pieno, colore brand.

- **Secondary**: azione secondaria (es. «Annulla»). Sfondo neutro o trasparente con bordo.
- **Ghost**: solo testo, niente bordo (per azioni terziarie)
- **Icon-only**: bottone con solo icona (es. ❤️ «aggiungi ai preferiti»)
- **Destructive**: azione distruttiva (es. «Elimina»). Sfondo o testo rosso.

> **PROMPT-READY**
>
> **«Bottone primario "Conferma ordine" con icona freccia destra. Bottone ghost "Modifica" a fianco.»**

Badge / Chip / Tag — etichette colorate

Badge, **chip** e **tag** sono piccoli elementi colorati che marcano qualcosa:

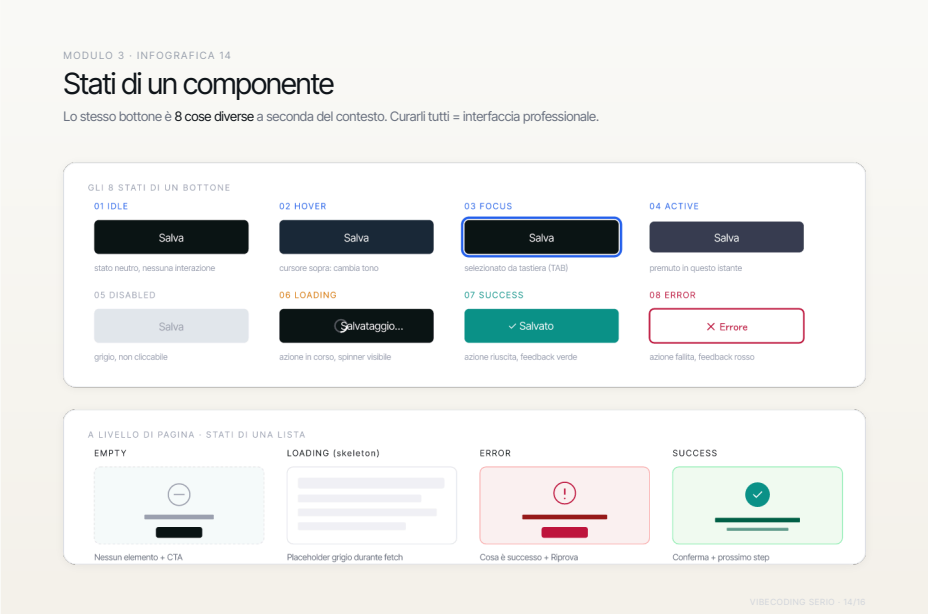
- **Badge**: numero piccolo (es. «3» sull'icona del carrello)
- **Chip**: etichetta interattiva, spesso con X per rimuoverla (es. tag selezionati nei filtri)
- **Tag**: etichetta di categoria (es. «Lavoro», «Famiglia» sui contatti)

> **PROMPT-READY**
>
> **«Chip per ogni filtro selezionato (es. "Marca: Nike"), con X per rimuoverlo. Click su "Cancella tutti" azzera la lista.»**

Cosa ti porti via

1. Tre famiglie di componenti interattivi: **finestre overlay** (modal, drawer, tooltip, dropdown), **input form** (8 tipi), **azioni** (button, badge/chip/tag).
2. Ognuno ha **un nome preciso** che, usato nel prompt, dice a Claude esattamente cosa generare.
3. La differenza tra «modal» e «drawer» sembra sottile ma cambia tutto: modal è centrale e blocca tutta la pagina, drawer entra dal bordo. Confonderli porta Claude a generare la cosa sbagliata.

3.5 — Stati e feedback visivi



Infografica 14 — Stati di un componente

L'ultimo capitolo del modulo, e uno dei più sottovalutati. **Ogni componente non è una cosa sola**: ha **più stati** in cui si può trovare. Un bottone non è «il bottone» — è 8 cose diverse a seconda di cosa sta succedendo.

Capire questo punto separa interfacce **mediocri** (dove ogni stato è gestito a caso) da interfacce **professionali** (dove ogni stato è curato).

Stati di un bottone (8 stati)

Lo stesso bottone può essere in:

1. **Idle / Default** — neutro, nessuna interazione
2. **Hover** — il mouse ci passa sopra
3. **Focus** — è selezionato da tastiera (TAB) o cliccato senza ancora rilasciare
4. **Active** — è premuto in questo istante
5. **Disabled** — grigiato, non cliccabile (es. «Submit» disattivato finché il form non è valido)
6. **Loading** — sta facendo qualcosa (compare uno spinner dentro)

7. **Success** — l'azione è andata a buon fine (icona ✓ verde)
8. **Error** — l'azione è fallita (icona ✕ rossa)

Otto stati. Otto resa visiva diversa. Quando chiedi a Claude «fai un bottone», senza specificare gli stati, lui ne fa uno o due e lascia il resto al caso.

> **PROMPT-READY**
>
> **«Bottone "Salva": stato disabled finché il form non è valido. Stato loading durante l'invio (spinner + testo "Salvataggio..."). Stato success con check verde 2 secondi. Stato error con bordo rosso e toast errore.»**

Stati di un input form

Un input testo ha anche lui i suoi stati:

- **Idle**: vuoto, in attesa
- **Focus**: il cursore è dentro
- **Filled**: ha del testo dentro
- **Error**: bordo rosso + messaggio sotto («Email non valida»)
- **Disabled**: grigiato
- **Read-only**: leggibile ma non modificabile

> **PROMPT-READY**
>
> **«Input email: bordo grigio idle, blu in focus, rosso in error con messaggio sotto, sfondo grigio in disabled.»**

Stati a livello di contenuto (page-level)

Spostiamoci da componenti singoli a **interi blocchi di contenuto**. Una lista può trovarsi in molti stati:

Empty state — lista vuota

Quando la lista non ha elementi (es. nessun prodotto, nessun ordine), **non lasciare uno spazio vuoto**. Mostra un **empty state**: un messaggio chiaro + CTA per fare qualcosa.

Esempio: «Non hai ancora ordini. **Inizia a comprare** →»

PROMPT-READY

«Empty state della lista cantieri quando l'utente non ha ancora cantieri: illustrazione, frase "Crea il tuo primo cantiere", bottone primario "Nuovo cantiere".»

Loading state — durante il caricamento

Quando la pagina sta caricando dati, **non lasciare lo schermo bianco**. Mostra qualcosa che dice «sto lavorando».

Due opzioni principali:

- **Spinner**: un'icona rotante (semplice, comune)
- **Skeleton loader**: dei rettangoli grigi che simulano la forma del contenuto in arrivo (più moderno, più rassicurante)

Lo skeleton loader oggi è preferito: dà all'utente l'impressione che il contenuto stia «comparendo», non «caricando da zero».

PROMPT-READY

«Skeleton loader per la lista prodotti durante il fetch: 6 rettangoli grigi con la forma di una card, animazione di pulsazione.»

Error state — qualcosa è andato storto

Quando una richiesta fallisce, **non lasciare l'utente al buio**. Mostra un **error state** con:

- Cosa è successo (in linguaggio chiaro, non «ERROR_500»)
- Cosa può fare (riprova, contatta supporto, ecc.)

ATTENZIONE

«Error state quando fallisce il fetch: icona warning, frase "Non sono riuscito a caricare i prodotti", bottone "Riprova".»

Disabled / hover / focus a livello pagina

Anche il **layout** può avere stati. Quando un modal è aperto, lo sfondo dietro è «disabled» (overlay scuro, non cliccabile). Quando un elemento ha focus tastiera, c'è un anello visibile attorno (importante per accessibilità).

> **PROMPT-READY**
>
> **«Outline ring blu su tutti gli elementi focusabili (button, input, link). 2px solid, offset 2px, non rimuovere mai per estetica.»**

Cosa ti porti via

1. **Ogni componente ha più stati**. Un bottone è 8 cose diverse.
2. Gli stati a livello di pagina più importanti: **empty state, loading state, error state**. Mai lasciare lo schermo bianco o senza feedback.
3. Lo **skeleton loader** è preferibile allo spinner per loading lunghi (>1 secondo).
4. Quando chiedi a Claude un componente, **descrivi anche i suoi stati**. Da «fai un form» a «fai un form con input idle, focus, error, e bottone con stati disabled e loading».

Chiusura del Modulo 3

Adesso hai un **lessico** di circa 50 termini per parlare di interfacce con precisione. Sono i mattoni che ogni vibecoder deve conoscere.

Dalla prossima volta che apri Claude:

- Non scrivere «un menu che esce». Scrivi **«un drawer mobile dal basso con i link principali»**.
- Non scrivere «una scritta di conferma». Scrivi **«un toast verde in basso a destra con testo "Salvato" che sparisce dopo 4 secondi»**.
- Non scrivere «un blocco con la roba». Scrivi **«una grid di card 3 colonne desktop, 1 colonna mobile, ogni card con immagine 4:3, titolo, prezzo, bottone aggiungi al carrello»**.

Ogni precisione che aggiungi al prompt riduce le iterazioni. È matematica.

Nel **Modulo 4** scendiamo nel tecnico del frontend: HTML, CSS, JavaScript, React, Server Components vs Client Components in Next.js. Ma se questo modulo l'hai assorbito, il resto del libro va in discesa.

🎯 Mini-quiz di autovalutazione

1. **Differenza in una frase tra modal e drawer.**
2. **Hai una pagina con la lista prodotti che inizialmente è vuota perché l'utente è nuovo. Cosa metti al posto della lista vuota?**
3. **Lo «skeleton loader» è preferito allo «spinner» perché...**
4. **Vuoi che l'utente, prima di confermare un'azione distruttiva (eliminare un cantiere), debba dare conferma. Quale componente usi?**
5. **Vero o falso: il termine corretto per la barra orizzontale in alto con logo e link è «navbar».**

Risposte

1. Il **modal** è una finestra che appare al **centro** della pagina con overlay scuro dietro, e blocca l'interazione col resto. Il **drawer** è un pannello che entra **dal bordo** (lato/alto/basso) della pagina, e tipicamente non blocca tutto.
2. Un **empty state**: illustrazione + frase chiara («Non hai ancora prodotti») + CTA per agire («Crea il tuo primo prodotto»).
3. ...è **più moderno e rassicurante**: simula la forma del contenuto che sta arrivando, dando l'impressione di «completamento» piuttosto che «attesa». Ottimo per loading >1 secondo.
4. Un **modal di conferma**: testo chiaro («Sei sicuro?»), bottone Annulla (ghost) e bottone Elimina (destructive, rosso).
5. **Vero**. La barra orizzontale in alto si chiama **navbar** (o navigation bar). Tecnicamente l'header è il blocco contenitore, la navbar è la barra di link dentro.

SUGGERIMENTO

Se sei arrivato fin qui hai assorbito il **lessico operativo** del web. Da qui in poi tutto è applicazione. **Stampa l'infografica 10** e tienila accanto al PC quando lavori — è il riferimento che useai più di tutti.

Modulo 3 redatto: 2026-04-25 · Versione 1.0 · 22 pagine · 5400 parole

Modulo 4 — Il browser è stupido (ma fa tutto lui)

SUGGERIMENTO

5 capitoli · 18 pagine · 1 ora 10 minuti
Pre-requisiti: Modulo 2 letto. Sai distinguere client da server, conosci HTML/CSS/JS come parole.

Mettiti comodo

Adesso scendiamo nel tecnico del **frontend**. Non per farti scrivere codice da zero — ma per farti **riconoscere** quello che Claude scrive, capire perché lo scrive così, e parlare con cognizione di causa.

Il browser è lo strumento più diffuso della storia dell'informatica, e fa molto più di quanto sembri. Ma è anche, in un certo senso, **stupido per design**: non si fida di niente, applica regole rigide, fa il minimo indispensabile da solo. Capirne il funzionamento ti permette di capire perché certe cose nei tuoi siti vanno e altre no.

Cinque capitoli:

1. HTML in 30 minuti — i tag con significato e perché contano
2. CSS responsive senza piangere — flexbox, grid, mobile-first
3. JavaScript: aggiornare vs ricaricare — la differenza tra SPA e MPA
4. Il DOM e React — cosa Claude tocca quando modifica la UI

5. Server Components vs Client Components — il concetto chiave di Next.js 14+

Andiamo.

4.1 — HTML in 30 minuti: i tag con significato

Perché HTML conta più di quanto sembri

L'HTML è il linguaggio più antico del web, e quasi tutti pensano sia banale: «sono solo tag». Sbagliato. **HTML scritto bene** vs **HTML scritto male** è la differenza tra un sito accessibile, performante, indicizzato da Google, e un sito che fatica a essere letto.

Quando Claude ti scrive una pagina, sceglie i tag in modo specifico. Vediamo perché.

Tag con significato vs `<div>` puro

Confronta questi due pezzi di codice. Fanno apparentemente la stessa cosa: una pagina con titolo, contenuto, link, pulsante.

Versione cattiva (full `<div>`):

```
<div>
  <div>Vibecoding Serio</div>
  <div>
    <div>Capitolo 1</div>
    <div>Cosa è il vibecoding...</div>
  </div>
  <div>
    <div onclick="acquista()">Compra</div>
  </div>
</div>
```

Versione buona (tag semantici):

```
<header>
  <h1>Vibecoding Serio</h1>
</header>
<main>
  <article>
    <h2>Capitolo 1</h2>
    <p>Cosa è il vibecoding...</p>
  </article>
  <button onclick="acquista()">Compra</button>
</main>
```

Visivamente identico. Ma **per il browser, per gli screen reader, per Google, per gli script di accessibilità** — completamente diverso.

I tag che contano davvero

Non hai bisogno di sapere tutti i 100 tag HTML. Te ne servono **15** ben usati:

Struttura della pagina (li hai già visti nel Modulo 3):
- `<header>` · `<nav>` · `<main>` · `<aside>` (sidebar) · `<footer>`

Contenuto:
- `<h1>` … `<h6>` — titoli (h1 = il principale, uno solo per pagina)
- `<p>` — paragrafo di testo
- `<article>` — un pezzo di contenuto autonomo (post blog, articolo, prodotto)
- `<section>` — una sezione tematica della pagina
- `<ul>` / `<ol>` / `<li>` — liste non ordinate / ordinate / item

Interazione:
- `<button>` — un bottone cliccabile
- `<a href="...">` — un link
- `<form>` / `<input>` / `<label>` — form, campo di input, etichetta
- `<img src="..." alt="...">` — immagine (l' `alt` è obbligatorio per accessibilità)

L'attributo `alt`: una piccola cosa che vale tanto

Una nota su `<img>`. Quando Claude scrive un'immagine, dovrebbe sempre includere l'attributo `alt`:

```
<img src="cantiere.jpg" alt="Operai al lavoro su una
facciata di palazzo">
```

L' `alt` :
- È letto agli **screen reader** (utenti non vedenti)
- Appare se l'immagine **non si carica**
- È letto da **Google** per indicizzazione

Se Claude te lo dimentica (succede), aggiungilo a mano. Costa zero, vale tanto.

Accessibilità in un paragrafo

L'**accessibilità** (a11y, in gergo) significa che il sito è usabile anche da persone con disabilità (vista, motorie, cognitive). Le best practice base sono tutte gratis:
- Usa tag semantici (`<button>` invece di `<div onclick>`)
- Aggiungi `alt` a tutte le immagini
- Usa `<label>` per gli input (così il click sull'etichetta porta nel campo)
- Garantisci buon contrasto colori (testo nero su sfondo scuro = no)
- Non rimuovere mai l'**outline focus** del browser per estetica (è la riga blu che vedi attorno agli elementi quando navighi con TAB)

Quando chiedi a Claude un componente, **specifica «accessibile» nel prompt**. Lui sa cosa vuol dire e generalmente lo rispetta.

> **PROMPT-READY**
>
> **Prompt-ready**: «Form di registrazione accessibile: ogni input con label associata, aria-label sui bottoni icon-only, focus visibile, errori dichiarati con aria-live.»

Cosa ti porti via

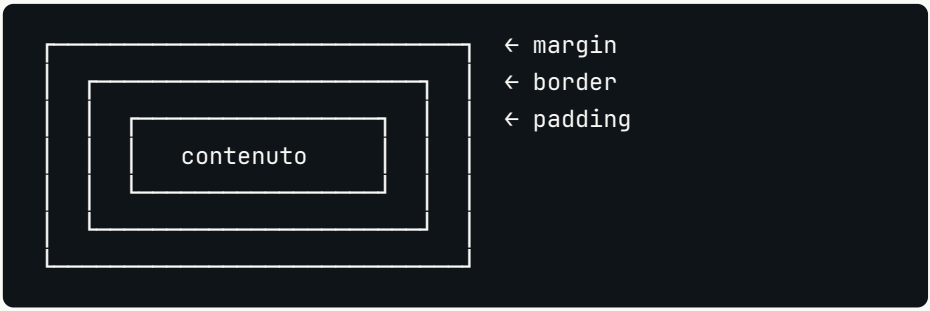
1. **Tag con significato** (`<header>` , `<button>` , `<article>`) sono meglio dei `<div>` puri. Lo capiscono il browser, gli screen reader, Google.
2. Ti bastano **15 tag** per coprire il 95% dei casi.
3. `alt` **sulle immagini** è obbligatorio. Non saltarlo.
4. **Accessibilità** non è un extra: è quello che separa siti professionali da siti amatoriali. Specificalo nei prompt.

4.2 — CSS: layout responsive senza piangere

Il box model in 30 secondi

Ogni elemento HTML è una **scatola rettangolare** composta da 4 strati concentrici, dall'interno verso l'esterno:

1. **Content** — il contenuto vero (testo, immagine)
2. **Padding** — spazio interno tra contenuto e bordo
3. **Border** — il bordo
4. **Margin** — spazio esterno verso gli altri elementi



Quando Claude scrive `padding: 16px` sta dicendo «16 pixel di spazio interno». Quando scrive `margin-top: 24px` sta dicendo «24 pixel di spazio sopra l'elemento».

Conoscere queste 4 parole ti permette di chiedere modifiche precise: «aumenta il padding interno della card a 24px» invece di «fai più aria nel box».

Flexbox vs Grid: la metafora del bancomat

Le due tecnologie principali per fare layout in CSS sono **Flexbox** e **CSS Grid**. Servono entrambe ma a cose diverse.

Flexbox è perfetto per **una fila**. Una riga, una colonna. Pensa al **bancomat con la fila di persone** — tutti incolonnati uno dietro l'altro, possono distribuirsi nello spazio disponibile.

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Risultato: gli elementi della navbar in fila orizzontale, distanziati uniformemente, allineati verticalmente al centro.

CSS Grid è perfetto per **una griglia 2D**. Pensa alla **griglia di sportelli del bancomat** — colonne e righe insieme.

```
.product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 24px;
}
```

Risultato: una griglia di 3 colonne uguali con 24px di gap tra ogni cella.

> **SUGGERIMENTO**
>
> **Regola pratica**: Flexbox per **navbar, footer row, lista pulsanti, single column**. Grid per **layout di pagina, griglie di card, dashboard**.

Se Claude ti propone Grid per una navbar, qualcosa è strano (stai facendo overengineering). Se ti propone Flexbox per un dashboard complesso, magari sta semplificando troppo.

Mobile-first: l'unica strategia che funziona

Mobile-first significa: progetta prima per mobile (schermo piccolo), poi aggiungi regole per schermi più grandi. Non viceversa.

Perché? Tre motivi:

1. La maggior parte degli utenti naviga da mobile (60-80% del traffico)
2. Vincolare lo schermo piccolo prima evita layout che funzionano solo su desktop

3. È più facile **espandere** un layout semplice che **comprimere** un layout complesso

Esempio pratico in CSS, con **media query**:

```
/* Mobile (default) */
.container {
  display: block;
  padding: 16px;
}

/* Tablet (≥768px) */
@media (min-width: 768px) {
  .container {
    display: flex;
    padding: 24px;
  }
}

/* Desktop (≥1024px) */
@media (min-width: 1024px) {
  .container {
    max-width: 1200px;
    margin: 0 auto;
  }
}
```

Tradotto: di default lo stile è mobile (block, 16px padding). A 768px (tablet) attiviamo il flex e aumentiamo il padding. A 1024px (desktop) limitiamo la larghezza.

> **PROMPT-READY**
>
> **Prompt-ready**: «Layout responsive mobile-first: 1 colonna mobile, 2 colonne tablet (≥768px), 4 colonne desktop (≥1024px). Gap di 16px mobile, 24px desktop.»

Tailwind: il modo moderno di scrivere CSS

Quasi tutti i siti che Claude genera oggi usano **Tailwind CSS**. Invece di scrivere blocchi CSS in un file separato, scrivi **classi nel tag HTML**.

CSS classico:

```
<button class="cta">Compra</button>

<style>
.cta {
  background: #2563EB;
  color: white;
  padding: 12px 24px;
  border-radius: 8px;
  font-weight: 600;
}
</style>
```

Tailwind:

```
<button class="bg-blue-600 text-white px-6 py-3 rounded-lg
font-semibold">
  Compra
</button>
```

Stesso risultato. Tailwind è **più verboso nell'HTML** ma elimina il file CSS separato e lo «switching cognitivo» (devi tenere a mente solo l'HTML, non due file).

Cosa ti porti via

1. **Box model** in 4 parole: content / padding / border / margin.
2. **Flexbox** per fila singola, **Grid** per griglie 2D. Flexbox per navbar, Grid per layout pagina.
3. **Mobile-first**: scrivi CSS prima per mobile, poi aggiungi regole per schermi più grandi con `@media`.
4. **Tailwind**: classi inline nell'HTML al posto di file CSS separati. Standard moderno.

4.3 — JavaScript: la pagina si aggiorna o si ricarica?

Due paradigmi: MPA e SPA

Quando un utente clicca un link in un sito, succede una di due cose:

MPA — Multi-Page Application (scuola classica)

Click sul link → il browser fa una **nuova richiesta HTTP completa** → riceve una nuova pagina HTML → la rendera da zero. Sito tipico: WordPress, e-commerce classici, blog.

Pro: semplice, robusto, SEO facile. Contro: a ogni navigazione la pagina si ricarica completamente (vedi flash bianco).

SPA — Single-Page Application (scuola moderna)

Click sul link → JavaScript intercetta il click → fa una richiesta solo per i **nuovi dati** (non per HTML completo) → aggiorna **solo il pezzo cambiato** della pagina. La URL nella barra cambia, ma il browser non ricarica. Sito tipico: Gmail, Notion, Linear, Twitter.

Pro: sensazione «app», veloce, fluido. Contro: più complesso, SEO richiede attenzione, usa più JavaScript.

«Aggiornare» vs «Ricaricare»: parole importanti

Quando spieghi a Claude cosa vuoi, queste due parole sono diverse:

- «Aggiorna i prodotti quando l'utente cambia filtro» → **JS modifica il DOM** senza ricarica. Tipico SPA.
- «Ricarica la pagina dopo il salvataggio» → **navigazione completa**, full reload. Tipico MPA, o caso particolare in SPA.

Sapere il termine giusto ti permette di descrivere il comportamento in 5 parole.

Next.js fa entrambi

Ecco la cosa interessante: **Next.js**, e i meta-framework moderni, **fanno entrambe le cose** a seconda del caso. Sono **ibridi**. La prima visita a una

pagina è server-rendered (come MPA, buon SEO). Le navigazioni successive interne al sito sono client-side (come SPA, sensazione veloce).

Quando vedi la «tipica app moderna», quasi sempre è questo: **best of both worlds**.

Il flash bianco: il sintomo del MPA puro

Se sviluppi un sito Next.js / React / Astro e vedi un **flash bianco** tra una pagina e l'altra, qualcosa non va. Probabilmente:

- Stai usando link `<a href="...">` «puri» invece dei componenti `<Link>` di Next.js
- O il framework non sta gestendo bene la navigazione

> **PROMPT-READY**
>
> **Prompt-ready**: «Navigazione tra pagine senza flash bianco: usare i componenti `<Link>` di Next.js, non i tag `<a>` standard. Mostra spinner inline nella sezione che cambia, non sostituire l'intera pagina.»

Cosa ti porti via

1. **MPA**: ogni navigazione = ricarica completa della pagina (WordPress).
2. **SPA**: JS aggiorna pezzi senza ricarica (Gmail).
3. **Next.js è ibrido**: server-rendered alla prima visita, SPA-like dopo.
4. Le parole «aggiorna» e «ricarica» hanno significati diversi nel parlare con Claude. Usa la giusta.

4.4 — Il DOM e React: dove Claude mette le mani

Il DOM è la pagina «viva»

Quando il browser parsa l'HTML che gli arriva, costruisce in memoria una **rappresentazione viva** della pagina chiamata **DOM** (Document Object Model). Pensa al DOM come a un albero: ogni tag HTML è un nodo, ogni nodo può avere figli.

```
<html>
├── <head>
│   ├── <title>
│   └── <meta>
└── <body>
    ├── <header>
    │   └── <h1>
    └── <main>
        └── <p>
```

Il DOM è **modificabile** in tempo reale. Quando JavaScript dice «aggiungi una classe a questo elemento» o «crea un nuovo paragrafo qui», sta modificando il DOM. Il browser **immediatamente** aggiorna la visualizzazione.

```
// Modifica diretta del DOM
document.getElementById('contatore').textContent = '5';
```

React: il modo moderno di parlare al DOM

Modificare il DOM «a mano» (con `document.getElementById`, `addEventListener`, ecc.) funziona ma diventa **infernale** quando l'app cresce. Per questo è nato React (e simili).

L'idea di React: **non modifichi il DOM direttamente**. Dichiari **com'è la UI in funzione dei dati**, e React si occupa di aggiornare il DOM quando i dati cambiano.

Un esempio:

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Hai cliccato {count} volte
    </button>
  );
}
```

Tradotto: «Un componente Counter ha un valore `count` che parte da 0. Mostra un bottone con scritto “Hai cliccato N volte”. Al click, incrementa `count`.»

Tu non scrivi mai «trova il bottone, cambia il testo a N+1». Dichiari come **deve apparire** in funzione del valore, e React fa il lavoro sporco.

Componenti, props, state

Tre concetti chiave di React. Te li spiego con un esempio della vita di tutti i giorni.

Componente: un pezzo di UI riusabile. Il bottone «Compra» che vedi in 100 punti del sito = un solo componente, riusato 100 volte.

```
function Button({ label, onClick }) {
  return <button onClick={onClick}>{label}</button>;
}
```

Props: i dati che passi al componente. Sono come **parametri di una funzione**.

```
<Button label="Compra" onClick={handleBuy} />
<Button label="Annulla" onClick={handleCancel} />
```

Stesso componente, props diverse, comportamento diverso.

State: i dati interni al componente, che cambiano nel tempo. Esempio: il valore del contatore, l'input del form, lo stato «modal aperto/chiuso».

```
const [isOpen, setIsOpen] = useState(false);
```

`isOpen` è uno stato. `setIsOpen` è la funzione per cambiarlo. Quando lo cambi, React **ri-renderizza** il componente.

Quando Claude scrive React, sa fare queste 3 cose

Tu, come vibecoder, devi **riconoscere** quando Claude scrive:

- Un **componente** (funzione che inizia con maiuscola: `Button`, `Card`, `ProductList`)
- **Props** (parametri della funzione: `function Card({ title, price })`)
- **State** (chiamate a `useState`: `const [x, setX] = useState(...)`)

Sono i tre pezzi più frequenti del codice React. Se questi tre li vedi, sei a posto. Il resto è dettaglio.

Cosa ti porti via

1. Il **DOM** è la rappresentazione viva della pagina, modificabile in tempo reale.
2. **React** ti permette di **descrivere come la UI deve essere** in funzione dei dati, invece di modificare il DOM a mano.
3. **Tre concetti chiave** in React: componenti (UI riusabili), props (parametri), state (dati interni mutabili).
4. Quando leggi codice React di Claude, **cerca i tre**: nome con maiuscola, parametri, `useState`.

4.5 — Server Components vs Client Components (Next.js 14+)

La cosa che il vibecoder Next.js incontra senza capire

Apri un progetto Next.js 14+. Trovi due tipi di file di componenti:

```
// Componente di default (Server Component)
export default function ProductPage({ id }) {
  // Qui posso fare query al database direttamente!
  const product = await db.product.findById(id);
  return <ProductDetail product={product} />;
}
```

```
// Componente con 'use client' (Client Component)
'use client';
import { useState } from 'react';

export default function AddToCartButton({ productId }) {
  const [adding, setAdding] = useState(false);
  return <button onClick={() => setAdding(true)}>Aggiungi</
button>;
}
```

La differenza? Il primo gira **sul server**, il secondo nel **browser**. Decide tutto la riga `'use client'` in cima al file.

Cosa cambia tra i due

Server Component (default — senza `'use client'`):

- Gira **una volta**, sul server, prima che la pagina arrivi al browser
- **Può accedere al database**, file system, API key segrete
- **Non può** usare interazioni utente (`onClick` , `useState` , `useEffect`)
- HTML risultante è già pronto, niente JS frontend → pagina più leggera

Client Component (con `'use client'`):

- Gira **nel browser** dopo il caricamento
- **Può usare** `useState` , `useEffect` , `onClick` , accesso al DOM
- **Non può** accedere a database o segreti (li vedrebbero gli utenti)
- Richiede JavaScript scaricato dal browser

Regola pratica per il vibecoder

Senza scrivere codice, ti basta sapere questo: **quando vedi `'use client'` in un file, quel file gira nel browser**. Senza, gira sul server.

Perché ti importa? Perché quando Claude ti propone una soluzione, devi capire dove gira il codice. Esempi:

- «Validazione del form lato client» → Client Component
- «Fetch dei prodotti dal database» → Server Component
- «Bottone con stato di loading durante il submit» → Client Component (perché `useState`)
- «Pagina pubblica indicizzata da Google» → Server Component (perché HTML già pronto)

Errori comuni del vibecoder che non sa la differenza

Errore 1: provare a usare `useState` in un Server Component.

Sintomo: Next.js dà errore «useState can only be used in Client Components». Fix: aggiungi `'use client'` in cima al file.

Errore 2: mettere `'use client'` ovunque «per sicurezza».

Sintomo: il sito carica più JavaScript del necessario, perdi i benefici del rendering server. Niente errore visibile, ma performance peggiori. Fix: usa `'use client'` solo nei componenti che ne hanno davvero bisogno (interazione, stato).

Errore 3: chiamare un'API key direttamente in un Client Component.

Sintomo: l'API key è visibile nel codice del browser (chi ispeziona la pagina la vede in chiaro). Disastro di sicurezza. Fix: tieni le chiamate con API key in Server Components, esponi solo i risultati al client.

Come parlarne a Claude

Quando descrivi un componente, **dichiara dove deve girare**:

> **PROMPT-READY**
>
> **«Pagina prodotto: Server Component che fetcha dal DB. Pulsante "Aggiungi al carrello" come Client Component separato (perché ha state). Form di recensione separato come Client Component.»**

Questa è la differenza tra un prompt che genera architettura sensata e uno che genera codice confuso.

Cosa ti porti via

1. In Next.js 14+, esistono due tipi di componente: **Server Component** (default) e **Client Component** (con `'use client'` in cima).
2. **Server Component**: gira sul server, accesso al DB, niente interazione utente.
3. **Client Component**: gira nel browser, accesso a `useState` / `useEffect`, niente segreti.
4. **Regola pratica**: usa `'use client'` solo dove serve davvero (interazione, stato), non ovunque.

Chiusura del Modulo 4

Adesso il **frontend non è più una scatola nera**. Sai:

- Cosa significa scegliere un tag HTML «con significato» invece di un `<div>` puro
- Come fare layout responsive con Flexbox e Grid, mobile-first
- La differenza tra MPA, SPA, e l'ibrido di Next.js
- Cosa è il DOM, cosa fa React, e i tre concetti chiave (componenti, props, state)
- Quando un componente Next.js gira sul server e quando nel browser

Quando Claude ti scrive 200 righe di codice frontend, non ti spaventi più. Ne riconosci l'architettura.

Nel **Modulo 5** ci spostiamo dall'altra parte: il backend. API REST, codici di stato HTTP, headers, JSON. Tutto quello che il frontend vede del server.

🎯 Mini-quiz di autovalutazione

1. **Vero o falso: usare `<button>` invece di `<div onClick>` cambia solo questione di stile.**
2. **Vuoi una griglia di 4 card prodotto su desktop e 1 colonna su mobile. Quale tecnologia CSS usi: Flexbox o Grid?**
3. **Cosa fa la riga `'use client'` in cima a un file Next.js?**
4. **In React, qual è la differenza tra «props» e «state»?**

5. **Hai un componente che fa fetch al database e mostra una lista. Vedi un errore Next.js «useState can only be used in Client Components». Cosa cambia?**

Risposte

1. **Falso**. Usare `<button>` invece di `<div onClick>` cambia: lo screen reader lo legge come bottone, è raggiungibile da tastiera (TAB), Google capisce che è interattivo, e il browser ti dà gratuitamente focus visivo. Tutto questo lo perdi col `<div>`.
2. **Grid**. Flexbox è perfetto per una fila singola; per layout 2D (colonne × righe), Grid è la scelta giusta. `grid-template-columns: repeat(4, 1fr)` su desktop, `grid-template-columns: 1fr` su mobile.
3. Dice a Next.js: **questo file è un Client Component**, deve essere bundlato e mandato al browser. Senza, sarebbe un Server Component (gira sul server).
4. **Props** sono dati che il componente **riceve dall'esterno** (immutabili dentro al componente). **State** sono dati **interni mutabili** (cambiano nel tempo, es. con `useState`). Props vengono «calate dall'alto», lo state nasce e vive dentro il componente.
5. Probabilmente stai cercando di usare `useState` (o un hook) in un Server Component. Aggiungi `'use client'` in cima al file. **Ma valuta**: se è un componente che fa fetch al DB, magari meglio **separare** la parte «fetch» (Server Component) dalla parte «interazione utente» (Client Component figlio), invece di rendere tutto Client.

> **SUGGERIMENTO**
>
> Hai assorbito il frontend a livello concettuale. Il **Modulo 5** ti porta dall'altra parte (backend) — più tecnico ma con la stessa logica: capire cosa Claude scrive, perché lo scrive così.

Modulo 4 redatto: 2026-04-25 · Versione 1.0 · 18 pagine · 4400 parole

Modulo 5 — Il server è il cameriere che nessuno vede

SUGGERIMENTO

5 capitoli · 20 pagine · 1 ora 15 minuti
Pre-requisiti: Modulo 2 letto. Conosci la divisione client/server.

Mettiti comodo

Il backend è la metà del web che l'utente non vede mai, ma da cui dipende tutto. Quando il tuo sito si «rompe» in un modo che non capisci, il 70% delle volte è qualcosa lato server. Capirlo a livello concettuale ti permette di **diagnosticare**, non più indovinare.

Cinque capitoli:

1. Frontend e backend: perché esistono separati (e quando no)
2. API REST — la metafora del menu del ristorante
3. I verbi HTTP: GET / POST / PUT / PATCH / DELETE
4. I codici di stato: cosa sta gridando il server
5. Headers, body, JSON: il formato di conversazione

5.1 — Frontend e backend: perché esistono separati

La domanda legittima

Se il browser sa già fare tante cose, **perché serve un server?** Non potrebbe il browser parlare direttamente al database, mandare email, processare pagamenti?

La risposta è una sola, ed è importante: **sicurezza**.

Il browser è un ambiente **non fidato**. Chiunque può aprire DevTools, vedere il codice, modificarlo, intercettare le richieste. Se il browser potesse parlare direttamente col database, qualunque utente con un po' di malizia potrebbe leggere i dati di tutti.

Il server è un ambiente **fidato**. Gira su una macchina che controlli tu, dietro a un firewall, con accesso solo via SSH/credenziali. Lì puoi tenere i segreti (API key, credenziali DB) e fare i controlli di sicurezza.

La regola d'oro

> **SUGGERIMENTO**
>
> **Tutto ciò che richiede un segreto o controlla autorizzazioni, deve passare dal server.**

Esempi:

- Verificare se un utente è autenticato → server
- Chiamare l'API di OpenAI con la tua API key → server
- Salvare un ordine nel database → server
- Mandare un'email transazionale → server
- Mostrare la lista dei prodotti pubblici → frontend (può farlo direttamente)
- Validare un form prima del submit → frontend (per UX) **e** server (per sicurezza)

Quando i due si fondono: il full-stack

In Next.js, Remix, e simili, **frontend e backend stanno nello stesso progetto**, nella stessa codebase, magari nello stesso file. Ma anche lì la divisione esiste: ci sono **Server Components / Server Actions** (girano sul server) e **Client Components** (girano nel browser). Lo abbiamo visto nel Modulo 4.

Anche quando «tutto è nello stesso progetto», la regola d'oro vale: i segreti restano lato server. Sempre.

Cosa ti porti via

1. **Frontend** è ambiente non fidato (browser dell'utente). **Backend** è ambiente fidato (server tuo).
2. Tutto ciò che richiede segreti o controlli autorizzazione **passa dal server**. Sempre.
3. Anche nei progetti full-stack moderni, la divisione esiste — anche se nascosta dietro `'use client'` / `'use server'`.

5.2 — API REST: la metafora del menu del ristorante

Il funzionamento di un ristorante

Sei seduto a un ristorante. Hai un **menu** davanti. Decidi cosa vuoi, lo dici al **cameriere**. Il cameriere porta l'ordine in **cucina**. La cucina prepara il piatto. Il cameriere te lo serve.

Tu (cliente) **non vai in cucina**. Non sai chi cucina, come, con quali ingredienti. Sai solo cosa puoi ordinare (il menu) e ricevi quello che hai chiesto.

Questo è esattamente come funziona un'**API REST**.

Il parallelismo tecnico

Ristorante	API REST
Tu, il cliente	Il client (browser, app mobile)

Ristorante	API REST
Il menu	La documentazione dell'API
Il cameriere	La rete (HTTP)
L'ordine	La richiesta HTTP
La cucina	Il backend / server
Il piatto servito	La risposta HTTP

Cosa è un'API REST nel concreto

Un'**API REST** è un insieme di **URL** (chiamati endpoint) che il tuo backend espone. Ogni endpoint risponde a un certo tipo di richiesta.

Esempio. Hai un sito di gestione cantieri. Il backend espone questi endpoint:

```
GET    /api/cantieri          → ottieni la lista cantieri
GET    /api/cantieri/42       → ottieni il cantiere con
id=42
POST   /api/cantieri          → crea un nuovo cantiere
PATCH  /api/cantieri/42       → modifica il cantiere 42
DELETE /api/cantieri/42       → elimina il cantiere 42
```

Il client (frontend) chiama questi endpoint quando ne ha bisogno. Il backend risponde.

Perché si chiama «REST»

REST sta per **Representational State Transfer**. Non ti serve sapere cosa significa esattamente. Ti basta sapere che è uno **stile di progettazione delle API** dove:

- Ogni «cosa» (utente, ordine, prodotto) è una **risorsa** identificata da un URL
- Le operazioni sulle risorse usano i **verbi HTTP** standard (GET, POST, PUT, PATCH, DELETE)
- Lo stato delle risorse è scambiato in **JSON**

È lo standard più diffuso al mondo. Quando Claude scrive un backend, di default fa REST.

Alternativa: GraphQL

Esiste un'alternativa moderna chiamata **GraphQL**. Funziona diversamente: invece di tanti endpoint, c'è **un solo endpoint** dove il client manda **query strutturate** specificando esattamente cosa vuole.

```
query {
  cantiere(id: 42) {
    nome
    cliente { nome }
    preventivi { totale }
  }
}
```

Vantaggio: prendi solo i dati che ti servono, non più, non meno. Svantaggio: più complesso da implementare.

Per il vibecoder oggi: **REST è la scelta default**. GraphQL solo se hai un caso d'uso specifico (es. mobile app con bandwidth limitata, o team con tante team frontend diverse).

Cosa ti porti via

1. Un'**API REST** è un menù di endpoint (URL) che il backend espone.
2. Ogni endpoint risponde a un tipo di richiesta (lista, singolo, crea, modifica, elimina).
3. **REST** è lo standard più diffuso. Claude di default fa REST.
4. **GraphQL** è un'alternativa moderna ma più complessa. Default REST, GraphQL solo per casi specifici.

5.3 — I verbi HTTP: GET, POST, PUT, PATCH, DELETE

Cinque verbi, cinque significati

Quando il client manda una richiesta al server, **specifica un verbo**. Il verbo dice «che tipo di operazione voglio fare».

I cinque verbi che userai il 99% delle volte:

Verbo	Significato	Esempio
GET	Leggi qualcosa	Dammi il cantiere 42
POST	Crea qualcosa	Crea un nuovo cantiere
PUT	Sostituisci interamente	Sostituisci il cantiere 42 con questo
PATCH	Modifica parzialmente	Cambia solo il nome del cantiere 42
DELETE	Elimina qualcosa	Cancella il cantiere 42

GET vs POST: la differenza più importante

GET e POST sono i due verbi che incontrerai di più. La differenza pratica è:

GET = «non sto cambiando niente, sto solo leggendo»
- I dati viaggiano nella **URL** (visibili, indicizzabili)
- Può essere **cachato** dal browser
- Si può **ripetere** senza conseguenze (10 GET = stesso risultato)
- Limite di dimensione (2KB tipici)

POST = «sto creando o cambiando qualcosa»
- I dati viaggiano nel **body** della richiesta (nascosti)
- **Non viene cachato**
- **Non si ripete** spensieratamente (10 POST = 10 oggetti creati)
- Nessun limite di dimensione pratico

> **SUGGERIMENTO**
>
> **Regola pratica**: se l'utente sta **leggendo**, GET. Se sta **modificando** dati nel sistema, POST/PUT/PATCH/DELETE.

Idempotenza in 30 secondi

Una parola che senti spesso in ambito API: **idempotente**. Significa: «se ripeto la stessa operazione 10 volte, il risultato è lo stesso che farla 1 volta».

Verbo	Idempotente?
GET	Sì (leggi 10 volte = stesso risultato)
POST	**No** (creo 10 oggetti se ripeto 10 volte)
PUT	Sì (sostituisci 10 volte = stesso stato finale)

Verbo	Idempotente?
PATCH	Dipende
DELETE	Sì (elimini 10 volte = comunque eliminato)

Perché ti importa? Quando Claude scrive un endpoint, deve **rispettare** la semantica del verbo. Un POST che si comporta come GET (idempotente) è un anti-pattern. Lo riconoscerai quando lo vedi.

PUT vs PATCH: la differenza sottile

Ti rincorrono entrambi a «modificare un oggetto», ma:

- **PUT** = «ti mando l'oggetto completo, sostituiscilo intero»
- **PATCH** = «ti mando solo i campi che cambiano, modifica solo quelli»

Esempio pratico. Il cantiere 42 ha 10 campi. Vuoi cambiare solo il nome.

PUT richiede di mandare **tutti i 10 campi** (anche quelli invariati). Se ne dimentichi uno, viene cancellato.

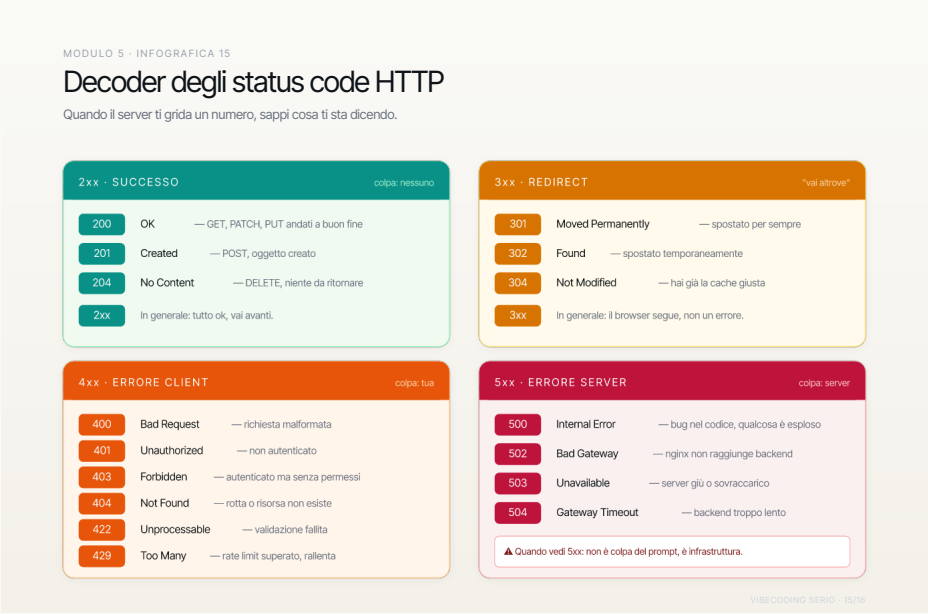
PATCH richiede di mandare **solo il nome**. Gli altri 9 restano com'erano.

Per questo PATCH è più moderno e flessibile. Quando Claude scrive un endpoint di modifica, di solito è PATCH.

Cosa ti porti via

1. Cinque verbi: **GET / POST / PUT / PATCH / DELETE**.
2. **GET** legge, gli altri modificano.
3. **POST** crea, **PATCH** modifica parzialmente, **DELETE** elimina, **PUT** sostituisce completamente (poco usato oggi).
4. **Idempotenza**: GET, PUT, DELETE sono ripetibili senza conseguenze. POST no.

5.4 — I codici di stato HTTP: cosa sta gridando il server



Infografica 15 — Decoder degli status code HTTP

Tre cifre che dicono molto

Ogni risposta HTTP del server include un **codice di stato** (status code) di tre cifre. Il codice ti dice **cosa è successo** alla tua richiesta. Imparare a leggerlo è fondamentale per debuggare.

I codici sono organizzati per **famiglie**:

Famiglia	Significato	Colore mentale
2xx	Successo	verde
3xx	Redirect	giallo
4xx	Errore client (colpa tua)	arancione
5xx	Errore server (colpa loro)	rosso

I 12 codici che ti servono davvero

2xx — Successo

- `200 OK` — Tutto ok. Risposta standard per GET, PATCH, PUT
- `201 Created` — Risorsa creata. Risposta standard per POST
- `204 No Content` — Tutto ok, ma niente da ritornare. Risposta standard per DELETE

3xx — Redirect

- `301 Moved Permanently` — Questa risorsa si è trasferita per sempre, vai al nuovo URL
- `302 Found` — Trasferito temporaneamente (raro oggi)
- `304 Not Modified` — Hai già la versione ultima in cache, non te la rispedisco

4xx — Errore client

- `400 Bad Request` — Hai mandato una richiesta malformata (JSON sbagliato, parametri mancanti)
- `401 Unauthorized` — Non sei autenticato (manca il token / sessione)
- `403 Forbidden` — Sei autenticato ma non hai i permessi per questa operazione
- `404 Not Found` — La rotta o la risorsa non esiste
- `422 Unprocessable Entity` — Validazione fallita (es. email malformata, campo obbligatorio vuoto)
- `429 Too Many Requests` — Hai superato il rate limit, rallenta

5xx — Errore server

- `500 Internal Server Error` — Bug nel codice del server, qualcosa è esploso
- `502 Bad Gateway` — Reverse proxy (nginx) ha problemi a comunicare col backend
- `503 Service Unavailable` — Server giù o sovraccarico
- `504 Gateway Timeout` — Backend troppo lento, il proxy ha desistito

Diagnosi rapida: il colpevole probabile

Codice	Tu...	Lui...
401	hai dimenticato di passare token/sessione	risponde «chi sei?»
403	sei loggato ma non admin	risponde «non puoi»
404	hai sbagliato URL	risponde «non c'è»
422	hai mandato dati invalidi	risponde «valori sbagliati»

Codice	Tu...	Lui...
500	hai trovato un bug	il server è esploso
502/504	infrastruttura ha problemi	nginx o backend giù

Come parlare a Claude di un errore

Quando hai un errore, **menziona il codice**:

❌ Vago: «il login non funziona» ✅ Preciso: «il POST a /api/login restituisce 401 in produzione. In locale funziona con stesso .env. Possibile problema di cookie httpOnly su HTTPS?»

Il secondo prompt fa risparmiare a Claude (e a te) 5 iterazioni.

Cosa ti porti via

1. Codici HTTP organizzati in 4 famiglie: **2xx successo, 3xx redirect, 4xx errore client, 5xx errore server**.
2. Te ne servono **12 davvero**: 200, 201, 204, 301, 400, 401, 403, 404, 422, 429, 500, 502.
3. Saperli leggere = saperli **diagnosticare** in pochi secondi.
4. **Cita sempre il codice** quando descrivi un errore a Claude.

5.5 — Headers, body, JSON: il formato di conversazione

Anatomia di una richiesta HTTP

Quando il client manda una richiesta, il messaggio è composto da tre parti:

```
POST /api/cantieri HTTP/1.1                    ← riga di
stato
Host: cantiere.miosito.it                      ← header 1
Content-Type: application/json                 ← header 2
Authorization: Bearer eyJhbGc...               ← header 3
Cookie: session_id=abc123                      ← header 4

{                                              ← body (JSON)
  "nome": "Ristrutturazione Bianchi",
  "cliente_id": 7,
  "budget": 12500
}
```

Tre parti:

1. **Riga di stato**: verbo + URL + versione protocollo
2. **Headers**: metadati della richiesta (chi sei, cosa accetti, autenticazione, cookie)
3. **Body**: i dati veri (in JSON di solito)

Headers che incontrerai

I più importanti:

- `Content-Type: application/json` — «il body è in formato JSON»
- `Authorization: Bearer xxx` — «sono autenticato, ecco il token»
- `Cookie: ...` — cookie inviati automaticamente dal browser
- `User-Agent: Chrome/...` — «sono Chrome su Windows»
- `Accept: application/json` — «voglio JSON in risposta»
- `Accept-Language: it-IT` — «preferisco contenuti in italiano»

Anatomia di una risposta HTTP

Stessa struttura, dall'altra parte:

```
HTTP/1.1 201 Created                              ← stato
(codice + messaggio)
Content-Type: application/json                    ← header
Set-Cookie: session_id=xyz789; HttpOnly; Secure ← header
Cache-Control: no-cache                           ← header

{                                                 ← body
  "id": 42,
  "nome": "Ristrutturazione Bianchi",
  "createdAt": "2026-04-25T14:32:00Z"
}
```

JSON: la lingua franca

JSON (JavaScript Object Notation) è il formato in cui parlano browser e server. Sintassi:

```
{
  "stringa": "ciao",
  "numero": 42,
  "booleano": true,
  "nullo": null,
  "lista": [1, 2, 3],
  "oggetto": {
    "annidato": "valore"
  }
}
```

Non hai bisogno di scrivere JSON a mano. Devi solo **leggerlo** quando lo vedi negli errori o nel network tab.

> **SUGGERIMENTO**
>
> **Network tab del browser** (F12 → Network) è il tuo migliore amico per debug. Mostra tutte le richieste HTTP che il browser fa, con headers, body, status code. Imparare a leggerlo = velocità di debug 10×.

Cosa ti porti via

1. Una richiesta HTTP ha 3 parti: **riga di stato, headers, body**.
2. **Headers** sono metadati (autenticazione, formato, cookie).
3. **Body** sono i dati veri, di solito in **JSON**.
4. **Apri il network tab del browser** (F12) ogni volta che debuggi un'API. È la tua finestra sul traffico HTTP.

Chiusura del Modulo 5

Adesso il backend non è più una scatola nera:

- Sai perché esiste separato dal frontend (sicurezza)
- Sai cos'è un'API REST e i 5 verbi
- Sai diagnosticare un codice di stato in 5 secondi
- Sai cosa contiene una richiesta HTTP

Quando aprirai il network tab del browser e vedrai una riga rossa con `401`, sai esattamente dove guardare e cosa dire a Claude.

Nel **Modulo 6** entriamo nei database. SQL, ORM, migration, backup.

🎯 Mini-quiz di autovalutazione

1. **Vuoi creare un nuovo prodotto. Quale verbo HTTP usi?**
2. **Il server risponde 401 alla tua richiesta. Cosa è successo?**
3. **Differenza tra PUT e PATCH in una frase.**
4. **Cosa è il `Content-Type: application/json` in un header HTTP?**
5. **Vero o falso: 502 e 504 sono entrambi errori che indicano problemi del client.**

Risposte

1. **POST**. Crea = POST. PUT è «sostituisci completamente un esistente», PATCH è «modifica parzialmente un esistente».
2. Non sei **autenticato**. Manca il token / cookie di sessione, oppure è scaduto. È diverso da 403 (sei loggato ma non hai permessi).
3. **PUT** sostituisce l'oggetto **completamente** (mandi tutti i campi). **PATCH** modifica **solo i campi che mandi** (gli altri restano com'erano). PATCH è più flessibile ed è usato di più oggi.

4. È un **header** che dice al server «il body della richiesta è in formato JSON». Senza di esso, il server potrebbe interpretare il body come testo semplice e non parsare il JSON.
5. **Falso**. 502 e 504 sono **5xx**, errori **server**. Indicano problemi infrastrutturali (proxy nginx non raggiunge il backend, o backend troppo lento). Gli errori client sono i 4xx.

Modulo 5 redatto: 2026-04-25 · Versione 1.0 · 20 pagine · 4500 parole

Modulo 6 — Il database non è un foglio Excel (quasi)

SUGGERIMENTO

5 capitoli · 18 pagine · 1 ora 10 minuti
Pre-requisiti: Modulo 2 letto. Sai che il database è il «terzo vertice» e parla solo col server.

Mettiti comodo

I database fanno paura. Si sente parlare di «schema», «migration», «JOIN», «indici», «ACID», e sembra roba da specialisti. In realtà i concetti che ti servono come vibecoder sono **molto pochi**, e li puoi capire in un'ora.

L'obiettivo di questo modulo non è renderti un database admin. È darti il vocabolario e i concetti per:

- Capire cosa Claude scrive quando crea il database del tuo progetto
- Riconoscere errori comuni (es. dati persi, query sbagliate)
- Sapere come fare backup senza piangere

Cinque capitoli:

1. Tabelle, righe, colonne — il foglio Excel che non puoi rompere
2. Chiavi primarie e relazioni — perché l'utente si lega all'ordine
3. SQL in 10 minuti — le 4 query che Claude scrive sempre
4. SQL vs NoSQL — Supabase, Firebase, Neon
5. Migration e backup — non perdere mai i dati

6.1 — Tabelle, righe, colonne

La metafora del foglio Excel

Il modo più semplice per pensare a un database **relazionale** (Postgres, MySQL, SQLite, Supabase) è come a un **foglio Excel rigoroso**.

Ogni database ha al suo interno **tabelle**. Ogni tabella ha **colonne** (campi) e **righe** (record).

Esempio: una tabella `users`:

id	first_name	last_name	email	city
1	Marco	Bianchi	marco@email.it	Como
2	Francesca	Colombo	fra@email.it	Lecco
3	Giuseppe	Riva	gpe@email.it	Cantù

Stessa cosa in Excel: foglio `users`, colonne `id`, `first_name`, `last_name`, `email`, `city`, e tre righe.

Cosa cambia rispetto a Excel

Tre differenze importanti:

1. **Schema rigido (i tipi di colonna)**

Ogni colonna in un database **ha un tipo**. La colonna `id` è un numero intero. `email` è una stringa. `created_at` è una data.

Se provi a inserire un valore del tipo sbagliato (es. la stringa «ciao» nella colonna `id` numerica), il database **rifiuta** l'operazione. In Excel puoi mettere quello che vuoi dove vuoi.

2. **Vincoli (constraints)**

Puoi dire al database regole come:

- « `email` non può essere vuoto» → NOT NULL
- « `email` deve essere unico» → UNIQUE
- « `age` deve essere ≥ 0» → CHECK
- «questo `user_id` deve esistere nella tabella `users`» → FOREIGN KEY

Excel non ha queste sicurezze. Un database sì.

3. **Performance**

Un database con **1 milione di righe** può cercare un singolo record in **millisecondi**, perché usa **indici** (strutture interne ottimizzate per ricerca veloce). Excel a 1 milione di righe piange.

Esempi di tipi di colonna

I più usati:

Tipo	A cosa serve	Esempio
`INTEGER`	numero intero	id, età, quantità
`TEXT` / `VARCHAR(n)`	testo	nome, email, indirizzo
`BOOLEAN`	vero/falso	isactive, isadmin
`TIMESTAMP`	data + ora	createdat, deletedat
`DECIMAL(10,2)`	numero decimale	prezzo, quantità float
`JSON` / `JSONB`	dati strutturati flessibili	metadati, configurazioni
`UUID`	identificatore unico globale	id moderni (es. `xyz-abc-123-...`)

Non ti serve impararli a memoria. Quando Claude crea uno schema, vede questi nomi → ora sai cosa significano.

Cosa ti porti via

1. Database = foglio Excel **rigoroso**.
2. Le **tabelle** hanno colonne (con tipo) e righe.
3. Tre cose che Excel non ha: **schema rigido, vincoli, performance**.
4. I **tipi di colonna** che incontri di più: integer, text, boolean, timestamp, decimal, json, uuid.

6.2 — Chiavi primarie e relazioni

La chiave primaria: l'identificatore univoco

Ogni tabella ha (deve avere) una colonna che identifica univocamente ogni riga: la **chiave primaria** (primary key, PK).

Tipicamente si chiama `id` ed è un numero o un UUID.

```
users:
  id (PK) | first_name | email
  1       | Marco      | marco@...
  2       | Francesca  | fra@...
```

Se chiedo «dammi l'utente 1», il database lo trova **istantaneamente** perché la PK è indicizzata. Se chiedessi «dammi l'utente di nome Marco», dovrebbe scorrere tutti gli utenti (a meno che non ci sia un altro indice).

Le relazioni: come le tabelle si parlano

Il vero potere dei database **relazionali** è proprio questo: **le tabelle si possono collegare tra loro**.

Esempio. Hai un'app di e-commerce. Hai due tabelle:

`users`:

id	name
1	Marco
2	Francesca

`orders`:

id	user_id	total
100	1	50.00
101	2	120.00
102	1	30.00

La colonna `user_id` in `orders` **punta** alla colonna `id` di `users`. Questo si chiama **foreign key** (chiave esterna, FK).

Adesso puoi fare domande del tipo: «dammi tutti gli ordini di Marco» → il database segue il riferimento e ti dà gli ordini con `user_id = 1`.

I tre tipi di relazione

1. **Uno a molti (1:N)**

Un utente ha molti ordini. Ogni ordine ha un solo utente. Il caso più comune. Si implementa con una **foreign key** sulla parte «molti» (in `orders`).

2. **Molti a molti (N:N)**

Un utente può avere molti ruoli. Un ruolo può essere assegnato a molti utenti. Si implementa con una **tabella intermedia**:

```
users_roles:
  user_id | role_id
  1       | 1
  1       | 3
  2       | 2
```

3. **Uno a uno (1:1)**

Un utente ha un solo profilo. Un profilo appartiene a un solo utente. Raro, di solito si fonde tutto in una tabella sola.

Cancellazione con CASCADE: una scelta importante

Quando elimini un utente, cosa succede ai suoi ordini? Tre opzioni:

- **CASCADE**: elimina anche gli ordini → «se sparisce Marco, spariscono i suoi ordini»
- **SET NULL**: imposta `user_id = NULL` → «gli ordini restano, ma ora "orfani"»
- **RESTRICT**: vieta la cancellazione finché ci sono ordini → «non puoi cancellare Marco perché ha ordini»

Quando Claude crea uno schema, sceglie una di queste. Per la cancellazione di un utente, di solito ha senso CASCADE per le **tabelle dipendenti** (come `sessions`, `tokens`) e RESTRICT o soft-delete per i **dati di business** (come `orders`, che vorresti tenere per la fattura).

Cosa ti porti via

1. Ogni tabella ha una **chiave primaria** (PK), tipicamente `id`.
2. Le tabelle si collegano via **foreign key** (FK). È l'essenza dei database «relazionali».
3. Tre tipi di relazione: **1:N** (più comune), **N:N** (con tabella intermedia), **1:1** (raro).
4. Quando si elimina, attenzione a **CASCADE / SET NULL / RESTRICT**.

6.3 — SQL in 10 minuti

Le 4 query che Claude scrive sempre

SQL (Structured Query Language) è il linguaggio per parlare ai database relazionali. Ha tante funzioni avanzate, ma il 95% del codice che leggi userà solo **4 verbi**:

```
SELECT  -- leggi
INSERT  -- crea
UPDATE  -- modifica
DELETE  -- elimina
```

SELECT — leggere

```
SELECT * FROM users WHERE city = 'Como';
```

Tradotto: «Dammi tutte le colonne (`*`) dalla tabella `users` dove `city = 'Como'`».

Varianti utili:

```
-- Solo certe colonne
SELECT first_name, email FROM users WHERE city = 'Como';

-- Ordinamento
SELECT * FROM users ORDER BY last_name ASC;

-- Limit
SELECT * FROM users LIMIT 10;

-- Conteggio
SELECT COUNT(*) FROM users;
```

INSERT — creare

```
INSERT INTO users (first_name, email, city)
VALUES ('Marco', 'marco@email.it', 'Como');
```

Tradotto: «Crea una nuova riga in `users` con questi valori». La PK (`id`) viene generata automaticamente.

UPDATE — modificare

```
UPDATE users SET city = 'Milano' WHERE id = 1;
```

Tradotto: «Cambia la città a "Milano" per l'utente con id=1».

> **ATTENZIONE**
>
> **Attenzione**: UPDATE senza WHERE modifica **tutte le righe** della tabella. Se Claude scrive `UPDATE users SET city = 'Milano'` senza WHERE, sta dicendo «cambia la città di TUTTI gli utenti a Milano». Errore catastrofico.

DELETE — eliminare

```
DELETE FROM users WHERE id = 1;
```

Stesso warning: DELETE senza WHERE = elimina tutto. Pericolo.

JOIN: collegare due tabelle

Il 5% più «avanzato» sono i JOIN, che ti permettono di interrogare più tabelle insieme:

```
SELECT users.first_name, orders.total
FROM users
JOIN orders ON orders.user_id = users.id
WHERE users.city = 'Como';
```

Tradotto: «Dammi nome utente e totale ordine, prendendo gli utenti di Como e i loro ordini». Sotto il cofano sta seguendo le foreign key.

SQL injection: il pericolo che NON devi mai vedere

Una nota di sicurezza importante. Mai concatenare input utente in una query SQL così:

```
// MALE
const query = `SELECT * FROM users WHERE email =
'${userInput}'`;
```

Se l'utente inserisce `marco@x.it' OR '1'='1`, la query diventa `SELECT * FROM users WHERE email = 'marco@x.it' OR '1'='1'` → ritorna tutti gli utenti. È **SQL injection**.

Il modo corretto è **parametrizzare**:

```
// BENE
const query = 'SELECT * FROM users WHERE email = $1';
const result = await db.query(query, [userInput]);
```

Con un ORM (vedi 6.4) questo lo fai senza pensarci. Ma sappi che esiste.

Cosa ti porti via

1. Le **4 query base** in SQL: SELECT (leggi), INSERT (crea), UPDATE (modifica), DELETE (elimina).
2. **WHERE** filtra le righe. Senza WHERE, UPDATE/DELETE colpiscono tutto. Pericolo.
3. **JOIN** unisce due tabelle attraverso le foreign key.
4. **SQL injection**: mai concatenare input utente. Sempre parametri. Gli ORM moderni lo fanno per te.

6.4 — SQL vs NoSQL: Supabase, Firebase, Neon

Le due famiglie

I database si dividono in due grandi famiglie. Te le ho accennate nel Modulo 2, qui le approfondiamo.

SQL / Relazionali:
- Schema rigido, tabelle, righe, colonne
- Linguaggio: SQL
- Esempi: **PostgreSQL** (Postgres), **MySQL**, **SQLite**, **MariaDB**
- Hosting moderno: **Supabase** (Postgres come servizio), **Neon** (Postgres serverless), **PlanetScale** (MySQL serverless)

NoSQL / Documentali:
- Schema flessibile, «documenti» tipo JSON, niente tabelle rigide
- Linguaggio: variabile (query API specifica)
- Esempi: **MongoDB**, **Firestore** (Google), **DynamoDB** (AWS)
- Hosting: **MongoDB Atlas**, **Firebase** (include Firestore)

Quando usare SQL

Default per il 90% dei casi. SQL vince quando:
- I dati hanno **struttura chiara** (utenti, ordini, prodotti)
- Hai **relazioni** (un utente, molti ordini)
- Vuoi **transazioni** (es. pagamento + decremento stock contemporanei)
- Vuoi **query complesse** con aggregazioni

Per un'app web standard moderna: **Postgres su Supabase o Neon**. È la scelta default. Affidabile, scalabile, ben supportato dagli ORM.

Quando usare NoSQL

NoSQL ha senso in casi specifici:
- Dati molto **eterogenei** (ogni documento ha campi diversi)
- **Scaling massiccio** in sola scrittura (milioni di record/sec)
- App **mobile real-time** dove Firestore brilla per le sue subscriptions

Esempi: app real-time di chat, dati IoT, analytics ad alto volume. Per il vibecoder che parte oggi: probabilmente **non è il tuo caso**. Postgres ti basta e ti avanza.

Supabase: la scelta moderna del vibecoder

Una nota su **Supabase**. È un servizio che ti dà:

- Un **Postgres** ospitato (database)
- Un'**API REST** generata automaticamente sopra le tue tabelle
- **Autenticazione** pronta (login, OAuth)
- **Storage file** (immagini, documenti)
- **Real-time** subscriptions

In pratica: con Supabase puoi avere il backend di un'app web in **15 minuti** senza scrivere codice server. Per questo è la scelta default di Bolt, Lovable, e Claude quando genera app moderne.

ORM: il traduttore tra codice e SQL

Lo abbiamo accennato nel Modulo 2: scrivere SQL a mano è scomodo. Si usa un **ORM** (Object-Relational Mapper) che traduce il tuo codice in SQL.

```
// Tu scrivi questo (Drizzle, l'ORM moderno per TypeScript):
const usersInComo = await db.select()
  .from(users)
  .where(eq(users.city, 'Como'));

// Drizzle traduce in:
// SELECT * FROM users WHERE city = 'Como';
```

ORM popolari:

- **Drizzle** (TypeScript, moderno) — il preferito oggi
- **Prisma** (TypeScript, enterprise) — più magico ma più pesante
- **Eloquent** (Laravel) — il «default» PHP
- **SQLAlchemy** (Python)
- **ActiveRecord** (Rails)

Quando Claude scrive un backend, **quasi sempre** usa un ORM. Tu non vedi più il SQL diretto, vedi il codice ORM.

Cosa ti porti via

1. Due famiglie: **SQL** (Postgres, MySQL, SQLite) e **NoSQL** (MongoDB, Firestore).
2. **Default**: SQL, in particolare **Postgres su Supabase o Neon**.
3. **Supabase** è il «Firebase di Postgres» — ti dà DB + auth + storage + real-time.
4. Gli **ORM** (Drizzle, Prisma) traducono il tuo codice in SQL. Più sicuri (no SQL injection automatica), più leggibili.

6.5 — Migration e backup: non perdere mai i dati

Migration: cambiare lo schema senza piangere

Hai creato la tabella `users` con 3 colonne. Domani vuoi aggiungere `phone_number`. **Come fai?**

Non puoi semplicemente cambiare lo schema con `ALTER TABLE` a mano in produzione: rischi di rompere tutto, non hai versionamento.

La soluzione si chiama **migration**: file SQL (o codice) versionati in git, applicati in ordine, che descrivono ogni modifica allo schema nel tempo.

Esempio Drizzle/Prisma genera un file tipo:

```
-- migrations/001_add_phone_number.sql
ALTER TABLE users ADD COLUMN phone_number TEXT;
```

Quando deployi in produzione, esegui questo script (`pnpm drizzle-kit push` o `prisma migrate deploy`). Il database aggiunge la colonna. Lo schema in locale e in produzione sono allineati.

Il bug classico del vibecoder: schema disallineato

Il caso tipico:

1. In locale aggiungi una colonna direttamente con un click su Supabase Studio
2. Modifichi il codice per usarla
3. Deployi in produzione
4. Crash → `ERROR: column "phone_number" does not exist`

Causa: hai cambiato lo schema solo in locale, non in produzione. La migration esiste apposta per evitare questo errore.

PROMPT-READY

Prompt-ready: «Aggiungi una colonna `phone_number` alla tabella `users`. Crea una migration con Drizzle. Dimmi anche come applicarla in produzione.»

Backup: la vita che ti salva

Storia vera (anonimizzata): un freelancer ha lavorato 6 mesi a un sito SaaS per un cliente. Decide di «ottimizzare» il database. Esegue uno script di pulizia. Cancella **tutto**. Niente backup. Sei mesi di lavoro andati. Cliente perso. Reputazione devastata.

Il **backup** è la differenza tra «errore recuperabile» e «carriera finita».

Strategie di backup per i 3 casi più comuni

Caso 1: Supabase

Supabase fa backup automatici **giornalieri** sul piano free, e on-demand sul piano Pro. Buono ma:

- Free: backup conservati per 7 giorni
- Pro (25$/mese): backup conservati per 30 giorni + Point-in-Time Recovery

Configurazione minima: passa al piano Pro **prima di andare live con clienti veri**. 25$ vale l'assicurazione.

Caso 2: Postgres su VPS Aruba (o altro VPS)

Devi configurare i backup tu. Setup in 10 minuti:

```
# Crea uno script di backup
#!/bin/bash
DATE=$(date +%Y-%m-%d)
pg_dump cantiere | gzip > /backups/cantiere-$DATE.sql.gz
# Tieni solo gli ultimi 30 giorni
find /backups -name "*.sql.gz" -mtime +30 -delete
```

Aggiungi al crontab per girare ogni notte. Copia i backup su un server esterno (es. Backblaze B2 a 6$/TB/mese).

Caso 3: SQLite (es. progetti hobby)

Il database SQLite è un file. Per fare backup, **copialo**. Una riga:

```
cp app.db /backups/app-$(date +%Y-%m-%d).db
```

Setup in 30 secondi. È un caso d'uso comune per progetti piccoli (es. blog personale, tool interno).

La regola d'oro del backup

> **SUGGERIMENTO**
>
> **Un backup non testato non è un backup. È una speranza.**

Una volta al mese (almeno):
1. Scarica un backup recente
2. Restoralo in un ambiente di test
3. Verifica che funzioni

Senza il test, non sai se i tuoi backup sono validi. Lo scopri solo nel momento sbagliato.

Cosa ti porti via

1. **Migration** = file versionati che descrivono cambiamenti allo schema. Mai modificare lo schema in produzione a mano.
2. Il **bug classico**: schema cambiato in locale ma non in produzione → crash.
3. **Backup automatico** sempre. Su Supabase: piano Pro. Su VPS: cron + script.
4. **Testa i backup**: una volta al mese, restoralo per davvero. Senza test, non sono backup.

Chiusura del Modulo 6

Adesso il database non è più magia nera:

- Sai cosa sono tabelle, colonne, chiavi, relazioni
- Sai leggere SELECT/INSERT/UPDATE/DELETE quando Claude le scrive
- Sai distinguere SQL e NoSQL e perché di default vai su Postgres
- Sai perché esistono migrations e come funzionano
- Sai come fare backup e perché è non-negoziabile

Nel **Modulo 7** torniamo all'infrastruttura: DNS in profondità, HTTPS, nginx, variabili d'ambiente, cookie, CORS. Tutto quello che separa «funziona in locale» da «funziona in produzione».

🎯 Mini-quiz di autovalutazione

1. **Vero o falso: in un database Postgres puoi mettere una stringa «ciao» in una colonna numerica.**
2. **Cos'è una «foreign key»?**
3. **Hai un comando `UPDATE users SET email = 'x@y.it'` senza WHERE. Cosa succede?**
4. **Per un'app web moderna che parte oggi, quale database conviene di default?**
5. **Hai aggiunto una colonna alla tabella `users` solo in locale. Deployi in produzione. Cosa succede?**

Risposte

1. **Falso**. Lo schema rigido di Postgres rifiuta l'inserimento se il tipo non corrisponde. È una protezione contro bug e data corruption.
2. Una colonna che **punta** alla chiave primaria di un'altra tabella. Es. `orders.user_id` punta a `users.id`. Permette di mettere in relazione le tabelle.
3. **Aggiorna l'email di TUTTI gli utenti** a `'x@y.it'`. Errore catastrofico. WHERE è obbligatorio (mentale, non sintattico).
4. **Postgres**, ospitato su **Supabase** o **Neon**. Default moderno per il 90% dei casi. NoSQL solo per casi specifici (real-time massivo, dati eterogenei).

5. **Crash in produzione**: il codice cerca la colonna nuova ma il DB di produzione non ce l'ha. Soluzione: usare le migration → modifiche allo schema versionate ed eseguite anche in produzione.

Modulo 6 redatto: 2026-04-25 · Versione 1.0 · 18 pagine · 4400 parole

Modulo 7 — Il viaggio del click in produzione

SUGGERIMENTO

6 capitoli · 22 pagine · 1 ora 30 minuti
Pre-requisiti: Modulo 1 letto. Conosci il flusso del click in 8 passi.

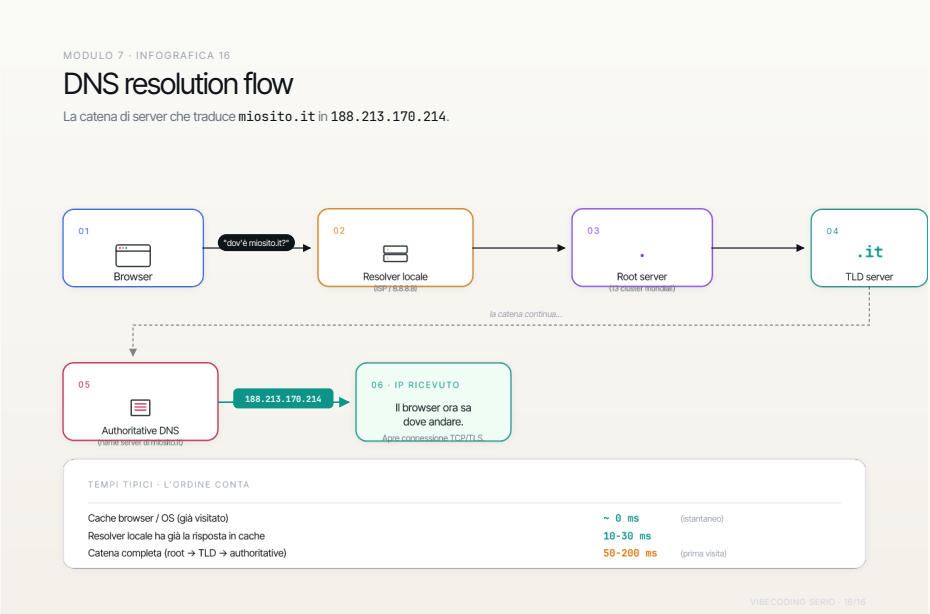
Mettiti comodo

Questo è il **modulo più tecnico** del libro. Ti porta a fondo nei meccanismi che separano «funziona in localhost» da «funziona in produzione». Sei concetti, sei capitoli.

Quando padroneggi questi sei, sei un **vibecoder serio**: puoi mettere online un'app, gestire il dominio, debuggare problemi di rete, capire come funziona la sicurezza dei cookie. Sono le competenze che il 90% dei vibecoder ingenui non ha, e che il cliente ti riconosce a vista.

I sei capitoli:
1. DNS — l'elenco telefonico di Internet (in profondità)
2. HTTPS e certificati SSL — il lucchetto che Aruba ti installa
3. Nginx — il buttafuori del server
4. Variabili d'ambiente — perché la tua API key non va nel codice
5. Cookie e sessioni — come il sito ricorda chi sei
6. CORS — l'errore più odiato spiegato in 3 paragrafi

7.1 — DNS in profondità



Infografica 16 — DNS resolution flow

Ripasso: cosa è il DNS

Lo abbiamo già toccato nel Modulo 0. Il **DNS** (Domain Name System) traduce nomi di dominio (`miosito.it`) in indirizzi IP (`188.213.170.214`). È un **elenco telefonico mondiale distribuito**.

Adesso entriamo in come funziona davvero la traduzione.

La catena della risoluzione DNS

Quando il browser vuole `miosito.it` , **non chiede direttamente** a un server. Segue una catena.

1. **Browser cache**: l'ho cercato di recente? Sì → uso quell'IP.
2. **OS cache**: anche il sistema operativo tiene una cache.
3. **Resolver locale (ISP)**: di solito il router di casa o il DNS del provider Internet (es. Telecom).
4. **Root server**: se il resolver non lo sa, chiede ai root server del DNS mondiale. Sono 13 cluster sparsi nel mondo.

5. **TLD server**: il root dice «per .it chiedi al server `.it`».
6. **Authoritative server**: il TLD `.it` dice «per miosito.it chiedi al name server X».
7. Il name server X risponde con l'IP.

Tempo totale: tipicamente 20-100ms se non in cache. Se è in cache (browser, OS, o resolver), tempo zero.

Record DNS che incontrerai

Quando configuri un dominio, scrivi **record DNS**. I 4 più importanti:

Record A — punta un dominio a un IP

```
miosito.it.    A    188.213.170.214
```

Tradotto: «Quando qualcuno chiede `miosito.it`, manda all'IP `188.213.170.214`». È il record più comune.

Record CNAME — punta un dominio a un altro dominio

```
www.miosito.it.    CNAME    miosito.it.
```

Tradotto: «`www.miosito.it` è un alias di `miosito.it`. Cerca quello». Comodo per non duplicare la configurazione.

Record MX — dice dove vanno le email per questo dominio

```
miosito.it.    MX    10 mail.aruba.it.
```

Senza record MX, il dominio non riceve email.

Record TXT — note libere

```
miosito.it.    TXT    "v=spf1 include:_spf.google.com ~all"
```

Usato per anti-spam (SPF, DKIM, DMARC), per verifica proprietà di servizi (Google, Cloudflare), e altri metadati.

TTL: time to live

Ogni record DNS ha un **TTL** (Time To Live): per quanto tempo le cache possono memorizzare la risposta. Tipico: 3600 secondi (1 ora) o 86400 (24 ore).

Quando cambi un record DNS, **devi aspettare il TTL** perché il cambiamento si propaghi globalmente. Se hai messo un TTL alto (24h) e cambi il record, alcuni resolver useranno la vecchia versione per 24h.

Pratica: prima di cambiare un record DNS importante, **abbassa il TTL** a 60 secondi una settimana prima. Poi cambia. Poi rialza il TTL.

Verificare la propagazione

Dopo aver cambiato un record, vuoi sapere se è propagato. Strumenti:

```
# Linux/Mac/WSL
dig miosito.it
nslookup miosito.it

# Tool web
# https://dnschecker.org → mostra il record da 30+ posti nel
mondo
```

Se il tuo record è propagato globalmente, dnschecker mostra l'IP nuovo da tutti i posti.

Cosa ti porti via

1. Il DNS è una **catena di server**: browser → OS → resolver → root → TLD → authoritative.
2. I 4 record principali: **A** (dominio→IP), **CNAME** (alias), **MX** (email), **TXT** (note libere).
3. **TTL** decide quanto le cache memorizzano. Abbassa prima di cambi importanti.
4. **dnschecker.org** ti dice se il tuo cambio è propagato globalmente.

7.2 — HTTPS e certificati SSL

Perché HTTPS è obbligatorio

Nel 2026, **HTTPS è obbligatorio**. Non è più «consigliato»: è il default. Senza HTTPS:

- Il browser mostra il warning «Sito non sicuro» → utenti scappano
- Google penalizza pesantemente nel ranking
- I cookie con flag `Secure` non funzionano (login si rompe)
- Le moderne API web (geolocalizzazione, camera, ecc.) non funzionano

Se vedi un sito senza HTTPS, è un sito **degli anni 2010**.

Cosa è un certificato SSL/TLS

Per attivare HTTPS, il server deve avere un **certificato SSL/TLS**. Pensa al certificato come a un **passaporto digitale** firmato da un'autorità riconosciuta.

Quando il browser si connette al server, il server gli mostra il certificato. Il browser verifica:

1. Il certificato è **valido** (non scaduto, non revocato)
2. È stato emesso per **questo dominio** (non per uno diverso)
3. È firmato da un'**autorità riconosciuta** (CA, Certificate Authority)

Se tutti i check passano, il browser **fida** il server e crea una connessione cifrata. Lucchetto verde.

Let's Encrypt: certificati gratis

Una volta i certificati SSL costavano 50-500€/anno. Oggi sono **gratis** grazie a **Let's Encrypt**, una CA non-profit che emette certificati validi 90 giorni con rinnovo automatico.

Il modo standard di usarli è con uno strumento chiamato **Certbot**:

```
# Su un server con nginx installato
sudo certbot --nginx -d miosito.it -d www.miosito.it
```

In 60 secondi:

1. Certbot verifica che tu controlli il dominio (con un file temporaneo)
2. Genera il certificato
3. Configura automaticamente nginx per usarlo
4. Imposta il rinnovo automatico ogni 60 giorni

Da quel momento, il sito è **HTTPS forever** (finché paghi il dominio e Certbot fa il suo lavoro).

Quando NON devi pensare al certificato

Se usi **Vercel, Netlify, Railway, Render**, il certificato è **gestito automaticamente**. Tu non vedi nemmeno il problema. Cloudflare proxy fa lo stesso.

Quando devi pensarci: **VPS Aruba self-hosted** (devi configurare Certbot manualmente, come abbiamo fatto per `vibecodingserio.vibecanyon.com`).

Errori comuni

Errore 1: «Certificato scaduto» Sintomo: warning sul browser. Causa: Certbot non ha rinnovato. Fix: `sudo certbot renew --force-renewal`.

Errore 2: «Certificato per dominio diverso» Sintomo: warning sul browser. Causa: hai un certificato per `miosito.it` ma l'utente apre `www.miosito.it`. Fix: includi tutti i sottodomini nel certificato (`certbot --nginx -d miosito.it -d www.miosito.it`).

Errore 3: «Mixed content» Sintomo: pagina HTTPS ma con risorse HTTP dentro (immagini, script). Browser blocca tutto. Fix: tutte le risorse devono essere HTTPS o relative (`//cdn.miosito.it/...`).

Cosa ti porti via

1. **HTTPS è obbligatorio** in produzione. Senza, sito morto.
2. **Let's Encrypt + Certbot** = certificati gratis, rinnovo automatico ogni 60 giorni.
3. Su Vercel/Netlify il certificato è gestito automaticamente. Su VPS lo configuri tu.
4. Errori comuni: certificato scaduto, dominio sbagliato, mixed content.

7.3 — Nginx: il buttafuori del server

Cosa fa nginx

Su un VPS Aruba (e simili) c'è quasi sempre **nginx** in mezzo tra il browser dell'utente e la tua app.

Pensa a nginx come a un **buttafuori**: quando un utente arriva al server (porta 80 HTTP o 443 HTTPS), nginx riceve la richiesta, decide a quale app inoltrarla, e gli passa il messaggio.

Perché serve? Perché su un solo server giri **molte app diverse**:

- `cantiere.vibecanyon.com` → app Next.js sulla porta interna 3010
- `safetrack.vibecanyon.com` → app Next.js sulla porta interna 3000
- `vibecodingserio.vibecanyon.com` → file statici nella cartella `/var/www/vibecodingserio/`

Nginx fa il **routing**: in base al dominio richiesto, manda la richiesta all'app giusta.

Il pattern del reverse proxy

Il caso più comune: hai un'app Next.js che gira sulla porta 3010. Vuoi che gli utenti la raggiungano su `cantiere.vibecanyon.com` (porta 443).

Configurazione nginx tipica:

```
server {
    listen 443 ssl;
    server_name cantiere.vibecanyon.com;

    ssl_certificate /etc/letsencrypt/live/
cantiere.vibecanyon.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/
cantiere.vibecanyon.com/privkey.pem;

    location / {
        proxy_pass http://localhost:3010;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

Tradotto: «Ascolta sulla porta 443 con HTTPS. Quando arriva una richiesta per `cantiere.vibecanyon.com`, inoltrala alla mia app sulla porta 3010».

Questo si chiama **reverse proxy** (proxy inverso) — nginx è davanti, l'app è dietro.

Statici vs reverse proxy

Per **siti statici** (HTML/CSS/JS senza backend), nginx serve i file direttamente:

```
server {
    listen 443 ssl;
    server_name vibecodingserio.vibecanyon.com;
    root /var/www/vibecodingserio;
    index index.html;
}
```

Per **app dinamiche** (Next.js, Express, Python), nginx fa reverse proxy a una porta interna.

Errori nginx più comuni

502 Bad Gateway: nginx non riesce a raggiungere il backend. Cause comuni: l'app sulla porta 3010 è morta (`pm2 status` per verificare), oppure è stata riavviata su porta diversa.

504 Gateway Timeout: il backend è troppo lento (>60 secondi default). Cause: query DB lente, codice bloccante. Il problema è dell'app, non di nginx.

413 Request Entity Too Large: hai mandato un file troppo grande (default nginx: 1MB). Fix: aggiungi `client_max_body_size 50M;` al config nginx.

Configurazione non si carica: hai cambiato il config ma il sito è uguale. Fix: `sudo nginx -t` (valida la sintassi) e poi `sudo systemctl reload nginx` .

Cosa ti porti via

1. **Nginx** sta davanti alle tue app, fa **routing** per dominio e **reverse proxy** alle porte interne.
2. Su Vercel/Netlify nginx è invisibile. Su VPS Aruba lo configuri tu (file in `/etc/nginx/sites-available/`).
3. **502** = backend giù. **504** = backend lento. **413** = upload troppo grande.
4. Dopo modifiche: `nginx -t` per validare, `systemctl reload nginx` per applicare.

7.4 — Variabili d'ambiente

Cosa sono e perché esistono

Le **variabili d'ambiente** (`.env`) sono **valori configurabili** che il tuo codice legge a runtime, ma che non vivono dentro il codice stesso.

Esempi tipici:

- `DATABASE_URL` — la stringa di connessione al database
- `OPENAI_API_KEY` — la tua API key segreta
- `NEXTAUTH_SECRET` — il segreto per firmare i token di sessione
- `STRIPE_SECRET_KEY` — la chiave Stripe per processare pagamenti

Perché non nel codice? Tre motivi:

1. **Sicurezza**: se metti `OPENAI_API_KEY = "sk-..."` nel codice e lo committi su GitHub, **il mondo intero la vede**. Bot scansionano GitHub e rubano API key in **pochi minuti**.
2. **Differenze tra ambienti**: in locale vuoi un DB locale, in produzione un DB cloud. Stesso codice, valori diversi.
3. **Non versionata**: le variabili non vivono in git, vivono nel **server** (o nel pannello del provider).

Anatomia di un file `.env`

Sintassi semplice:

```
DATABASE_URL=postgresql://postgres:password@localhost:5432/
cantiere
OPENAI_API_KEY=sk-proj-abcd1234...
NEXTAUTH_SECRET=randomstring32characterslong...
STRIPE_SECRET_KEY=sk_test_abcd...
```

Una variabile per riga, formato `NOME=valore`. Niente spazi attorno all' `=`. Niente virgolette (di solito).

`.env.local`, `.env`, `.env.example`

Tre file con tre ruoli:

`.env.local` — i tuoi valori in locale

- È nel tuo PC. Mai committato in git.
- Contiene i valori veri (`OPENAI_API_KEY=sk-...`).
- `.gitignore` lo esclude.

`.env` — il default condiviso
- Più raro oggi. A volte usato per valori non sensibili.

`.env.example` — il template
- Versionato in git.
- Contiene **i nomi** delle variabili necessarie, **senza valori veri**:

```
DATABASE_URL=postgresql://username:password@localhost:5432/your_db
OPENAI_API_KEY=your-openai-key-here
NEXTAUTH_SECRET=generate-random-string
```

- Quando un nuovo collaboratore clona il repo, fa `cp .env.example .env.local` e riempie i suoi valori.

Variabili in produzione

In locale: leggi da `.env.local`. In produzione: dipende dal provider.
- **Vercel / Netlify**: pannello → «Environment Variables» → aggiungi le coppie nome=valore.
- **Aruba VPS**: crea il file `.env.production` o `.env.local` direttamente sul server (via SSH). Esegui `pm2 restart` per fargli ricaricare.
- **Docker**: passi le variabili al container all'avvio (`docker run -e VAR=value ...`).

Il bug classico: variabile mancante in produzione

Storia tipica: «Il sito funziona in locale, in produzione il login dà 500 senza spiegazioni.»

Causa probabile: hai aggiunto una variabile (`NEXTAUTH_SECRET`) in `.env.local` ma hai dimenticato di settarla nel pannello Vercel.

Diagnosi:
1. Apri i log del server (Vercel → Functions → Logs)
2. Cerca un messaggio tipo `NEXTAUTH_SECRET is not defined`
3. Aggiungi la variabile nel pannello
4. Redeploy

Variabili `NEXT_PUBLIC_*` in Next.js

Una nota importante. In Next.js, le variabili che iniziano con `NEXT_PUBLIC_` vengono **bundlate nel JavaScript del browser**. Sono **visibili** a chiunque ispezioni la pagina.

```
NEXT_PUBLIC_API_URL=https://api.miosito.it    # OK, è un URL
pubblico
NEXT_PUBLIC_OPENAI_KEY=sk-...                 # ⚠ DISASTRO,
key esposta!
```

Regola: se è una **chiave segreta**, NON metterla in `NEXT_PUBLIC_*`. Tienila come variabile normale (server-only).

Cosa ti porti via

1. **Variabili d'ambiente** = valori che il codice legge ma non sono dentro il codice.
2. Tre file: `.env.local` (tuo, mai committato), `.env.example` (template versionato), produzione gestita dal provider.
3. **Bug classico**: variabile in locale ma non in produzione. Crash silenzioso.
4. In Next.js, `NEXT_PUBLIC_*` è visibile al browser. Mai metterci segreti.

7.5 — Cookie e sessioni: come il sito ricorda chi sei

Il problema fondamentale

HTTP è **stateless**: ogni richiesta è indipendente. Il server non ricorda automaticamente chi sei tra una richiesta e l'altra.

Ma allora come fa il sito a sapere che hai fatto login? Come fa a non chiederti la password ad ogni click?

Risposta: usa **cookie** (lato browser) e **sessioni** (lato server) per simulare uno stato.

Cookie: il foglietto che il browser tiene per te

Un **cookie** è un piccolo testo che il server invia al browser. Il browser lo **salva** e lo **rispedisce automaticamente** ad ogni richiesta successiva verso lo stesso dominio.

Esempio. Server al login:

```
HTTP/1.1 200 OK
Set-Cookie: session_id=abc123xyz; HttpOnly; Secure;
SameSite=Strict; Max-Age=604800
```

Tradotto: «Browser, salva un cookie con nome `session_id` e valore `abc123xyz`. È valido 7 giorni.»

Da quel momento, ad ogni richiesta successiva il browser aggiunge automaticamente:

```
GET /api/profile HTTP/1.1
Cookie: session_id=abc123xyz
```

Il server vede il cookie, riconosce la sessione, sa chi sei.

Le flag che salvano la vita

I cookie hanno **flag** importantissime:

`HttpOnly` — il cookie non è leggibile da JavaScript
- Senza HttpOnly, qualunque script malevolo (es. XSS injection) può rubare il cookie.
- **SEMPRE settare HttpOnly** sui cookie di sessione.

`Secure` — il cookie viaggia solo su HTTPS
- Senza Secure, il cookie viaggia anche su HTTP, intercettabile.
- **SEMPRE settare Secure** in produzione.

`SameSite` — quando il cookie è inviato a richieste cross-site
- `Strict`: solo richieste dallo stesso sito (più sicuro)
- `Lax`: anche da link esterni (default ragionevole)
- `None`: sempre (rischioso, richiede `Secure`)

`Max-Age` o `Expires` — quando il cookie scade
- Senza, il cookie sparisce alla chiusura del browser (cookie «di sessione»).
- Con, dura un tempo definito (es. 7 giorni).

Sessione lato server

Il cookie da solo non basta. Il valore `abc123xyz` deve corrispondere a **qualcosa** sul server.

Tipicamente, il server tiene una **sessione** in memoria o nel database:

```
sessions:
session_id  | user_id | expires_at
abc123xyz   | 7       | 2026-05-02 14:00:00
```

Quando il browser manda il cookie `session_id=abc123xyz`, il server cerca nella tabella, trova `user_id=7`, sa chi è l'utente.

JWT: l'alternativa moderna

JWT (JSON Web Token) è una tecnica diversa: invece di tenere la sessione sul server, il server crea un token **firmato** che contiene direttamente le info utente.

```
Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo3LCJleHAiOj
```

Quando il client manda il token, il server **verifica la firma** (con un segreto). Se valido, sa che è autentico e legge `user_id=7` dal token stesso. Niente lookup nel database.

Pro: scalabile (no DB lookup ad ogni richiesta), stateless. **Contro**: revoca difficile (un JWT valido è valido finché scade), token più grandi.

Supabase e NextAuth usano JWT di default oggi.

Il bug classico: login si comporta diverso in locale e produzione

Storia tipica: «In locale il login funziona. In produzione l'utente fa login ma viene rimandato al login dopo il redirect alla dashboard.»

Cause possibili:

1. **Cookie `Secure` su HTTP**: in locale usi HTTP `localhost:3000`, ma il cookie ha flag `Secure` → il browser non lo invia. Soluzione: in dev usa `Secure: false`, in prod `Secure: true`.
2. **Dominio del cookie**: il cookie è impostato per `app.miosito.it` ma il redirect va a `miosito.it`. Browser lo blocca.
3. `SameSite: Strict` **con redirect cross-site**: usi un OAuth provider che ti rimanda al sito → cookie non inviato.

Diagnosi: apri DevTools → Application → Cookies. Vedi se i cookie ci sono, hanno le flag giuste.

Cosa ti porti via

1. HTTP è **stateless**: cookie + sessioni servono per simulare uno stato.
2. **Flag fondamentali**: `HttpOnly` (no JS), `Secure` (solo HTTPS), `SameSite`, `Max-Age`.
3. **Sessione lato server** (DB lookup) o **JWT** (token autocontenuto firmato).
4. **Bug classico**: cookie con `Secure` non funziona in HTTP. In locale usa HTTPS o disattiva `Secure` in dev.

7.6 — CORS in 3 paragrafi (l'errore più odiato spiegato)

Cosa è CORS

CORS sta per **Cross-Origin Resource Sharing**. È un meccanismo di sicurezza del browser.

Regola: un sito su `miosito.it` non può fare richieste JavaScript verso `api.altrosito.it` **a meno che il secondo non lo autorizzi esplicitamente**.

Perché esiste? Senza CORS, qualunque sito malevolo che apri potrebbe fare richieste JavaScript verso il tuo home banking, sfruttando i cookie già loggati. Catastrofe. CORS impedisce questo.

Quando CORS NON è un problema

Se frontend e backend sono **sullo stesso dominio**, CORS non si pone:

- `miosito.it/index.html` chiama `miosito.it/api/users` → **same-origin**, nessun problema.

In **Next.js full-stack**, frontend e backend sono nello stesso progetto e quindi lo stesso dominio. Niente CORS.

Quando CORS È un problema

Frontend e backend su **domini diversi**:

- `miosito.it` chiama `api.miosito.it` → **different-origin**, CORS attivo.
- `miosito.it` chiama `api.openai.com` → **different-origin**.
- `localhost:3000` chiama `localhost:8080` → **different-origin** (porta diversa = origin diverso!).

In questi casi, il browser fa una richiesta speciale chiamata **preflight** (`OPTIONS`) prima della richiesta vera. Chiede al server: «autorizzi `miosito.it` a chiamarmi?». Se il server risponde con i giusti header, il browser fa la richiesta vera. Se no, blocca tutto.

Configurare CORS lato server

Il server deve rispondere con questi header:

```
Access-Control-Allow-Origin: https://miosito.it
Access-Control-Allow-Methods: GET, POST, PATCH, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Allow-Credentials: true
```

In Express (Node.js):

```
import cors from 'cors';
app.use(cors({
  origin: 'https://miosito.it',
  credentials: true,
}));
```

In Laravel: `config/cors.php`. In Django: `django-cors-headers`. Ogni framework ha il suo modo.

Errori CORS che vedrai

```
Access to fetch at 'https://api.miosito.it' from origin
'https://miosito.it'
has been blocked by CORS policy: No 'Access-Control-Allow-
Origin' header...
```

Quando lo vedi, **è sempre lo stesso problema**: il server non sta rispondendo con `Access-Control-Allow-Origin` corretto.

> **ATTENZIONE**
>
> **Mai usare** `Access-Control-Allow-Origin: *` in produzione con autenticazione. Significa «autorizza chiunque». Buco di sicurezza enorme. Specifica sempre i domini esatti.

Il bug classico in produzione

Storia tipica: «In locale tutto funziona. In produzione: errore CORS.»

Causa: in locale entrambi su `localhost`, in produzione frontend su `miosito.it` e backend su `api.miosito.it`.

Fix:

1. Identifica il backend
2. Configuralo per accettare il dominio del frontend in produzione
3. Aggiungi `https://miosito.it` (e magari `http://localhost:3000` per dev) all'allow list

> **PROMPT-READY**
>
> **Prompt-ready**: «Configura CORS sul backend Express per accettare richieste da `https://miosito.it` (produzione) e `http://localhost:3000` (dev). Includi credentials per i cookie.»

Cosa ti porti via

1. **CORS** = il browser blocca chiamate cross-origin a meno che il server destinatario non autorizzi esplicitamente.
2. **Same-origin** (Next.js full-stack) = niente CORS. **Cross-origin** = CORS attivo.
3. Per sbloccare: il **server** deve mandare `Access-Control-Allow-Origin: <dominio frontend>`.
4. **Mai** `*` in produzione con autenticazione. Sempre dominio specifico.

Chiusura del Modulo 7

Questo era il modulo tecnicamente più denso del libro. Adesso conosci a fondo i meccanismi che separano «funziona in locale» da «funziona in produzione»:

- DNS — come il dominio si traduce in IP
- HTTPS — perché serve, come si configura
- Nginx — il routing davanti alle tue app
- Variabili d'ambiente — perché esistono e i loro tre file
- Cookie e sessioni — come il sito ricorda chi sei
- CORS — l'errore più odiato e come fixarlo

Quando vedrai un problema in produzione, sai dove guardare. È esattamente il **livello 2** di cui parlavamo nel Modulo 1.

Nel **Modulo 8** mettiamo tutto insieme: come usare questa conoscenza per parlare a Claude in modo che lui generi codice migliore alla prima iterazione.

🎯 Mini-quiz di autovalutazione

1. **Hai cambiato un record DNS un'ora fa. Il tuo amico vede ancora il vecchio sito. Perché?**
2. **Vero o falso: HTTPS è ancora opzionale per siti piccoli o personali.**
3. **Il tuo sito Next.js gira sulla porta 3000. L'utente arriva su `miosito.com` (porta 443 HTTPS). Cosa fa nginx?**
4. **Hai messo `OPENAI_API_KEY=sk-...` in una variabile `NEXT_PUBLIC_OPENAI_KEY` in Next.js. Cosa è successo di disastroso?**
5. **Vedi in console: `blocked by CORS policy`. Dove devi mettere mano?**

Risposte

1. **Cache DNS**: il TTL del record precedente non è ancora scaduto. Il resolver del tuo amico (o del suo OS, o del browser) ha memorizzato la vecchia risposta. Si risolve da solo quando il TTL scade. Per cambi importanti, abbassa il TTL prima.
2. **Falso**. HTTPS è obbligatorio per qualsiasi sito moderno. Senza, browser warning, Google penalty, cookie `Secure` non funzionano. Let's Encrypt rende HTTPS gratis. Niente scuse.
3. **Reverse proxy**: nginx riceve la richiesta su 443 (HTTPS), la inoltra al Next.js sulla porta 3000 internamente. Aggiunge gli header (Host, X-Forwarded-For), riceve la risposta, la rimanda al browser HTTPS. L'utente non sa nemmeno che esiste la porta 3000.
4. La tua **API key Stripe/OpenAI** è ora **visibile a chiunque ispezioni il sito**. È nel JavaScript del browser. Bot la rubano in pochi minuti. Devi: (1) cambiare immediatamente la API key, (2) togliere il `NEXT_PUBLIC_`, (3) usarla solo in Server Components o API routes.
5. **Sul backend**, non sul frontend. È il backend che deve mandare l'header `Access-Control-Allow-Origin` con il dominio del frontend. Sul frontend non c'è niente da fare: è il browser che blocca, e accetta solo se il server autorizza.

Modulo 7 redatto: 2026-04-25 · Versione 1.0 · 22 pagine · 5500 parole

Modulo 8 — Parlare a Claude come uno sviluppatore

SUGGERIMENTO

5 capitoli · 16 pagine · 1 ora
Pre-requisiti: tutti i moduli precedenti. Hai il vocabolario. Adesso lo applichi.

Mettiti comodo

Sei arrivato all'ultimo modulo. Hai imparato la mappa del sistema (moduli 0-1), la divisione client-server (modulo 2), l'anatomia di una pagina (modulo 3), il frontend (modulo 4), il backend (modulo 5), i database (modulo 6), il deploy (modulo 7).

Adesso **applichiamo tutto** al modo di parlare a Claude. È un lavoro di traduzione: prendere quello che hai in testa e dirlo in modo che Claude generi codice migliore alla prima iterazione.

Cinque capitoli:
1. Il problema del «fai una cosa bella»
2. Descrivere il layout con nomi corretti
3. Descrivere il comportamento con nomi corretti
4. Spiegare un bug a Claude senza sapere il bug
5. Il ciclo debug del vibecoder serio

8.1 — Il problema del «fai una cosa bella»

Anatomia di un prompt vago

> **SUGGERIMENTO**
>
> «Fai una pagina di login carina.»

Cosa succede quando mandi questo a Claude?

Claude **deve indovinare**. Quale framework? Quale stile? Quali campi? Validazione? OAuth o credenziali? Cookie o JWT? Come gestisce gli errori? Layout mobile? Animazioni? Persistenza di sessione?

20 decisioni, tutte prese da Claude per te. Il risultato:

- 70% delle volte: una versione **generica e plausibile**, che non corrisponde al tuo progetto reale
- 30% delle volte: qualcosa che **non funziona** col tuo stack, che devi rifare

In entrambi i casi: **2-5 iterazioni** per arrivare al risultato giusto.

Anatomia di un prompt informato

> **SUGGERIMENTO**
>
> «Aggiungi una pagina di login al mio progetto Next.js 14 con NextAuth v5 e provider Credentials. Form con email + password, validazione client+server con Zod, errori inline sotto i campi. Stile Tailwind, palette terra (vedi `globals.css`). Il submit chiama l'endpoint `/api/auth/callback/credentials` esistente. Gestisci stato loading sul bottone e errore 401 mostrando "Email o password non corretti".»

Lo stesso obiettivo, prompt diverso. Claude:

- Non indovina niente — ha tutto specificato
- Genera codice che **si integra** col progetto esistente
- Genera **al primo colpo** quello che vuoi

Differenza: 1 iterazione invece di 5.

La regola d'oro

Più contesto dai, meno iterazioni servono. **È matematica.**

Ogni dettaglio omesso = una decisione che Claude prende da solo = una probabilità che la sua decisione non sia quella giusta = un'iterazione di correzione.

Il template di prompt informato

Quando devi chiedere un componente o feature, struttura così:

1. **Stack**: che framework, librerie, versioni stai usando
2. **Cosa esiste già**: file/componenti rilevanti del progetto
3. **Cosa vuoi**: la feature in linguaggio funzionale
4. **Come deve apparire**: lessico UI preciso (vedi 8.2)
5. **Come deve comportarsi**: lessico tecnico preciso (vedi 8.3)
6. **Edge case**: errori, stati vuoti, validazioni
7. **Cosa NON fare**: vincoli espliciti

Esempio compatto:

```
Stack: Next.js 14, Drizzle, Tailwind, Postgres su Supabase.

Esiste: tabella `users` con id, email, password_hash.
Endpoint /api/auth/login già funzionante.

Voglio: pagina /login con form.

Layout: Hero centrato 400px width, card bianca con shadow
soft.
Form con due input (email, password con toggle show/hide),
checkbox "ricordami",
bottone primary full-width "Accedi", link sotto "Non hai un
account? Registrati".

Comportamento: validazione client onChange con Zod (email
valida, password min 8).
Submit fetch POST a /api/auth/login. Stato loading sul
bottone (spinner + "Accesso..."),
errori 401 mostrati sopra il form ("Email o password
errati").
Su 200, redirect a /dashboard.
```

```
Edge case: campo vuoto = errore inline. Rete giù = toast
rosso.

NON: nessun OAuth provider per ora. NON usare 'use client'
se non strettamente necessario.
```

Lungo? 12 righe. Tempo di scriverlo: 90 secondi. Tempo risparmiato in iterazioni: 30 minuti.

Cosa ti porti via

1. **Prompt vago** = Claude indovina = 5 iterazioni.
2. **Prompt informato** = Claude esegue = 1 iterazione.
3. **Più contesto = meno iterazioni**. È matematica.
4. Usa il **template a 7 punti**: stack, esistente, voglio, layout, comportamento, edge case, NON.

8.2 — Descrivere il layout con nomi corretti

Il vocabolario UI applicato

Nel Modulo 3 hai imparato il **lessico operativo** dei componenti UI. Adesso lo usi.

Confronta le due colonne:

❌ Vago	✅ Preciso
«Un menu che esce»	«Drawer mobile dal basso, dismissibile col swipe down»
«Una scritta di conferma»	«Toast verde in basso a destra, “Salvato”, auto-dismiss 4s»
«Una barra in cima»	«Navbar sticky con logo a sinistra, 4 link al centro, CTA primary a destra»
«Un blocco con la roba»	«Grid 3 colonne desktop / 1 mobile, ogni card con immagine 4:3, titolo, prezzo, bottone “Aggiungi”»

❌ Vago	✅ Preciso
«Un popup di conferma»	«Modal centrale con overlay scuro, testo “Sei sicuro?”, bottoni Annulla (ghost) e Elimina (destructive rosso)»
«Una scelta tra opzioni»	«Radio group esclusivo verticale, 3 opzioni, valore selezionato ha bordo accent»

La colonna destra è 3-4 volte più lunga ma è **infinite volte più chiara**.

Componenti che il vibecoder confonde più spesso

Modal vs Drawer

- **Modal**: appare al **centro** della pagina, overlay scuro dietro, **bloccante** (devi interagire prima di tornare).
- **Drawer**: entra **dal bordo** (sinistra/destra/alto/basso), tipicamente **non bloccante** (puoi anche cliccare fuori per chiudere).

Confonderli è il bug più frequente. Se vuoi un menu mobile, è un **drawer**. Se vuoi una conferma critica, è un **modal**.

Toast vs Snackbar vs Banner

- **Toast** / **Snackbar**: notifica **temporanea** (auto-dismiss), in un angolo della pagina.
- **Banner**: notifica **persistente**, in cima alla pagina, dismissibile dall’utente con X.

Se devi avvisare «Salvato!» → toast. Se devi dire «Manutenzione il 15 maggio» → banner.

Dropdown vs Select

- **Select** (input form): l’utente sceglie **un valore** da una lista. È un input.
- **Dropdown**: menu di **azioni cliccabili**. Non è un input.

Se l’utente sceglie «Italia» tra paesi → select. Se l’utente clicca «Modifica/Elimina/Esporta» → dropdown.

Tooltip vs Popover

- **Tooltip**: piccolo testo al **hover** del mouse. Solo informativo, non interattivo.
- **Popover**: come tooltip ma **al click**, può contenere bottoni e link interattivi.

Vuoi spiegare un’icona → tooltip. Vuoi un pannello quick-action → popover.

Esempio applicato: la dashboard di SafeTrack

Caso reale. Vuoi una dashboard tipo SafeTrack: sidebar laterale, area centrale con KPI in card, lista di «alert recenti» sotto, e un bottone «Nuovo alert» che apre un modal.

❌ Vago:

> **SUGGERIMENTO**
>
> «Fai una dashboard con menu laterale e dentro le notifiche e un bottone per aggiungere.»

✅ Preciso:

> **SUGGERIMENTO**
>
> «Dashboard con **sidebar nav** fissa sinistra (240px, collassabile su mobile in **hamburger drawer**). **Main content** con padding 32px:
> - **Grid** di 4 **card KPI** in alto (4 colonne desktop, 2 tablet, 1 mobile), ogni card con icona, valore grande, label
> - Sotto, **sezione "Alert recenti"** con **lista** (max 10), ogni riga con avatar tipo, titolo, timestamp relativo
> - **Bottone primary** sticky in alto a destra "Nuovo alert" che apre **modal** con form di creazione.»

Differenza: il primo prompt produce 4 iterazioni di chiarimenti. Il secondo produce codice subito utilizzabile.

Cosa ti porti via

1. Usa il **lessico UI preciso** del Modulo 3 nei prompt.
2. Distingui sempre **modal vs drawer**, **toast vs banner**, **dropdown vs select**, **tooltip vs popover**.
3. Specifica sempre **comportamento responsive** (1 col mobile, 2 tablet, 4 desktop).
4. Costruire prompt visivi precisi è **3-4× più lungo**, ma riduce le iterazioni del **70%**.

8.3 — Descrivere il comportamento con nomi corretti

Il vocabolario tecnico applicato

Stesso esercizio del 8.2, ma per il comportamento. Con il vocabolario dei moduli 4-7:

❌ Vago	✅ Preciso
«Carica le cose»	«Fetch GET /api/products on mount con `useEffect`, mostra skeleton loader durante il caricamento»
«Salva quando preme invio»	«Submit handler con preventDefault, validation Zod, fetch POST, toast success o error»
«Cambia pagina»	«router.push("/dashboard") con replace=true (no back button)»
«Riempi quando arriva»	«useState con stato dei dati, useEffect per il fetch iniziale, refetch on focus con SWR»
«Aggiorna automaticamente»	«Polling ogni 30s con setInterval cleanup, oppure WebSocket subscribe ai cambi»
«Validalo»	«Validation client con Zod schema, mostrare errori sotto i campi con aria-invalid, validazione anche server-side per sicurezza»

Termini tecnici che cambiano tutto nel prompt

fetch / mutate / refetch
- Fetch: prendi i dati una volta
- Mutate: modifica i dati e re-fetch ottimisticamente
- Refetch: scarica di nuovo i dati esistenti

onMount / onChange / onSubmit
- Specifica quando l'azione si scatena. Sono diversi.

state / props / context
- State: dati interni al componente
- Props: dati passati dall'esterno
- Context: dati condivisi tra componenti senza prop drilling

redirect / router.push / window.location

- Redirect: cambio pagina lato server (Next.js: `redirect()` in server component)
- router.push: navigazione SPA lato client (Next.js: dopo submit)
- window.location: full page reload (raro, da evitare)

optimistic update / pessimistic update

- Optimistic: aggiorna la UI subito, rollback se fallisce
- Pessimistic: aspetta la conferma del server, poi aggiorna

Esempio: form di registrazione con tutti gli stati

Compara:

❌ Vago:

> **SUGGERIMENTO**
>
> «Form di registrazione che salva l'utente.»

✅ Preciso:

SUGGERIMENTO

«Form di registrazione con campi email, password, conferma password.

Comportamento:
- Validation **client onChange con Zod** (email valida, password min 12 char, conferma uguale)
- Errori **inline sotto i campi** con `aria-invalid`
- **Submit handler**: preventDefault, valida, fetch POST a `/api/auth/register`
- **Stato loading**: bottone disabled + spinner + "Creazione account..."
- **Success (201)**: toast verde "Account creato!", router.push("/dashboard")
- **Error 422** (validation server): mostra errori inline (server può dire "email già registrata")
- **Error 500**: toast rosso "Errore del server, riprova tra qualche minuto"
- **Network error** (fetch fallisce): toast rosso "Connessione assente, controlla la rete"

Sicurezza: email check anche server-side (ovviamente). Password hashata con bcrypt cost 12. Mai loggare la password in chiaro.»

Lungo? Sì. Risultato: codice **production-ready** alla prima iterazione, con tutti gli edge case gestiti.

Cosa ti porti via

1. Usa termini tecnici precisi: **fetch, mutate, onMount, redirect, optimistic**.
2. Descrivi sempre **tutti gli stati**: idle, loading, success, error, edge case.
3. Le **frasi di sicurezza** («hashed password», «validation server-side») attivano in Claude pattern di codice migliori.
4. Più precisione = codice **production-ready** alla prima iterazione.

8.4 — Spiegare un bug a Claude senza sapere il bug

La situazione

Hai un bug. Il sito non funziona, vedi un errore, non sai cosa significhi. Apri Claude e... cosa scrivi?

❌ Pessimo:

> **SUGGERIMENTO**
>
> «non funziona»

❌ Medio:

> **SUGGERIMENTO**
>
> «il login dà errore»

✅ Bene:

> **SUGGERIMENTO**
>
> «[contesto stack] + [errore esatto] + [quando succede] + [cosa hai già provato]»

I 4 ingredienti del bug report perfetto

1. **Contesto stack**

Cosa stai usando:

- Framework e versione (Next.js 14 / 15 / 16)
- Database (Supabase / Neon / Postgres locale)
- Auth (NextAuth / Supabase Auth / Clerk)
- Hosting (Vercel / VPS / Railway)
- Dominio (production / dev / localhost)

2. **Errore esatto**

NON: «errore di rete». SÌ: l'errore copiato pari pari.

```
Network: POST https://miosito.it/api/auth 500 (Internal
Server Error)
Console: TypeError: Cannot read property 'session' of
undefined
  at handleLogin (login.tsx:42:18)
Server log: Error: NEXTAUTH_SECRET is not defined
```

Tre log diversi (browser console, network, server) che dicono **la stessa cosa** in modi diversi. Claude legge tutti e tre e capisce.

3. **Quando succede**

- Sempre? Solo a volte?
- In locale? In produzione? Su mobile?
- Per tutti gli utenti o solo per alcuni?
- Dopo un'azione specifica?

> **SUGGERIMENTO**
>
> «Succede solo in produzione, in locale funziona. Solo dopo che l'utente clicca submit. Su tutti i browser.»

4. **Cosa hai già provato**

Per non far ripetere a Claude soluzioni inutili:

> **SUGGERIMENTO**
>
> «Ho già controllato che la variabile NEXTAUTH_SECRET sia settata in Vercel (sì lo è). Ho riavviato il deploy. Ho fatto pulizia dei cookie.»

Esempio completo: bug report perfetto

SUGGERIMENTO

Stack: Next.js 14, NextAuth v5 con provider Credentials, Supabase per il DB. Deployato su Vercel, dominio `miosito.it` con SSL.

Errore:
- Browser console: nessun errore
- Network tab: `POST /api/auth/callback/credentials → 200 OK`, ma redirect a `/dashboard` poi torna a `/login`
- Server log Vercel: `[next-auth][warn][NO_SESSION_FOUND]`

Quando: solo in produzione (`miosito.it`). In locale (`localhost:3000`) funziona perfettamente. Tutti i browser. Sempre dopo il submit del login.

Già provato:
- Variabile NEXTAUTHSECRET settata in Vercel √
- Variabile NEXTAUTHURL settata su `https://miosito.it` √
- Cookie clear, riprovato √
- Verificato che Supabase response sia corretta (utente autenticato, sessione creata) √

Sospetto: cookie httpOnly+Secure non viene impostato correttamente sul dominio production, oppure il middleware NextAuth non lo legge. Ma non sono sicuro.

Con questo prompt, Claude in 1 iterazione ti porta alla causa probabile (config NextAuth `useSecureCookies: true` e `cookies.sessionToken.options.domain` mal configurato).

Cosa ti porti via

1. Mai «non funziona». Sempre **i 4 ingredienti**: stack, errore esatto, quando, cosa già provato.

2. **Copia gli errori pari pari** dalla console, dal network, dai log. Non riassumere.
3. **Distingui** sempre tra in locale funziona e in produzione no — è la prima domanda diagnostica.
4. **Già provato** è importante quanto il bug stesso. Risparmia 5 risposte di Claude inutili.

8.5 — Il ciclo debug del vibecoder serio

Algoritmo di debug

Quando hai un bug, segui questo flusso. **Non saltare passi**.

Step 1 — Riproduci il bug

Riesci a farlo succedere a comando? Sì → bene, hai il riproducibile. No → prima cosa: capire le condizioni esatte. Senza riproducibile non si debugga.

Step 2 — Localizza il bug

Il bug è:
- **Frontend**? Apri DevTools, errori in console.
- **Backend**? Apri i log del server.
- **Database**? Verifica le query nel pannello Supabase / log DB.
- **Network**? Apri il network tab, status code delle richieste.

In 9 casi su 10 puoi localizzare il bug **in 30 secondi** con DevTools. È il tuo strumento principale.

Step 3 — Isola la variabile

Prima di chiedere a Claude, isola **esattamente quale parte** del sistema ha problemi. Tre tecniche:
- In locale funziona, in produzione no: ambiente.
- Funzionava ieri, oggi no: cosa è cambiato? Git log.
- Funziona per X, non per Y: dato. Confronta i due casi.

Step 4 — Formula l'ipotesi

Prima di chiedere a Claude, scriviti la tua ipotesi:

> **SUGGERIMENTO**
>
> Penso che sia X perché Y. Per verificarlo dovrei controllare Z.

Spesso, scrivendo l'ipotesi, ti accorgi della soluzione da solo. Sennò, la dai a Claude come spunto.

Step 5 — Chiedi a Claude

Con i 4 ingredienti del bug report (8.4) + la tua ipotesi.

Step 6 — Verifica

Applica la soluzione di Claude. Verifica che il bug è davvero risolto (non solo nascosto). Se è risolto, ✓. Se no, riformula con più info.

3 esempi reali di applicazione

Esempio 1 — Errore CORS in produzione

- Riproduzione: ✓ ogni volta in production, mai in locale.
- Localizzazione: console browser → «blocked by CORS policy»
- Variabile isolata: dominio diverso tra frontend (Vercel) e backend (Aruba VPS)
- Ipotesi: backend non ha CORS configurato per accettare il dominio Vercel
- A Claude: stack + errore esatto + ipotesi → fix in 1 iterazione

Esempio 2 — 401 Unauthorized in produzione

- Riproduzione: ✓ solo in production, dopo qualche minuto di sessione
- Localizzazione: network tab → `/api/profile` ritorna 401
- Variabile isolata: il cookie session esiste in DevTools ma non viene mandato
- Ipotesi: cookie `Secure` su HTTPS production, ma il flag non è settato giusto
- A Claude: il prompt completo → soluzione `cookies.sessionToken.options` di NextAuth

Esempio 3 — Query DB lenta

- Riproduzione: ✓ ogni caricamento della dashboard
- Localizzazione: server log → «Query took 4523ms»
- Variabile isolata: query specifica con JOIN su tre tabelle senza indici
- Ipotesi: manca un indice sulla foreign key `orders.user_id`
- A Claude: con la query EXPLAIN analyze → fix con migration di indice

La differenza tra vibecoder ingenuo e serio

Vibecoder ingenuo	Vibecoder serio
«non funziona» → Claude → 8 tentativi a caso	DevTools → log → ipotesi → Claude → fix
2 ore di debug confuso	15 minuti di debug strutturato
Bugs che tornano	Bugs risolti alla radice
«non capisco perché»	«è successo X perché Y»

Il libro ti ha portato a essere il secondo. Adesso è esercizio: fallo per le prossime 10 issue che ti capitano. Diventerà naturale.

Cosa ti porti via

1. **Algoritmo di debug**: Riproduci → Localizza → Isola → Ipotizza → Chiedi → Verifica.
2. **DevTools** è il tuo strumento principale. Console, Network, Application tabs.
3. Prima di chiedere a Claude, **scrivi la tua ipotesi**. Spesso ti accorgi della soluzione.
4. Bug in produzione vs locale = **prima domanda**. Quasi sempre la causa è ambiente.

Chiusura del Modulo 8 (e del libro)

Hai finito il libro.

Ti ricordi com'eri all'inizio? Forse non sapevi cosa è un cookie. Sentivi parlare di CORS e ti scappava l'emoji 🤯. Davanti a un errore 500 chiudevi gli occhi.

Adesso:

- Sai mappare un'app web (browser, server, DB)
- Sai distinguere client da server e che linguaggi vivono dove
- Sai descrivere componenti UI con i nomi giusti
- Sai cosa è un'API REST, un cookie, un certificato SSL
- Sai cosa fare quando la produzione si rompe

- Sai parlare a Claude in modo che generi codice sensato alla prima iterazione

Non sei diventato un programmatore senior. Ma sei diventato un **vibecoder serio**: uno che capisce cosa sta costruendo, sa diagnosticare quando si rompe, e parla con cognizione di causa con sviluppatori veri.

Il libro finisce, il lavoro vero inizia. Continua a costruire. Ogni progetto è un'occasione di rinforzare quello che hai imparato.

Una cosa importante prima di andare:

> **SUGGERIMENTO**
>
> **Ogni volta che fai una correzione a un bug, scriviti cosa hai imparato.**

Tieni un file `IMPARATO.md` nei tuoi progetti. Ogni volta che risolvi un bug serio, una riga: «Bug X causato da Y, risolto con Z. Lezione: ...». In 6 mesi avrai un libretto personale che vale più di 10 corsi.

Buon vibecoding. Sul serio.

🎯 Mini-quiz finale di tutto il libro

1. **Sai dirmi in 3 frasi cosa fa un server in un sistema web?**
2. **Cosa rende un prompt «informato» diverso da uno «vago»?**
3. **Quando vedi un errore 500, dove guardi prima?**
4. **Differenza tra modal e drawer in una frase.**
5. **La regola d'oro della divisione client/server, in una frase.**

Risposte

1. Il server riceve richieste HTTP dai client (browser, app), esegue codice (verifica auth, query al DB, chiama API esterne), e ritorna una risposta. Vive su una macchina dietro a un firewall, ha accesso a segreti, e fa da filtro sicuro tra il browser dell'utente e i dati.
2. Il prompt **informato** specifica stack, contesto esistente, layout preciso (con lessico UI corretto), comportamento esatto (con stati e edge case),

e cosa NON fare. Il prompt **vago** lascia 20 decisioni a Claude da prendere indovinando, e produce 5× più iterazioni.

3. Nei **log del server**, non nel browser. Il 500 è un errore lato server (codice esploso, eccezione non gestita). Il browser non sa cosa è successo, sa solo che qualcosa è andato storto. Per capire, serve il log server (Vercel Functions log, PM2 logs, ecc.).
4. **Modal è una finestra centrale con overlay che blocca il resto della pagina (per conferme critiche). Drawer è un pannello che entra da un bordo, tipicamente non bloccante (per menu mobile, filtri).**
5. **Tutto ciò che richiede un segreto o controlla autorizzazioni deve passare dal server. Sempre.**

Se hai sbagliato anche solo una di queste 5, hai imparato qualcosa di valore. Se non hai sbagliato niente: bravo.

Modulo 8 redatto: 2026-04-25 · Versione 1.0 · 16 pagine · 4500 parole

SUGGERIMENTO

Fine del libro.
Glossari e appendici nelle pagine successive.
Buon vibecoding.

PARTE II

I 5 capitoli killer

I quick wins che da soli giustificano il prezzo del libro.

Perché il tuo sito è bellissimo in localhost e si rompe in produzione

SUGGERIMENTO

Quick Win 1 di 5 · Tempo di lettura: 12 minuti · Modulo collegato: 4 (il viaggio del click)
Versione standalone pubblicabile come articolo blog. Il capitolo nel libro è la stessa cosa, con riferimenti incrociati ai moduli.

Il momento esatto in cui ti rendi conto

Sono le 22:47 di un martedì. Hai appena fatto deploy della prima versione del sito che hai promesso al cliente. In localhost tutto funzionava: il login, il form contatti, il salvataggio dei prodotti. Apri il dominio in produzione, clicchi «accedi», inserisci email e password, premi invio.

Niente.

Il browser si ferma per un secondo, poi una scritta rossa appare in console che non avevi mai visto prima. Il cliente ti scrive: «Funziona?». Tu rispondi «sì sì, qualche test finale».

Apri ChatGPT o Claude e scrivi: «il login non funziona in produzione, aiutami».

Otto messaggi dopo, dopo aver provato cinque modifiche al codice che peggiorano la situazione, ti chiedi se non sia il caso di chiamare un amico programmatore vero. Vai a dormire alle due. Il giorno dopo riapri il PC pensando di aver dimenticato qualcosa di banale. **Avevi dimenticato qualcosa di banale.**

Questo capitolo ti racconta cosa, perché, e — soprattutto — come riconoscerlo in cinque minuti la prossima volta.

Cosa pensa il vibecoder

> **SUGGERIMENTO**
>
> «Se in localhost funziona, in produzione deve funzionare. È lo stesso codice.»

Errato. Il codice è lo stesso, ma il **contesto in cui gira** è completamente diverso. Localhost e produzione sono due ambienti che si somigliano ma non sono identici, e ogni differenza tra i due è un punto in cui qualcosa può rompersi.

Pensa a una macchina. La porti dal meccanico, la prova in officina, frena bene, accelera bene. Tu la porti a casa e ti accorgi che il navigatore non funziona. Cosa è cambiato? Non la macchina. È cambiato il GPS, perché la spia del GPS è collegata al satellite, non al meccanico. Il tuo sito ha decine di «satelliti» invisibili: il dominio, l'HTTPS, le variabili d'ambiente, il database remoto, il CORS, le porte del firewall. In localhost non hai bisogno di nessuno di questi. In produzione li usi tutti.

Cosa succede davvero

Quando il tuo sito è in **localhost** (`http://localhost:3000` o `http://127.0.0.1:3000`):

- Il server gira sulla tua macchina, sulla porta 3000
- Frontend e backend stanno **sullo stesso indirizzo** (entrambi su `localhost:3000`)
- Il browser riconosce localhost come «ambiente fidato» e fa eccezioni di sicurezza
- Le variabili d'ambiente le legge dal tuo `.env.local`
- Il database è un container Docker che gira sul tuo PC, oppure un'istanza Supabase di sviluppo
- HTTPS non c'è, viaggi in HTTP semplice (i cookie non hanno bisogno della flag `Secure`)
- Non ci sono altri utenti, non c'è traffico, non c'è cache CDN

Quando il sito è in **produzione** (es. `https://miosito.com`):

- Il server gira su una macchina remota (VPS Aruba, Vercel, Railway), porta interna 3000 ma esposta come 443 (HTTPS)
- Il frontend e il backend potrebbero essere su domini diversi (es. `miosito.com` e `api.miosito.com`)
- Il browser applica tutte le regole di sicurezza standard, senza eccezioni
- Le variabili d'ambiente le legge dal pannello del provider, non dal tuo `.env`
- Il database è un'istanza separata, su un server cloud, raggiungibile solo via stringa di connessione configurata
- HTTPS è obbligatorio, i cookie devono avere flag `Secure` per funzionare
- Esiste traffico, esiste cache CDN, esiste rate limiting, esiste fail2ban

Già qui hai sei differenze. La maggior parte dei «non funziona in produzione» è una di queste sei.

I 5 motivi per cui il tuo sito si è rotto (in ordine di frequenza)

1. Le variabili d'ambiente non sono settate

Il tuo `.env.local` ha 12 righe. `OPENAI_API_KEY=sk-...` , `DATABASE_URL=postgres://...` , `NEXTAUTH_SECRET=...` . In localhost le legge au-

tomaticamente. In produzione, su Vercel/Netlify/Aruba, **devi inserirle manualmente** nel pannello del servizio. Se ne dimentichi una, l'app crasha o si comporta in modo imprevedibile.

Sintomo: l'app si avvia, la home si vede, ma appena tocchi una funzione che usa quella variabile (es. login, fetch da API esterna, query DB) ricevi un 500 senza spiegazione, oppure un comportamento «vuoto» senza errore.

Diagnosi rapida: apri i log del server. Cerca righe tipo `undefined is not a function` o `Cannot read property 'X' of undefined` o `connection refused`. Se vedi qualcosa che si chiama proprio come una tua variabile (`OPENAI_API_KEY` , `DATABASE_URL`), hai trovato il colpevole.

Soluzione: vai nel pannello del provider di hosting → sezione «Environment Variables» → aggiungi la riga mancante → redeploy. Su Vercel basta un click. Su Aruba VPS è un edit del file `.env.production` e un restart PM2.

2. CORS

Il tuo frontend è su `https://miosito.com` e il tuo backend è su `https://api.miosito.com`. Il browser, per sicurezza, vieta al primo di parlare col secondo se il secondo non lo autorizza esplicitamente. È il **CORS** (Cross-Origin Resource Sharing).

In localhost non succede mai perché frontend e backend stanno entrambi su `localhost:3000` — stesso «origine», nessun problema. In produzione, appena il dominio cambia, il browser chiede al backend «ehi, autorizzi anche miosito.com a parlarti?». Se il backend non risponde con il giusto header (`Access-Control-Allow-Origin`), il browser blocca la richiesta e tu vedi un errore rosso in console che inizia con «has been blocked by CORS policy...».

Sintomo: vedi un errore esplicito «CORS policy» o «Access-Control-Allow-Origin» nella console del browser (non nei log del server, è il browser che lo blocca prima che la richiesta arrivi). Spesso questo è il **primo** errore che si manifesta dopo un deploy.

Soluzione: nel codice del backend devi configurare CORS per accettare richieste da `https://miosito.com`. In Express è una riga (`app.use(cors({ origin: 'https://miosito.com' }))`). In Next.js API routes è

qualche riga in più. **NON usare mai** `*` in produzione: lascia entrare chiunque, è un buco di sicurezza.

3. La build non è quella che pensi

Tu hai modificato un file alle 22:30. Hai fatto `git push` alle 22:35. Sono le 22:50 e il sito mostra ancora la vecchia versione. **Hai fatto la build?**

Molti deploy automatici (Vercel, Netlify, GitHub Actions) ricostruiscono ogni volta che fai push, ma su VPS Aruba con PM2 devi farlo a mano: `pnpm build` e `pm2 restart`. Se non lo fai, il server gira ancora con la build precedente. La cache del browser e della CDN possono peggiorare la cosa.

Sintomo: hai modificato qualcosa di evidente (testo di un bottone, colore di un'intestazione) e il sito mostra la versione vecchia. Cancelli la cache, hard reload (Ctrl+Shift+R), niente.

Diagnosi: sul server, controlla la data di modifica della cartella `.next` (Next.js) o `dist` (Vite). Se è di ieri, non hai ricompilato.

Soluzione: `git pull && pnpm build && pm2 restart <progetto>`. Se anche dopo il browser mostra la vecchia versione, è cache CDN/browser: hard reload, oppure aggiungi `?v=2` all'URL come query param di test.

4. Cookie e HTTPS si tradiscono a vicenda

Il tuo login funzionava in localhost. In produzione l'utente fa login, viene reindirizzato alla dashboard, e... viene rimandato al login. Continua all'infinito.

Quasi sempre è la flag `Secure` del cookie di sessione. In produzione il sito è HTTPS, il cookie viene impostato come `Secure` (cioè «viaggia solo su HTTPS»). Se per qualche motivo c'è un redirect su HTTP, il cookie non viene inviato e il server pensa che l'utente non sia loggato. Loop infinito.

Sintomo: login che funziona ma poi si comporta come se non avessi fatto login. Oppure login che funziona oggi e domani sei sloggato.

Soluzione: forzare HTTPS dovunque (redirect `http://` → `https://` in nginx). Verificare che il dominio del cookie sia corretto (se sei su

`app.miosito.com` e il cookie è settato su `miosito.com`, non funziona). Verificare le configurazioni di NextAuth/Supabase per `useSecureCookies` e `cookieDomain`.

5. Il database remoto non è lo stesso del database locale

In localhost hai un Postgres che gira sul tuo PC (Docker o installato direttamente). Hai 12 utenti di test, hai aggiunto la colonna `phone_number` ieri, hai fatto qualche query che ha funzionato.

In produzione il database è un'altra cosa: un'istanza Supabase, o un Postgres su Neon, o uno spuntato in un container del provider. Quel database **non ha la colonna** `phone_number` se non hai fatto la migration. Quando il codice prova a leggere `user.phone_number`, riceve `undefined`. Quando prova a scriverlo, riceve un errore SQL `column "phone_number" does not exist`.

Sintomo: errore 500 con messaggio nei log che cita una colonna o tabella inesistente. Oppure dati che non vengono salvati silenziosamente.

Soluzione: applicare le migration in produzione. Con Drizzle: `pnpm drizzle-kit push` (o `migrate`) puntando al `DATABASE_URL` di produzione. Con Prisma: `prisma migrate deploy`. Con Supabase via dashboard: SQL editor + script DDL.

Il framework «Diagnostica in 5 minuti»

Quando il sito si rompe in produzione, segui questo ordine. Non saltare passi.

Minuto 1 — Apri la console del browser (F12) Vai su Network tab. Ricarica la pagina rotta. Cerca richieste in rosso. Cliccaci sopra e guarda lo status code. **Annota il numero**: 401, 403, 404, 500, CORS error, Failed to fetch.

Minuto 2 — Apri i log del server Vercel: Functions tab → Logs. Aruba VPS: `pm2 logs <progetto> --lines 50`. Cerca riga con timestamp vicino a quando hai cliccato. **Copia l'errore completo, non solo il primo paragrafo.**

Minuto 3 — Identifica la categoria

- Errore in console che dice CORS → è CORS (motivo 2)
- 500 + log che dice «undefined» su una variabile dal nome familiare → variabili d'ambiente (motivo 1)
- 500 + log che dice «column does not exist» → migration DB (motivo 5)
- Login loop → cookie/HTTPS (motivo 4)
- Sito vecchio nonostante deploy → build (motivo 3)

Minuto 4 — Leggi la sezione corrispondente di questo capitolo Una pagina, soluzione concreta.

Minuto 5 — Applica la soluzione Una modifica alla volta. Non tutto insieme. Se la prima soluzione non funziona, **rimettila esattamente come prima** e prova la seconda.

Se in 5 minuti non capisci la categoria, è il momento di descrivere il problema a Claude. Ma ora sai descriverlo bene.

Come parlarne a Claude per ottenerlo bene

> **SUGGERIMENTO**
>
> **Prompt da NON usare** (e perché ti farà perdere 30 minuti):
> «il sito non funziona in produzione, in locale sì, aiutami»
>
> Claude non ha visibilità sul tuo ambiente, sui log, sul codice, sul provider. Risponderà con una lista generica di «controlla A, B, C» che troverai inutile. Ti farà perdere 5 iterazioni a chiarire cose che potevi dire dall'inizio.

SUGGERIMENTO

Prompt da usare (e perché ti farà ottenere la soluzione in 1 iterazione):

«Sito Next.js 16 deployato su Vercel con dominio custom HTTPS (`miosito.com`). Backend e frontend stesso progetto. Database Supabase EU. NextAuth v5 con provider credenziali.

Sintomo: l'utente fa login con successo (vedo `200 OK` su `/api/auth/callback/credentials` nel network tab), viene reindirizzato a `/dashboard` , ma `/dashboard` lo rimanda subito a `/login` . Loop infinito.

In `localhost:3000` funziona perfettamente. Cosa potrebbe essere e in che ordine controllare?»

Differenza nel risultato:

- Prima versione: Claude propone 8 cose generiche, ne provi 4, ne nessuna funziona.
- Seconda versione: Claude in genere ti chiede una verifica precisa (es. «stampa qui il valore di `cookies()` dentro la dashboard server component») perché sa già che è probabilmente un problema di cookie/secure flag su NextAuth v5.

Il pattern è sempre questo:

1. **Stack**: dichiara il framework, il provider di hosting, il database, lo strumento di auth.
2. **Sintomo preciso**: cosa fa il sito? Cosa vedi nei log? Cosa vedi nel network tab? Quale status code?
3. **Confronto localhost/produzione**: cosa è uguale, cosa è diverso?
4. **Cosa hai già provato**: per non far ripetere a Claude soluzioni già scartate.

Più contesto dai, meno iterazioni servono. Una volta che hai questo riflesso, hai trasformato Claude da «suggeritore generico» a «secondo paio di occhi specializzato».

La regola che ti porti a casa

> **SUGGERIMENTO**
>
> **Localhost mente.** Funziona in localhost vuol dire «il codice è probabilmente giusto», non «il sito è pronto per la produzione».

Le cose che si rompono in produzione si rompono per via dell'**ambiente**, non del codice. Variabili, domini, HTTPS, database remoto, build, CORS. Cinque dei sei sono cose configurate fuori dal tuo IDE, in pannelli di provider di hosting o in file `.env` che non versioni.

Per ognuna delle cinque, ora hai un sintomo, una diagnosi, una soluzione, e un prompt buono per Claude. Tienile a portata di mano la prossima volta — possibilmente prima delle 22:47.

> **SUGGERIMENTO**
>
> Ti è piaciuto questo capitolo? È uno dei 5 Quick Win del libro **Vibecoding Serio**. Gli altri 4 trattano: cosa è davvero un'API REST, perché il login dimentica l'utente (cookie e sessioni), come non perdere i dati del database, e CORS spiegato per fixarlo da solo.
> Pre-order a 19€ (early bird) → vibecodingserio.vibecanyon.com

---

Capitolo redatto: 2026-04-25 · Ultimo aggiornamento: 2026-04-25 · Versione 1.0

Cosa è davvero un'API REST e perché la tua app si pianta a chiamarla

SUGGERIMENTO

Quick Win 2 di 5 · Tempo di lettura: 12 minuti · Modulo collegato: 5
Capitolo standalone pubblicabile come articolo blog.

Il momento «boh»

Hai chiesto a Claude di collegare il tuo sito a OpenAI. Lui ha scritto qualcosa con `fetch()` e una URL strana. Funziona. Sei contento. Poi succede una di queste cose:

- L'utente clicca su «Genera testo» e non succede niente
- Il sito mostra `Error: 401`
- Tutto si blocca dopo qualche utilizzo
- In locale funziona, in produzione no

In ognuno di questi casi, il problema è lo stesso: **stai usando un'API senza capire cosa è un'API**. Risolviamo.

Cosa pensa il vibecoder

> **SUGGERIMENTO**
>
> «Un'API è una cosa magica che mi permette di parlare con un altro sito. Claude scrive il codice, io lo metto, funziona. Non capisco i dettagli.»

Capisci la prima parte (è una «porta» verso un altro sistema). La seconda parte (i dettagli) è dove vivono **tutti** i tuoi problemi.

Cosa succede davvero

Quando il tuo codice fa una «chiamata API» — per esempio a OpenAI — sta facendo una **richiesta HTTP**. Non è magia. È un messaggio strutturato che parte dal tuo server, attraversa Internet, arriva ai server di OpenAI, viene processato, e una risposta torna indietro.

Il messaggio che parte ha **3 parti**:

1. **Il verbo + URL**

— cosa vuoi fare e dove

```
POST https://api.openai.com/v1/chat/completions
```

Tradotto: «Voglio creare (POST) una chat completion all'URL X.»

I 5 verbi che incontrerai:

- **GET** = leggi qualcosa (es. lista utenti)
- **POST** = crea qualcosa (es. nuovo ordine)
- **PATCH** = modifica parzialmente
- **PUT** = sostituisci completamente
- **DELETE** = elimina

2. **Gli header**

— metadati della richiesta

```
Authorization: Bearer sk-proj-abcd1234...
Content-Type: application/json
```

Tradotto: «La mia API key è sk-proj-... — il body è in formato JSON.»

L'**Authorization** è dove metti la tua API key (la tua «carta d'identità» per quel servizio). Senza, il server risponde **401 Unauthorized**: «chi sei?».

3. **Il body**

— i dati veri

```
{
  "model": "gpt-4",
  "messages": [
    { "role": "user", "content": "Ciao!" }
  ]
}
```

In JSON. Il body è quello che mandi al servizio per dirgli cosa vuoi.

La risposta che torna indietro

Il server di OpenAI processa, e risponde con un altro messaggio HTTP:

```
200 OK
Content-Type: application/json

{
  "choices": [
    { "message": { "role": "assistant", "content": "Ciao!
Come posso aiutarti?" } }
  ]
}
```

Status code 200 = tutto ok. Il body contiene la risposta. Il tuo codice legge il body e lo mostra all'utente.

I codici di errore che non sono «errori vaghi»

Ogni risposta ha uno **status code** preciso. Imparare a leggerlo è metà del debug:

- **200 OK** = tutto bene
- **400 Bad Request** = hai mandato qualcosa di malformato (es. JSON sbagliato)

- **401 Unauthorized** = manca o è sbagliata la API key
- **403 Forbidden** = la key è valida ma non hai i permessi
- **429 Too Many Requests** = hai superato il limite (rate limit), rallenta
- **500 Internal Server Error** = bug del server di OpenAI, non colpa tua
- **503 Service Unavailable** = servizio momentaneamente giù

Quando vedi un 401, non è un «errore generico»: è il server che ti dice «chi sei?». Andresti a verificare la API key.

Perché in locale funziona e in produzione no

Il caso più comune: la tua API key è in `.env.local` sul tuo PC, ma in produzione (Vercel, Aruba) **non l'hai impostata**. Sintomo: 401 sistematico in produzione, mai in locale.

Fix: pannello Vercel → Environment Variables → aggiungi `OPENAI_API_KEY`. Redeploy. Funziona.

Il rate limit (429)

OpenAI (e ogni API) ha un **rate limit**: massimo X richieste al minuto. Se superi, riceve **429 Too Many Requests**. Non è un bug — è una protezione. Soluzioni:

- Implementa **backoff exponential**: se 429, aspetta 1s e ritenta. Se ancora 429, aspetta 2s, poi 4s, poi 8s.
- **Cachea** le risposte (non fare la stessa chiamata 10 volte)
- **Coda** le richieste (massimo N concurrent)

Claude può scriverlo — basta dirglielo nel prompt.

Come parlarne a Claude per ottenerlo bene

Prompt da NON usare

> **SUGGERIMENTO**
>
> «Collega il mio sito a OpenAI.»

Claude dovrà indovinare: dove giri questo codice? Server o client? Quale endpoint? Quale modello? Come gestisci errori? Cosa fai se 429? Streaming o no?

10 decisioni per Claude → 5 iterazioni di chiarimento.

Prompt da usare

SUGGERIMENTO

«Crea un endpoint API server-side in Next.js 14 (`app/api/generate/route.ts`) che riceve un POST con `{ prompt: string }` , chiama OpenAI Chat Completions con modello `gpt-4o-mini` , e ritorna `{ result: string }` .

La API key OpenAI è in env `OPENAI_API_KEY` (già settata in Vercel).

Gestisci:
- **400** se il body è malformato o `prompt` mancante
- **401** se la API key è invalida (loggato, non esposto al client)
- **429** rate limit con backoff esponenziale (3 retry)
- **500** generico se altro

Il client (Client Component) chiama `fetch('/api/generate', { method: 'POST', body: JSON.stringify({ prompt }) })` con stato loading sul bottone, errori inline mostrati all'utente.»

Differenza:
- Prima versione: Claude propone qualcosa generico, tu correggi 5 volte
- Seconda versione: Claude scrive **production-ready** alla prima iterazione

Il pattern

Quando integri un'API, dichiari sempre:
1. **Dove gira** (client/server)
2. **Quale endpoint e modello**
3. **Dov'è la API key** (env variable)

4. **Come gestire ogni status code** (400, 401, 429, 500)
5. **Il flow client** (loading, errori inline)

5 dettagli, 90 secondi di prompt, codice solido alla prima.

La regola che ti porti a casa

> **SUGGERIMENTO**
>
> Un'API non è magia. È un messaggio HTTP con un verbo, un URL, header (con la tua key), e un body JSON. Il server risponde con uno status code preciso e un body.

Quando qualcosa non funziona, **apri il network tab del browser** (F12 → Network). Vedi la richiesta che parte, lo status code che torna, il body. Il bug è quasi sempre lì.

> **SUGGERIMENTO**
>
> Quick Win 2 di 5. I prossimi 3: cookie e sessioni · backup database · CORS spiegato.
> **Vibecoding Serio** — il libro completo a giugno 2026.

Capitolo redatto: 2026-04-25 · Versione 1.0

Perché il tuo login non ricorda l'utente (e cosa sono i cookie)

SUGGERIMENTO

Quick Win 3 di 5 · Tempo di lettura: 11 minuti · Modulo collegato: 7
Capitolo standalone pubblicabile come articolo blog.

Il momento «ma come»

Hai costruito un sito con login. Funziona: l'utente inserisce email e password, viene rimandato alla dashboard. Tutto bene. Però:

- **Sintomo 1**: l'utente fa login, ma dopo qualche minuto è disconnesso. Senza ragione.
- **Sintomo 2**: l'utente fa login in produzione, viene rimandato alla dashboard, e poi **subito al login**. Loop infinito.
- **Sintomo 3**: in locale tutto perfetto. Online no.

In tutti e tre i casi, il problema è **cookie** e **sessioni**. Risolviamo.

Cosa pensa il vibecoder

> **SUGGERIMENTO**
>
> «Login = utente loggato. Punto. Non capisco perché se ne dimentica.»

Il fraintendimento è qui: pensare che «loggato» sia uno stato che il server **ricorda automaticamente**. Non è così. HTTP è **stateless**: ogni richiesta è indipendente, il server non ricorda niente da una richiesta all'altra.

Per dare l'illusione di «rimanere loggati», servono **cookie** e **sessioni**. Sono il trucco che fa funzionare il web come lo conosci.

Cosa succede davvero

Il problema fondamentale: HTTP è smemorato

Tu fai login. Il server ti riconosce. Ottimo. **Ma alla richiesta successiva**, il server non ha la minima idea di chi sei. Le richieste HTTP sono indipendenti.

Per simulare uno stato, si usa un **cookie**: un piccolo testo che il server invia al browser. Il browser lo salva e lo **rispedisce automaticamente** ad ogni richiesta successiva verso lo stesso dominio.

```
1. Login: server → browser
   Set-Cookie: session_id=abc123xyz; HttpOnly; Secure

2. Richiesta successiva: browser → server (automatico)
   Cookie: session_id=abc123xyz

3. Server riconosce abc123xyz → "ah, è Marco"
```

Le 4 flag che cambiano tutto

I cookie hanno **flag** importantissime. Ognuna ha un perché.

`HttpOnly` — il cookie non è leggibile da JavaScript

Senza HttpOnly, qualunque script malevolo nella pagina (es. da una libreria compromessa) può leggere il cookie e rubare la sessione. **Sempre HttpOnly** per i cookie di sessione.

`Secure` — il cookie viaggia solo su HTTPS

Senza Secure, il cookie passa in chiaro su HTTP, intercettabile. **Sempre Secure** in produzione (HTTPS).

`SameSite` — quando il cookie è inviato cross-site

- `Strict` : solo per richieste dallo stesso sito (più sicuro, ma ti rompe gli OAuth)
- `Lax` : anche da link esterni, default ragionevole
- `None` : sempre, richiede `Secure` (rischioso, solo se sai cosa fai)

`Max-Age` o `Expires` — quando il cookie scade

Senza, il cookie sparisce alla chiusura del browser. Con, dura un tempo definito (es. 7 giorni → `Max-Age=604800`).

I 3 sintomi spiegati

Sintomo 1: utente sloggato dopo qualche minuto

Il cookie ha **scadenza breve** (default di NextAuth: 1 ora) e non c'è refresh automatico. L'utente «scade» senza accorgersene.

Fix: configura `maxAge: 30 * 24 * 60 * 60` (30 giorni) e abilita refresh token.

Sintomo 2: login loop in produzione

In produzione il dominio è HTTPS. Il cookie dovrebbe avere flag `Secure: true` . Ma se la config dice `Secure: false` (per dev) e non distingui ambiente, il cookie viene impostato come HTTP-only. Browser non lo rinvia su HTTPS → server non ti riconosce → redirect al login.

Fix: in NextAuth/Express, `useSecureCookies: process.env.NODE_ENV === 'production'` .

Sintomo 3: in locale sì, in produzione no

Causa più frequente: il **dominio del cookie** è sbagliato. Il cookie è impostato per `app.miosito.it` ma il redirect manda a `miosito.it`. Domini diversi = cookie non inviato.

Fix: configura `cookie.domain = 'miosito.it'` (con il punto davanti `.miosito.it` se vuoi che valga su tutti i sottodomini).

Sessione lato server vs JWT

Il valore del cookie (`abc123xyz`) deve corrispondere a **qualcosa** lato server. Due tecniche:

Sessione classica: il server tiene una tabella `sessions` con `session_id`, `user_id`, `expires_at`. Ad ogni richiesta, lookup nella tabella. Più richieste DB, più scalabile su sistemi piccoli.

JWT (JSON Web Token): il «session_id» è un token **firmato** che contiene direttamente le info utente. Il server verifica la firma (con un segreto in env) e legge `user_id` direttamente dal token. Stateless, scalabile, ma revoca difficile.

NextAuth, Supabase Auth, Clerk usano **JWT di default** oggi. Tu come vibe-coder devi sapere che:

- Il token JWT vive nel cookie (httpOnly, Secure)
- Ha una scadenza incorporata (es. 30 giorni)
- È firmato con un segreto chiamato spesso `NEXTAUTH_SECRET` o `JWT_SECRET`
- Se cambi il segreto, **tutti gli utenti vengono sloggati**

Diagnosi in 30 secondi

Apri DevTools → **Application** tab → **Cookies** → seleziona il tuo dominio.

Vedi:

- I cookie esistono?
- Hanno le flag giuste? (HttpOnly ✓, Secure ✓ in produzione)
- Il dominio è corretto?
- La scadenza è ragionevole?

Quasi tutti i bug login si vedono qui.

Come parlarne a Claude per ottenerlo bene

Prompt da NON usare

> **SUGGERIMENTO**
>
> «il login non funziona dopo un giorno»

Troppo vago. Claude propone 8 cause possibili.

Prompt da usare

> **SUGGERIMENTO**
>
> «Sto usando NextAuth v5 con provider Credentials su Next.js 14, deployato su Vercel con dominio custom HTTPS `miosito.com`.
>
> Sintomo: l'utente fa login (vedo `200 OK` su `/api/auth/callback/credentials`), redirect a `/dashboard`, ma `/dashboard` lo rimanda subito a `/login`. Loop infinito.
>
> In `localhost:3000` funziona perfettamente.
>
> Sospetto problema di cookie httpOnly + Secure + dominio. Come configuro NextAuth `cookies` options per il dominio production HTTPS, mantenendo localhost funzionante?»

Differenza: il primo prompt produce 8 ipotesi e iterazioni. Il secondo va dritto al punto e Claude ti dà la config esatta in una iterazione.

La regola che ti porti a casa

> **SUGGERIMENTO**
>
> Un cookie senza `HttpOnly` è un buco di sicurezza. Un cookie senza `Secure` è un altro buco di sicurezza. Un cookie con `Secure` su HTTP è invisibile. Un cookie con dominio sbagliato è invisibile.

Quando il login non ricorda, è quasi sempre **una di queste 4 cose** sbagliate. Vai in DevTools → Application → Cookies, le verifichi in 30 secondi.

> **SUGGERIMENTO**
>
> Quick Win 3 di 5. I prossimi 2: backup database · CORS spiegato.
> **Vibecoding Serio** — il libro completo a giugno 2026.

Capitolo redatto: 2026-04-25 · Versione 1.0

Il tuo database ha perso i dati. Perché e come non succede più

> **SUGGERIMENTO**
>
> **Quick Win 4 di 5** · Tempo di lettura: 10 minuti · Modulo collegato: 6
> Capitolo standalone pubblicabile come articolo blog.

Il momento «no, no, no, no»

Sei un freelancer. Hai costruito un'app per un cliente. Funziona, è online da 4 mesi, ha 200 utenti reali con i loro dati. Una mattina apri il pannello admin per fare manutenzione e:

- I dati sono **vuoti**
- Oppure: corrotti
- Oppure: il database non parte più
- Oppure: hai cancellato la tabella sbagliata col tuo script di pulizia

Il telefono squilla. È il cliente. Non sai cosa rispondergli.

Questo capitolo ti racconta i 4 modi in cui un database «perde i dati», e — soprattutto — come non finire mai in questa situazione.

Cosa pensa il vibecoder

> **SUGGERIMENTO**
>
> «Il database è il database, sta lì, ricorda i dati. Cosa vuoi che gli succeda?»

Tante cose. La maggior parte non sono attacchi hacker. Sono **errori normali** che succedono. La differenza tra freelancer professionista e freelancer disastrato è **l'avere un piano** quando succede.

Cosa succede davvero: i 4 modi di perdere dati

1. Cancellazione accidentale

Caso più comune. Esempi reali:

- Esegui una query `DELETE FROM users WHERE active = false` ma scrivi `DELETE FROM users` per errore
- Lanci uno script di «pulizia» che cancella troppo
- L'utente clicca su «Elimina account» senza conferma e perdi i suoi dati legati
- Modifichi una migration di Drizzle/Prisma e rinomini una colonna → i dati vecchi vanno persi

Il bug è semplice: **un comando di troppo, una WHERE dimenticata**, e secondi dopo migliaia di righe sono sparite.

2. Schema disallineato (migration sbagliata)

Caso classico:

1. Aggiungi una colonna in locale a mano (Supabase Studio, pgAdmin)
2. Modifichi il codice per usarla
3. Deployi in produzione
4. Crash → `ERROR: column "phone_number" does not exist`

In sé non hai «perso dati» — ma la app è ferma. Per molti clienti è equivalente.

Variante peggiore: tu **sì** hai applicato una migration in produzione, ma scritta male, e ha **rinominato** una colonna esistente perdendo il legame con i dati.

3. Backup mai testati = backup non esistenti

Vai sereno perché «Supabase fa il backup automatico». Un giorno il database si corrompe, vai a ripristinare, e scopri che:

- Il backup è corrotto anche lui
- Il backup non include una tabella critica
- Il piano free conserva backup solo per 7 giorni
- Il backup è in un formato che non sai restorare

Hai imparato sulla pelle viva che **backup non testato = backup inesistente**.

4. Hosting che muore o cambia regole

Storia vera: provider di database «free tier» che decide di chiudere il servizio in 30 giorni. O alza i prezzi del 1000%. O ti blocca per «violation of terms» senza spiegazione, e il tuo DB è loro ostaggio.

Probabilità bassa? No. Negli ultimi 5 anni questo è successo a Heroku Postgres free, Planetscale free, AWS Free Tier varie volte, Firebase free reset. **Sempre**.

Come non succede più: 4 strategie

Strategia 1 — Backup automatici (sempre)

Per ognuno dei tre casi più comuni:

Caso A: Supabase

- Free tier: backup automatico **giornaliero**, conservato 7 giorni. **Insufficiente** per produzione vera.
- Pro tier ($25/mese): backup giornalieri 30 giorni + Point-in-Time Recovery.

Regola: appena il sito ha clienti veri, **passa a Pro**. 25$ vale come assicurazione.

Caso B: Postgres su VPS Aruba (o altro VPS self-hosted)

I backup li configuri tu. Setup completo:

```
# /usr/local/bin/backup-db.sh
#!/bin/bash
DATE=$(date +%Y-%m-%d)
BACKUP_DIR=/backups
DB=cantiere

mkdir -p $BACKUP_DIR
pg_dump $DB | gzip > $BACKUP_DIR/$DB-$DATE.sql.gz

# Tieni solo gli ultimi 30 giorni
find $BACKUP_DIR -name "*.sql.gz" -mtime +30 -delete

# Copia su storage esterno (es. Backblaze B2 a 6$/TB/mese)
rclone copy $BACKUP_DIR b2:miei-backup/cantiere/ --include
"$DB-$DATE.sql.gz"
```

Aggiungi al crontab per girare ogni notte alle 3:

```
0 3 * * * /usr/local/bin/backup-db.sh
```

5 minuti di setup. 6$/mese di storage esterno. Vita salvata.

Caso C: SQLite

Database file. Per backup, **copialo**:

```
sqlite3 app.db ".backup /backups/app-$(date +%Y-%m-%d).db"
```

Cron giornaliero. Banale.

Strategia 2 — Testa i backup (mensile)

Una volta al mese, **restora un backup in un ambiente di test**. Verifica che funziona.

Senza questo passo, non hai backup. Hai una **speranza**.

```
# Esempio Postgres
gunzip < cantiere-2026-04-25.sql.gz | psql cantiere_test
# Verifica: numero righe nelle tabelle critiche, query di
esempio
```

15 minuti al mese. Ti salva la carriera.

Strategia 3 — Migration disciplinate

Mai modificare lo schema in produzione a mano. Mai. Usa sempre migration versionate.

Con Drizzle:

```
# Genera migration dalla diff schema
pnpm drizzle-kit generate

# Applica in produzione
pnpm drizzle-kit migrate
```

Il file di migration è in git, versionato, riproducibile. Se qualcosa va storto, sai esattamente cosa è cambiato.

Per **modifiche distruttive** (rinomina colonna, drop tabella): **mai direttamente**. Usa il pattern in 3 step:

1. **Aggiungi** la nuova colonna senza toccare la vecchia
2. **Migra i dati** dal vecchio al nuovo (script)
3. **Rimuovi** la vecchia colonna (in una migration successiva, dopo aver verificato)

Lento? Sì. Reversibile in qualunque momento? Sì. Vita salvata? Sì.

Strategia 4 — Esci sempre dalla dipendenza vendor

Non legarti a un singolo provider in modo che, se chiude o cambia regole, sei morto. Strategia minima:

- **Backup esterni**: anche se usi Supabase, fai dump settimanali su Backblaze o S3 personale

- **Postgres standard**: scegli database «portabili» (Postgres standard) invece di vendor-specifici (es. Firebase Firestore non si esporta facilmente)
- **Schema documentato**: il file delle migration in git è il tuo schema, riproducibile ovunque

In caso di emergenza con il provider, in 2 ore hai un nuovo Postgres altrove con i dati restorati.

Come parlarne a Claude per ottenerlo bene

Prompt da NON usare

> **SUGGERIMENTO**
>
> «Fai backup del database.»

Prompt da usare

> **SUGGERIMENTO**
>
> «Sto deployando un'app Next.js + Postgres su VPS Aruba. Il DB si chiama `cantiere`, user `postgres`. Configura uno script di backup con queste caratteristiche:
>
> 1. Backup giornaliero alle 3am via cron
> 2. `pg_dump` con compressione gzip
> 3. Salva in `/var/backups/cantiere/`
> 4. Conserva 30 giorni, elimina i più vecchi
> 5. Copia ogni backup anche su Backblaze B2 (rclone configurato come `b2:`)
> 6. Notifica via email su `luca@miosito.it` se il backup fallisce
>
> Dammi anche uno script di test restore che prende l'ultimo backup e lo restora in un DB di test `cantiere_test`.»

Risultato: setup completo in 1 iterazione. Production-ready.

La regola che ti porti a casa

> **SUGGERIMENTO**
>
> **Un backup non testato è una speranza, non un piano.**

E:

> **SUGGERIMENTO**
>
> **Lo schema in produzione si modifica solo via migration versionate. Mai a mano.**

Se rispetti queste due regole, il 99% dei disastri di «ho perso i dati» non ti succedono. Mai.

SUGGERIMENTO

Quick Win 4 di 5. Prossimo: CORS spiegato.
Vibecoding Serio — il libro completo a giugno 2026.

Capitolo redatto: 2026-04-25 · Versione 1.0

CORS: l'errore più odiato spiegato in modo che tu possa fixarlo da solo

SUGGERIMENTO

Quick Win 5 di 5 · Tempo di lettura: 10 minuti · Modulo collegato: 7
Capitolo standalone pubblicabile come articolo blog.

Il momento «questa cosa di nuovo»

Apri la console del browser durante il debug. Vedi questa scritta in rosso:

```
Access to fetch at 'https://api.miosito.it/users' from
origin 'https://miosito.it'
has been blocked by CORS policy: No 'Access-Control-Allow-
Origin' header
is present on the requested resource.
```

Senti il sangue salire alla testa. Apri Claude. «Errore CORS, aiuto.». Claude propone 5 fix. Ne provi 3, peggiora. Ne provi un quarto, un altro errore. Finisci a copiare configurazioni a caso.

Stop. Adesso CORS lo capisci, e in 10 minuti sai fixarlo da solo.

Cosa pensa il vibecoder

> **SUGGERIMENTO**
>
> «CORS è una protezione del server che blocca le mie richieste. Devo disattivarla.»

Errore. CORS non è una protezione del **server**. È una protezione del **browser**. E non si «disattiva» — si configura correttamente.

Cosa succede davvero

Il problema che CORS risolve

Immagina questo scenario, senza CORS. Tu, mentre sei loggato sul tuo home banking, apri un sito qualunque. Il sito ha JavaScript nascosto che fa:

```
fetch('https://homebanking.com/transfer?amount=10000&to=
hacker', { credentials: 'include' });
```

Il browser, vedendo che sei loggato su homebanking (cookie già presente), **manderebbe la richiesta con i tuoi cookie**. Il server bancario vedrebbe una richiesta autenticata e... farebbe il bonifico.

Catastrofe.

CORS è la regola del browser che dice: **un sito (origin) non può fare richieste JavaScript verso un altro origin a meno che il secondo non lo autorizzi esplicitamente.**

Cosa è «origin»

Un **origin** è la combinazione di **protocollo + dominio + porta**.

Esempi di origin diversi:

- `https://miosito.it` ≠ `http://miosito.it` (protocollo diverso)
- `https://miosito.it` ≠ `https://api.miosito.it` (sottodominio diverso)
- `http://localhost:3000` ≠ `http://localhost:8080` (porta diversa)

Stesso origin = no CORS. Origin diversi = CORS attivo.

Quando CORS NON è un problema

Se frontend e backend sono **stesso origin**, niente CORS:

- `https://miosito.it/index.html` chiama `https://miosito.it/api/users` → **OK**

In Next.js full-stack, frontend e backend stanno nello stesso progetto sullo stesso dominio. **Niente CORS, mai**. È una delle ragioni per cui Next.js è popolare.

Quando CORS È un problema

Frontend e backend su domini diversi:

- `https://miosito.it` (frontend Vercel) chiama `https://api.miosito.it` (backend Aruba)
- `http://localhost:3000` (frontend Vite) chiama `http://localhost:8080` (backend Express in dev)
- Qualunque chiamata a un'API esterna **dal browser** (lato client)

In questi casi, prima della richiesta vera, il browser fa una richiesta speciale chiamata **preflight**:

```
OPTIONS /api/users HTTP/1.1
Origin: https://miosito.it
```

E aspetta che il server risponda con header CORS:

```
Access-Control-Allow-Origin: https://miosito.it
Access-Control-Allow-Methods: GET, POST, PATCH, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Allow-Credentials: true
```

Tradotto: «Sì, autorizzo `miosito.it` a chiamarmi con questi metodi e questi header».

Se il server **non risponde con CORS giusto**, il browser **blocca la richiesta vera**. Vedi l'errore in console.

Come si fixa

Il fix è SUL BACKEND

Punto importante. CORS non si fixa sul frontend. Sul frontend non c'è niente da fare. È il **backend** che deve mandare gli header giusti.

Express (Node.js)

```
import cors from 'cors';
app.use(cors({
  origin: ['https://miosito.it', 'http://localhost:3000'],
  credentials: true,
}));
```

3 righe. `origin` è un array che include sia produzione che dev.

Next.js API routes

In `next.config.js`:

```
module.exports = {
  async headers() {
    return [{
      source: '/api/:path*',
      headers: [
        { key: 'Access-Control-Allow-Origin', value:
'https://miosito.it' },
        { key: 'Access-Control-Allow-Methods', value:
'GET,POST,PATCH,DELETE' },
        { key: 'Access-Control-Allow-Headers', value:
'Content-Type, Authorization' },
        { key: 'Access-Control-Allow-Credentials', value:
'true' },
      ],
    }];
  },
};
```

Laravel (PHP)

`config/cors.php`:

```
'allowed_origins' => ['https://miosito.it', 'http://
localhost:3000'],
'allowed_methods' => ['*'],
'supports_credentials' => true,
```

Django (Python)

`pip install django-cors-headers`, poi in `settings.py`:

```
CORS_ALLOWED_ORIGINS = [
    'https://miosito.it',
    'http://localhost:3000',
]
CORS_ALLOW_CREDENTIALS = True
```

Nginx (se usi reverse proxy)

```
add_header Access-Control-Allow-Origin "https://miosito.it"
always;
add_header Access-Control-Allow-Methods "GET, POST, PATCH,
DELETE" always;
add_header Access-Control-Allow-Headers "Authorization,
Content-Type" always;
add_header Access-Control-Allow-Credentials "true" always;
```

MAI usare `*` con autenticazione

Tentazione del vibecoder disperato:

```
// MALE
app.use(cors({ origin: '*' }));
```

Tradotto: «Autorizza chiunque al mondo a chiamarmi». Buco di sicurezza enorme se hai autenticazione. **Sempre dominio specifico.**

Diagnostica in 60 secondi

Quando hai un errore CORS:

1. **Apri DevTools** → Network tab → cerca la richiesta in rosso
2. Click sulla richiesta → tab **Headers**
3. Guarda **Request Headers** — c'è un `Origin: https://miosito.it` ?
4. Guarda **Response Headers** — c'è un `Access-Control-Allow-Origin` ?

Se Response **non ha** `Access-Control-Allow-Origin` , il backend non sta rispondendo CORS. Vai a fixare il backend.

Se l'header c'è ma il valore non corrisponde all' `Origin` (es. `Allow-Origin: http://localhost:3000` ma tu sei su `https://miosito.it`), correggi la lista.

Come parlarne a Claude per ottenerlo bene

Prompt da NON usare

> **SUGGERIMENTO**
>
> «errore CORS aiuto»

Inutile, troppo poco.

Prompt da usare

> **SUGGERIMENTO**
>
> «Errore CORS in produzione. Il mio frontend è Next.js su Vercel (`https://app.miosito.it`). Il backend è Express + Postgres su Aruba VPS (`https://api.miosito.it`).
>
> Errore in console: `Access to fetch at 'https://api.miosito.it/users' from origin 'https:/ miosito.it' has been blocked by CORS policy`
>
> Il backend non ha il middleware `cors` configurato. Aggiungilo per accettare:
> - `https://app.miosito.it` (produzione)
> - `http://localhost:3000` (dev)
>
> Includi `credentials: true` per i cookie httpOnly. Includi handling del preflight (OPTIONS).»

Risultato: fix in 1 iterazione. Production-ready.

La regola che ti porti a casa

> **SUGGERIMENTO**
>
> CORS non si «disattiva». CORS si **configura** sul backend. Specifica i domini esatti del frontend, mai `*`. Includi `credentials: true` se usi cookie.

E:

> **SUGGERIMENTO**
>
> Il browser è il vigile. Il backend è chi deve esibire i documenti giusti. Se ti dicono «blocked by CORS», è il backend che non ha mostrato i documenti.

> **SUGGERIMENTO**
>
> Quick Win 5 di 5. Hai finito i quick win — i 5 capitoli «killer» del libro **Vibecoding Serio**.
> Per il resto del lessico operativo (anatomia pagina, glossario tecnico, debug avanzato): preordina il libro completo a giugno 2026.

Capitolo redatto: 2026-04-25 · Versione 1.0

PARTE III

Glossari

130+ termini, organizzati per categoria. Da consultare come dizionario.

Glossario 1 — Programmazione & architettura

SUGGERIMENTO

80 termini · 18 pagine · Riferimento rapido

Per ogni voce: nome italiano + inglese · descrizione 1-2 righe · come usarlo nel prompt a Claude (quando applicabile).
Organizzato per categoria. Stampabile come dizionario di riferimento.

A. Ambienti e deploy (10 termini)

Localhost (local host) Il tuo PC, raggiungibile da `localhost:3000` o `127.0.0.1`. Lo vedi solo tu. Sandbox per sviluppo. ↳ «Voglio testare la feature X in localhost prima di deployare.»

Sviluppo (Dev) (development) Ambiente non-produttivo, di solito condiviso col team, accessibile online ma non pubblico (es. `dev.miosito.com`). Per test prima del rilascio.

Staging Copia esatta della produzione usata per test finali. Stesso DB schema, dati spesso anonimizzati. «Prova generale».

Produzione (Prod) (production) Il sito live, visibile a tutti. Quello che vedono i clienti veri. Errori = email arrabbiate. ↳ «Deploya in produzione solo dopo aver testato in staging.»

Deploy L'azione di «spostare» il codice da un ambiente a quello superiore (locale → dev → staging → prod). Tipicamente automatizzato. ↳ «Configura il deploy automatico su Vercel da push a main.»

Build Il processo che trasforma il tuo codice sorgente in artefatti pronti per il server (HTML, CSS, JS bundlato, ottimizzato). ↳ «Lancia `pnpm build` e poi controlla la cartella `.next/`.»

Hot reload (hot module replacement, HMR) Tecnica che durante lo sviluppo ricarica solo il modulo cambiato senza riavviare tutto. Velocissimo. Tipico di Vite/Next dev.

Environment variable (.env) Valore configurabile letto a runtime, non hardcoded nel codice. Tipico per segreti (API key) o config diverse per ambiente. ↳ «Sposta `OPENAI_API_KEY` in env, mai nel codice.»

`.env.example` File template versionato in git che mostra **i nomi** delle variabili necessarie senza i valori veri. Aiuta i collaboratori.

CI/CD (Continuous Integration / Continuous Deployment) Automazione: ogni push fa partire test, build, deploy. Strumenti tipici: GitHub Actions, GitLab CI, CircleCI. ↳ «Aggiungi un workflow GitHub Actions che esegue test e linting su ogni PR.»

B. Infrastruttura web (14 termini)

Server La macchina (fisica o virtuale) che esegue il tuo codice backend. Sta in un data center, attiva 24/7.

Hosting Il fornitore che ti vende il server (Aruba, Vercel, AWS). «Agenzia immobiliare» del web.

VPS (Virtual Private Server) Macchina virtuale dedicata. Hai accesso root completo. Costo: 5-30€/mese. Tipica per progetti seri self-hosted. ↳ «Prendi un VPS su Aruba a 5€/mese per ospitare l'app Next.js.»

Dominio (domain) Il nome che digiti nel browser (`miosito.it`). Si compra annualmente da un registrar (Namecheap, Aruba domini).

Sottodominio (subdomain) Prefisso al dominio principale: `app.miosito.it` , `api.miosito.it` , `dev.miosito.it` . Gratis, configuri da DNS.

IP (Internet Protocol address) Numero univoco di un server, tipo `188.213.170.214` (IPv4) o equivalente IPv6.

Porta (port) Numero che identifica un servizio specifico su un server. HTTP = 80, HTTPS = 443, app dev tipiche = 3000, 8080.

DNS (Domain Name System) Sistema mondiale che traduce nomi (miosito.it) in IP. Elenco telefonico distribuito. ↳ «Configura il record A su Cloudflare per puntare miosito.it al VPS.»

Record A Record DNS che lega un dominio a un IP. Esempio: `miosito.it → 188.213.170.214` .

Record CNAME Record DNS che lega un dominio a un altro dominio (alias). Esempio: `www.miosito.it → miosito.it` .

CDN (Content Delivery Network) Rete di server distribuiti nel mondo che cachano file statici. Cloudflare, Cloudfront, Fastly. ↳ «Metti Cloudflare davanti al sito per cache CDN globale.»

Reverse proxy Server intermedio che riceve richieste e le inoltra a un'app interna. Nginx fa questo.

Load balancer Bilanciatore: distribuisce traffico tra più server identici. Per scalabilità.

Firewall Filtro che decide quale traffico passa al server. Su VPS Linux: `ufw` , `iptables` , fail2ban.

C. Trasferimento codice (6 termini)

FTP (File Transfer Protocol) Protocollo vecchio per copiare file dal PC al server. FileZilla è il client più diffuso. Insicuro: usa **SFTP** invece.

SFTP (Secure FTP) Versione cifrata di FTP. Usabile dovunque sia disponibile SSH.

SSH (Secure Shell) Apre una shell remota sul server. Esegui comandi come se fossi seduto davanti. Modo principale di gestire un VPS. ↳ «Connettiti via SSH al VPS per riavviare PM2.»

Git Sistema di versionamento del codice. Traccia ogni modifica nel tempo, permette branch/merge, lavoro in team.

Git push (push to remote) Manda i tuoi commit locali al repository remoto (GitHub, GitLab). Innesca il deploy se hai CI/CD.

Pull request (PR) (merge request) Proposta di merge da un branch a un altro. Permette code review prima dell'integrazione.

D. Linguaggi e framework (16 termini)

HTML (HyperText Markup Language) Linguaggio di markup per la struttura della pagina. Tag come `<h1>`, `<p>`, `<button>`.

CSS (Cascading Style Sheets) Linguaggio per lo stile della pagina. Colori, font, layout, animazioni.

JavaScript (JS) Linguaggio di programmazione del web. Gira nel browser e — grazie a Node.js — anche sul server.

TypeScript (TS) JavaScript con controllo dei tipi. Riduce bug. Default oggi nei progetti seri. ↳ «Inizializza il progetto in TypeScript invece di JavaScript.»

React Libreria UI di Meta. La più diffusa. Si scrive in JSX (JS + HTML inline). Componenti, props, state, hook.

Vue Alternativa a React, più semplice da imparare. Popolare in Europa.

Svelte Framework UI moderno (2019). Più snello, performance migliori.

Next.js Meta-framework basato su React. Aggiunge routing, server-side rendering, API routes. Default oggi per app web moderne.

PHP Linguaggio classico del web. WordPress, Drupal. Ancora il 40% di Internet.

Laravel Framework PHP elegante e moderno. Sintassi pulita, batterie incluse.

Python Linguaggio scientifico per eccellenza. AI/ML, data science. Anche per web (Django, Flask, FastAPI).

Django Framework Python «batterie incluse» per app web complete.

Flask Framework Python minimalista. Tu costruisci tutto.

Node.js Runtime che esegue JavaScript fuori dal browser. Permette JS lato server.

Express Framework Node.js minimalista per API. Il «Flask di Node».

Tailwind CSS Libreria di classi CSS atomiche. Scrivi `className="px-6 py-3 rounded-lg"` invece di file CSS separati. Standard moderno.

E. Concetti backend (12 termini)

API (Application Programming Interface) Lista di endpoint che un server espone, accessibili via HTTP. Il «menu» del server.

Endpoint Singolo URL di un'API che risponde a un certo tipo di richiesta. Es. `POST /api/users`.

REST (Representational State Transfer) Stile di progettazione delle API basato su HTTP standard. Risorse identificate da URL, operazioni con verbi HTTP.

GraphQL Alternativa moderna a REST. Il client specifica esattamente quali campi vuole. Un solo endpoint.

Webhook URL che il **tuo** server espone per **ricevere** notifiche da servizi esterni. Es. Stripe ti notifica un pagamento andato a buon fine. ↳ «Configura un webhook Stripe in `/api/webhooks/stripe`.»

Microservizio (microservice) Architettura con tanti servizi piccoli specializzati invece di un'app grossa.

Monolite (monolith) Architettura «tutto in uno». App singola che fa tutto. Più semplice da gestire per progetti piccoli/medi.

Serverless Codice che gira «su richiesta» senza server dedicato sempre attivo. Vercel Functions, AWS Lambda. Paghi a esecuzione.

Container Pacchetto isolato che contiene app + dipendenze. Tecnicamente: Docker.

Docker Tecnologia di containerizzazione. Permette «funziona qui = funziona ovunque».

Kubernetes (K8s) Orchestratore di container. Per produzioni grosse con tanti container distribuiti. Overkill per progetti piccoli.

CI (Continuous Integration) Pratica di integrare codice frequentemente con test automatici. Vedi anche CI/CD.

F. Sicurezza e autenticazione (10 termini)

HTTPS (HTTP Secure) HTTP cifrato con TLS. Obbligatorio in produzione moderna. Lucchetto verde nella barra del browser.

Certificato SSL/TLS «Passaporto digitale» del server. Emesso da una CA (Certificate Authority). Let's Encrypt = gratis.

Let's Encrypt CA non-profit che emette certificati SSL gratis con rinnovo automatico ogni 90 giorni.

CORS (Cross-Origin Resource Sharing) Regola del browser: richieste cross-origin (dominio diverso) bloccate a meno che il server destinatario non autorizzi. ↳ «Configura CORS sul backend per accettare il dominio del frontend.»

JWT (JSON Web Token) Token di autenticazione firmato. Contiene info utente direttamente. Stateless, scalabile.

OAuth Protocollo per «login con Google/GitHub/Facebook». Permette a un sito di autenticare via terzi.

Session Stato che il server tiene per ricordarsi chi è l'utente tra le richieste. Tipicamente collegato a un cookie.

Cookie Piccolo testo che il server invia al browser. Il browser lo rispedisce automaticamente nelle richieste successive.

Token Stringa che identifica un utente o autorizza un'operazione. Es. `Bearer eyJhbGc...`.

Hashing password (bcrypt) Trasformazione one-way della password prima di salvarla in DB. Mai salvare password in chiaro. Standard: bcrypt cost 12+.

G. Database (12 termini)

SQL (Structured Query Language) Linguaggio per parlare ai database relazionali. SELECT, INSERT, UPDATE, DELETE.

NoSQL Famiglia di database senza schema rigido. MongoDB, Firestore. Documenti tipo JSON.

PostgreSQL (Postgres) Database relazionale open source. Default moderno per app web serie. Robusto, ricco di feature.

MySQL Database relazionale classico. Open source. Ancora ovunque (WordPress lo usa).

SQLite Database «in un file». Perfetto per piccoli progetti, tool, dev. Non per produzione web seria.

MongoDB Database NoSQL documentale. Schema flessibile.

Supabase Servizio che ti dà Postgres + auth + storage + real-time + API REST auto-generata. «Firebase di Postgres».

Neon Postgres serverless. Scaling automatico. Ottimo con Vercel.

Firebase Suite Google di servizi BaaS. Firestore (NoSQL), Auth, Storage. Mobile-first.

ORM (Object-Relational Mapper) Libreria che traduce il tuo codice in SQL. Drizzle, Prisma, Eloquent, SQLAlchemy. ↳ «Usa Drizzle ORM invece di scrivere SQL a mano.»

Migration File versionato che descrive una modifica allo schema (es. aggiungi colonna). Permette di tenere allineati DB locale e produzione. ↳ «Crea una migration con Drizzle per aggiungere la colonna `phone_number`.»

Backup Copia dei dati del database conservata altrove. Da fare automaticamente. Da testare mensilmente. Senza, sei nelle mani del fato.

Glossario 1 redatto: 2026-04-25 · Versione 1.0 · 80 termini in 7 categorie.

Glossario 2 — Anatomia di una pagina web

SUGGERIMENTO

50 termini · 14 pagine · Riferimento rapido per UI/UX

Per ogni voce: nome italiano + inglese · come si disegna · uno-due esempi · prompt-ready per Claude (quando applicabile).
Organizzato per zona della pagina. Da stampare e tenere accanto al PC.

A. Layout (5 termini)

Header (intestazione) La fascia in cima alla pagina. Contiene logo, nav, CTA. Spesso sticky. ↳ «Header sticky con logo a sinistra, nav centrale, CTA primary a destra.»

Footer (piè di pagina) La fascia in fondo. Link legali, social, copyright, eventuali contatti. ↳ «Footer 3 colonne: contatti, link rapidi, newsletter.»

Main (contenuto principale) Il cuore della pagina, l'area che cambia tra una pagina e l'altra. Tag HTML `<main>`.

Sidebar (barra laterale) Colonna laterale (destra o sinistra). Tipica nelle dashboard. Su mobile spesso sparisce o diventa drawer. ↳ «Sidebar 280px sticky a sinistra, collassabile su mobile in hamburger drawer.»

Navigation (nav) (navigazione) Insieme dei link principali del sito. Può essere dentro l'header o separata.

B. Navigazione (8 termini)

Navbar (barra di navigazione) Barra orizzontale con logo + link + CTA. Tipica posizione: dentro l'header. ↳ «Navbar con 4 link orizzontali, dropdown "Servizi" a tendina, hamburger su mobile.»

Sidebar nav (navigazione laterale) Barra verticale con icone + label. Tipica delle dashboard SaaS. ↳ «Sidebar nav 240px collassabile, sezioni raggruppate, icona + label.»

Breadcrumb (briciole di pane) Traccia «sei qui»: `Home > Categoria > Sotto > Pagina`. ↳ «Breadcrumb in cima al main, separatore ">", ultimo livello non cliccabile.»

Tab (linguette) Linguette che cambiano contenuto senza ricaricare la pagina. Tipico delle pagine prodotto. ↳ «Tabs orizzontali con 3 sezioni — Info, Foto, Recensioni — bordo sotto la tab attiva.»

Stepper (indicatore di avanzamento) Mostra a che punto sei in un processo multi-step (checkout, onboarding). ↳ «Stepper orizzontale a 4 step: completati con check, attivo con sfondo accent, futuri grigi.»

Pagination (paginazione) Navigazione tra pagine: `« 1 2 3 ... 10 »`. Sotto liste lunghe. ↳ «Pagination con frecce + numeri + ellisse per pagine intermedie. 10 risultati per pagina.»

Hamburger menu (menu a panino) Icona ☰ (3 linee orizzontali) tipica mobile. Al click apre un drawer di nav. ↳ «Hamburger su mobile, click apre drawer dal lato sinistro.»

Anchor link (link interno) Link che porta a una sezione della stessa pagina (`#sezione`). ↳ «Anchor link “Vai ai prezzi” che porta alla sezione `#pricing` con scroll smooth.»

C. Contenuto (12 termini)

Hero (sezione di apertura) Banda alta sopra la fold. Titolo, sottotitolo, CTA, immagine. La prima impressione. ↳ «Hero a piena larghezza con h1 grosso, sottotitolo, due CTA (primary + secondary), illustrazione a destra.»

Card (carta, riquadro) Riquadro autocontenuto: immagine + titolo + descrizione + CTA. Mattone delle pagine moderne. ↳ «Card prodotto: immagine 4:3, titolo, prezzo grande, badge promo, bottone “Aggiungi”.»

Banner (insegna, avviso) Barra orizzontale stretta con avviso o promozione. Spesso dismissibile. ↳ «Banner giallo in cima “Spedizione gratuita sopra 50€”, dismissibile con X.»

List (elenco) Lista verticale di elementi simili. Email, conversazioni, todo.

Grid (griglia) Griglia 2D di elementi (di solito card). 4 colonne desktop → 1 mobile. ↳ «Grid prodotti 4 col desktop, 2 tablet, 1 mobile, gap 24px.»

Carousel / Slider (carosello) Elementi scorrevoli con frecce o swipe. Dot indicator sotto. ↳ «Carousel hero con 5 slide, autoplay 4s, frecce + dots, pause on hover.»

Accordion (fisarmonica) Sezioni espandibili al click. Tipico delle FAQ. ↳ «Accordion FAQ con 6 domande. Click espande la risposta, chiude le altre.»

Table (tabella) Dati in righe e colonne. Header sticky in scroll. ↳ «Tabella ordini: colonne Data, Cliente, Importo, Stato, Azioni. Header sticky, righe zebra.»

Gallery (galleria) Collezione di immagini. Spesso con lightbox per zoom. ↳ «Gallery con 12 immagini in grid 3 colonne, click apre lightbox a piena pagina.»

Embed Contenuto incorporato da altro sito (video YouTube, mappa Google, tweet).

Blockquote (citazione) Citazione visivamente distinta. Tag HTML `<blockquote>`.

Divider (divisore) Linea orizzontale o verticale che separa sezioni.

D. Interazione (15 termini)

Modal (finestra modale) Finestra centrale con overlay scuro dietro. Bloccante: devi interagire prima di tornare. ↳ «Modal di conferma "Sei sicuro?", bottoni Annulla (ghost) + Elimina (destructive).»

Drawer (cassetto) Pannello che entra dal bordo (lato/alto/basso). Tipicamente non bloccante. ↳ «Drawer dal basso su mobile per il menu, dismissibile con swipe down.»

Popover Pannello al click su un elemento. Diverso da tooltip (al hover) — il popover può contenere bottoni interattivi.

Tooltip (suggerimento) Bubble di testo al hover. Solo informativo, non interattivo. ↳ «Tooltip sull'icona "i" che spiega cosa fa il filtro.»

Dropdown (menu a tendina) Menu che si apre al click, con opzioni cliccabili. ↳ «Dropdown "Account" nell'header con Profilo, Settings, divisore, Logout (rosso).»

Button (bottone) Elemento cliccabile per azioni. Varianti: primary, secondary, ghost, destructive, icon-only. ↳ «Bottone primary "Salva" + bottone ghost "Annulla" affianco.»

Link Elemento testuale cliccabile per navigazione. Tag `<a href>`.

Form (modulo) Insieme di campi per inserire dati. Submit invia al server.

Input (campo di input) Singolo campo del form. Tipo testo, numero, password, email, ecc.

Checkbox (casella) Casella ☐/☑ per opzioni multiple selezionabili.

Radio (scelta esclusiva) Cerchi ⚪ / 🔘 per opzioni mutuamente esclusive.

Select (menu a tendina di scelta) Input che permette di scegliere un valore da una lista. Diverso da dropdown (che è di azioni).

Slider (cursore) Cursore per scegliere un valore in un range (es. prezzo 0-200€).

Toggle (interruttore) Switch on/off. Tipico delle settings.

Datepicker (selettore data) Calendario per scegliere una data.

E. Feedback (8 termini)

Toast / Snackbar (notifica temporanea) Notifica piccola in un angolo, auto-dismiss dopo qualche secondo. ↳ «Toast verde in basso a destra dopo submit: "Salvato", auto-dismiss 4s.»

Alert (avviso) Avviso visivamente forte (banner persistente o popup) per messaggi importanti.

Progress bar (barra di avanzamento) Barra che mostra avanzamento di un processo. Determinato (% nota) o indeterminato.

Spinner Icona rotante per indicare «sto caricando». Generico.

Skeleton loader (scheletro di caricamento) Placeholder grigio che simula la forma del contenuto in arrivo. Più moderno e rassicurante dello spinner. ↳ «Skeleton loader durante il fetch dei prodotti: 6 rettangoli grigi a forma di card.»

Empty state (stato vuoto) Schermata mostrata quando una lista non ha contenuto. Illustrazione + messaggio + CTA. ↳ «Empty state lista cantieri: illustrazione, frase "Crea il tuo primo cantiere", bottone "Nuovo cantiere".»

Error state (stato di errore) Schermata mostrata quando una richiesta fallisce. Messaggio chiaro + pulsante «Riprova».

Success state (stato di successo) Conferma visiva dopo operazione riuscita. Icona verde, messaggio, eventuale «Vai avanti».

F. Tipografia e elementi fini (5 termini)

Heading (h1-h6) (titolo) Titoli gerarchici. h1 = principale (uno solo per pagina), h2 = sezione, h3 = sottosezione, ecc.

Paragraph (p) (paragrafo) Blocco di testo normale.

List (ul/ol/li) (lista) Lista non ordinata (`<ul>` con bullet) o ordinata (`<ol>` con numeri). Item con `<li>` .

Badge / Chip / Tag (etichetta) Piccolo elemento colorato. Badge = numero piccolo (es. notifiche carrello). Chip = tag interattivo con X. Tag = etichetta di categoria. ↳ «Badge rosso con numero notifiche sull'icona campanella.»

Avatar Immagine profilo circolare. Fallback con iniziali se manca la foto. ↳ «Avatar 40px con iniziali su sfondo accent se la foto manca.»

📝 Bonus: come trasformare in prompt prompt-ready

Per ogni termine, il pattern del prompt ben fatto è:

> **SUGGERIMENTO**
>
> «[componente] [posizione/dimensione] [contenuto] [comportamento] [responsive] [stati]»

Esempio applicato:

> **SUGGERIMENTO**
>
> «Card prodotto (componente) in grid 3 col desktop, 1 mobile (posizione/dimensione), con immagine 4:3, titolo, prezzo, bottone “Aggiungi” (contenuto). Hover scale 1.02 (comportamento). Card vuota mostra skeleton loader (stato).»

Lungo? Sì. Ma è codice production-ready alla prima iterazione.

Glossario 2 redatto: 2026-04-25 · Versione 1.0 · 50 termini in 6 categorie.

APPENDICI

Strumenti pratici

Cheat sheet, checklist, prompt template, GDPR. Stampabili.

Appendice A — Cheat Sheet del Debug

SUGGERIMENTO

4 pagine · Riferimento rapido stampabile
Per ogni errore frequente: **cosa significa · dove guardare · cosa dire a Claude**.

Errori di rete / API

`Failed to fetch` / `NetworkError`

Cosa significa: il browser non riesce a raggiungere il server. **Dove guardare**: Network tab → richiesta fallita → status `(failed)`. **Cause tipiche**: server giù, rete locale assente, CORS bloccante. **A Claude**: «Errore Failed to fetch su POST /api/X dal frontend Next.js a backend Express. Server raggiungibile via curl. Sospetto CORS.»

`CORS policy: No 'Access-Control-Allow-Origin'`

Cosa significa: il backend non autorizza il dominio del frontend. **Dove guardare**: Network tab → richiesta in rosso → tab Headers → manca `Access-Control-Allow-Origin` in Response. **Fix**: configurare CORS sul backend con il dominio del frontend. **A Claude**: «Errore CORS in produzione tra

`app.miosito.it` e `api.miosito.it`. Configura cors middleware Express per accettare entrambi gli origin con credentials true.»

`401 Unauthorized`

Cosa significa: non sei autenticato (manca token / sessione / cookie). **Dove guardare**: Network → richiesta 401 → tab Headers → c'è `Authorization` o `Cookie`? **Cause tipiche**: token scaduto, cookie non inviato, NEXTAUTH_SECRET mancante in produzione.

`403 Forbidden`

Cosa significa: sei autenticato ma non hai permessi per questa operazione. **Dove guardare**: log del server → cerca controlli di autorizzazione.

`404 Not Found`

Cosa significa: la rotta non esiste sul server. **Dove guardare**: hai sbagliato URL? Il file di route esiste? In Next.js: `app/api/X/route.ts`?

`500 Internal Server Error`

Cosa significa: bug nel codice del server, eccezione non gestita. **Dove guardare**: **log del server** (Vercel Functions Logs, PM2 logs). NON in console browser. **A Claude**: «500 su /api/X. Log server: `[errore copiato]`. Codice attuale: `[snippet]`.»

`502 Bad Gateway`

Cosa significa: nginx non riesce a parlare col backend. **Dove guardare**: il backend gira? `pm2 status`, `systemctl status myapp`.

`429 Too Many Requests`

Cosa significa: rate limit superato. **Fix**: backoff esponenziale, caching, code di richieste.

Errori JavaScript browser

`Cannot read property 'X' of undefined`

Cosa significa: stai accedendo a una proprietà di un oggetto che non esiste. **Causa tipica**: dati non ancora caricati, optional chaining mancante. **Fix**: usa `?.` (optional chaining) o controlla `if (data)` prima di accedere.

`X is not a function`

Cosa significa: stai chiamando come funzione qualcosa che non lo è. **Cause**: import sbagliato (default vs named), variabile reassegnata.

`Hydration failed because...` (Next.js / React)

Cosa significa: il server ha renderizzato HTML diverso da quello che il client si aspetta. **Cause tipiche**: usare `Date.now()`, `Math.random()`, `localStorage` in Server Components. **Fix**: spostare in Client Component (`'use client'`) o usare `useEffect`.

`useState is not defined / can only be used in Client Components`

Cosa significa: stai usando hook React in un Server Component. **Fix**: aggiungi `'use client'` in cima al file.

Errori database

`column "X" does not exist`

Cosa significa: il codice cerca una colonna che il DB non ha. **Causa tipica**: migration applicata in locale ma non in produzione. **Fix**: applica le migration in produzione (`drizzle-kit push`, `prisma migrate deploy`).

`relation "X" does not exist`

Cosa significa: la tabella non esiste. **Fix**: applica le migration di creazione iniziale.

`connection refused` (DB)

Cosa significa: non riesce a connettersi al DB. **Cause**: DATABASE_URL sbagliata, DB giù, firewall che blocca.

`duplicate key violates unique constraint`

Cosa significa: stai inserendo un valore già presente in una colonna unique. **Fix**: gestisci il caso (es. mostrare «Email già registrata»).

Errori build / deploy

`Module not found: Can't resolve 'X'`

Cosa significa: import a un pacchetto non installato. **Fix**: `pnpm install X`.

`Type error: Property X does not exist`

Cosa significa: errore TypeScript. **Fix**: aggiusta i tipi, oppure aggiungi i missing field.

`EADDRINUSE: address already in use :::3000`

Cosa significa: la porta 3000 è già occupata da un altro processo. **Fix**: `lsof -i:3000` per trovarlo, killalo. Oppure usa porta diversa.

`Build failed` (Vercel/Netlify)

Dove guardare: log del provider → errore di build. **Cause comuni**: variabile env mancante in produzione, type error, dipendenza non installata.

Errori cookie / sessione

«Login funziona ma l'utente è sloggato dopo 1 ora»

Causa: sessione JWT con scadenza default 1h. **Fix**: configura `maxAge: 30 * 24 * 60 * 60` in NextAuth.

«Login loop in produzione»

Causa probabile: cookie `Secure` non rinviato perché HTTP da qualche parte, oppure dominio cookie sbagliato. **Dove guardare**: DevTools → Application → Cookies. Verifica flag.

«In locale funziona, in produzione no» (login)

Cause possibili: HTTPS vs HTTP, dominio cookie sbagliato, `Secure: true` non differenziato per ambiente.

Pattern di prompt per ogni famiglia di errore

Per **errori HTTP/API**: «Stack [framework]. POST/GET a [URL]. Status [codice]. Log server: [paste]. In locale: [funziona/non funziona]. Già provato: [...].»

Per **errori JavaScript browser**: «In componente [nome], errore [paste console]. Codice rilevante: [snippet]. Stack: Next.js 14 / React 19.»

Per **errori database**: «Errore: [paste]. Stack: Drizzle + Postgres su Supabase. Migration applicate: [sì/no]. Schema attuale: [snippet].»

Per **build/deploy fallito**: «Build fallita su Vercel. Log: [paste]. Funziona in locale con `pnpm build`. Vars env settate.»

Appendice A redatta: 2026-04-25 · Versione 1.0

Appendice B — Checklist Pre-Deploy in 15 punti

SUGGERIMENTO

2 pagine · Da spuntare prima di andare live con un sito serio
Stampabile come check-list cartacea

Sicurezza & autenticazione

- [ ]
  1. **Variabili d'ambiente settate in produzione**

Tutte le variabili in `.env.example` esistono nel pannello del provider (Vercel, Railway, Aruba). Specialmente: API keys, secret tokens, DATABASE_URL.

- [ ]
  2. **Secret tokens generati con valori forti**

`NEXTAUTH_SECRET`, `JWT_SECRET`, ecc. devono essere stringhe casuali di 32+ caratteri. Mai valori default tipo «secret» o «changeme».

- [ ]
  3. **HTTPS attivo, certificato valido**

Verifica `https://miosito.it` mostra il lucchetto verde. Su VPS: certbot ha generato il certificato. Il certificato non scade nei prossimi 30 giorni.

- [ ]
  4. **CORS configurato con domini specifici (no `*`)**

Sul backend, `Access-Control-Allow-Origin` accetta solo i domini frontend reali. Mai `*` con autenticazione.

- [ ]

5. **Cookie con flag corrette**

Sessione/auth: `HttpOnly: true`, `Secure: true` in produzione, `SameSite: Lax` o `Strict`. Verifica in DevTools → Application → Cookies.

Database & dati

- [ ]

6. **Migration applicate in produzione**

Lo schema del DB di produzione è allineato col codice. Niente «column does not exist».

- [ ]

7. **Backup automatici attivi e testati**

Almeno 1 backup giornaliero con retention ≥ 7 giorni. Hai fatto **almeno un test di restore** in un ambiente di test.

- [ ]

8. **Dati di test rimossi dalla produzione**

Nessun «Lorem ipsum», nessun utente «test@test.it» admin con password «password».

Performance & UX

- [ ]

9. **Errori console puliti**

Apri il sito in produzione. F12 → Console. **Zero errori rossi**. Eventuali warning gialli valutati.

- [ ]

10. **Lighthouse score accettabile**

`npx lighthouse https://miosito.it --view` o devtools → Lighthouse. Target: Performance ≥ 70, Accessibility ≥ 90, Best Practices ≥ 90.

- [ ]

11. **Loading e error states gestiti**

Per ogni fetch importante: hai un loading state (skeleton/spinner) e un error state (messaggio + retry). Niente schermi bianchi.

- [ ]

12. **Mobile responsive verificato su device reale**

Apri il sito su iPhone reale, non solo Chrome DevTools. Tap target ≥ 48px. Niente zoom orizzontale.

SEO & analytics

- [ ]
  13. `robots.txt` e `sitemap.xml` presenti e corretti

`https://miosito.it/robots.txt` esiste e non blocca tutto per errore. Sitemap aggiornata. Per Next.js: `app/robots.ts` e `app/sitemap.ts`.

- [ ]
  14. Meta tag base presenti

Title, description, OG tags (per condivisioni social), favicon. Verifica con `view-source:` o estensione browser.

- [ ]
  15. Monitoring degli errori attivo

Sentry, Logflare, o simili: gli errori in produzione ti arrivano via email/Slack. Senza, non sai mai quando si rompe.

⚠ Bonus: stop-the-press

Prima del primo deploy con utenti veri:

- [ ] **Hai una pagina contatto / supporto** funzionante?
- [ ] **Hai una privacy policy** anche minima (vedi Appendice D)?
- [ ] **Hai un cookie banner** se usi cookie (analytics, marketing)?
- [ ] **Hai testato il flusso principale** (registrazione, login, acquisto, ecc.) end-to-end con un utente diverso da te?
- [ ] **Hai un piano di rollback** se il deploy va male? (Vercel ha rollback con un click; su VPS: `git revert` + redeploy)

Appendice B redatta: 2026-04-25 · Versione 1.0

Appendice C — I 10 Prompt da Salvare

SUGGERIMENTO

3 pagine · Prompt copiabili Claude-ready
Adatta i placeholder `[in parentesi]` al tuo contesto.

1. Setup nuovo progetto Next.js full-stack

```
Inizializza un nuovo progetto Next.js 14 con:
- TypeScript
- Tailwind CSS v4
- App Router (non Pages)
- Drizzle ORM + Postgres su [Supabase / Neon]
- NextAuth v5 con provider Credentials
- ESLint + Prettier configurati

Setup in `cd [nome-progetto]`. Includi un file
`.env.example` con tutte le variabili necessarie. Crea uno
schema base con tabella `users` (id, email, password_hash,
created_at) e una migration iniziale.
```

2. Aggiungere autenticazione email/password

```
Aggiungi autenticazione email + password al progetto Next.js
esistente.

- Form di login `/login` e registrazione `/register` come
Client Components
- Validation Zod (email valida, password min 12 char)
- Endpoint API `/api/auth/register` e `/api/auth/login`
- Password hashing con bcrypt cost 12
- Sessione via NextAuth v5 JWT, cookie httpOnly + Secure
- Middleware che protegge `/dashboard/*`
- Errori inline sotto i campi, toast per success/error
globale

Stack: [versione Next.js], NextAuth v5, Drizzle ORM,
Postgres.
```

3. Debug API che ritorna 401 in produzione

```
Stack: [framework e versione]. Deployato su [Vercel/Aruba/
altro].

Sintomo: POST a `/api/[endpoint]` ritorna 401 in produzione
(`https://miosito.it`). In locale (`localhost:3000`)
funziona.

Browser console: [paste errore]
Network tab: [paste request/response headers se rilevanti]
Server log: [paste]

Sospetto: [tua ipotesi: variabile env mancante, cookie non
```

```
inviato, ecc.]

Già verificato:
- Variabile env settata in pannello provider [sì/no]
- Cookie presente in DevTools → Application [sì/no]
- API key valida (test con curl) [sì/no]

Aiutami a localizzare la causa e fornire un fix specifico.
```

4. Configurare CORS correttamente

```
Backend: [Express / Next.js API routes / Laravel / Django]
su [URL backend].
Frontend: [Next.js / Vite] su [URL frontend].

Configura CORS sul backend per:
- Accettare richieste da `[URL frontend produzione]` e
`http://localhost:3000` (dev)
- Permettere metodi GET, POST, PATCH, DELETE
- Permettere header Content-Type, Authorization
- credentials: true per i cookie httpOnly

Dammi il codice completo da aggiungere e dove va inserito.
```

5. Aggiungere upload di file

```
Aggiungi upload di immagini al progetto.

- Frontend: input `<input type="file" accept="image/*">` con
preview
- Validazione client: max 5MB, solo image/*
- Storage: [Supabase Storage / Cloudflare R2 / AWS S3]
- Endpoint API che riceve il file, lo carica sul storage,
ritorna URL pubblico
- Salvare URL nella tabella [nome] colonna [colonna]
- Gestione errori: file troppo grande (413), tipo sbagliato
(415), upload fallito (500)
- Loading state durante upload con progress bar

Stack: Next.js 14 + [storage scelto].
```

6. Query filtrata e paginata

```
Crea endpoint API GET `/api/[risorsa]` che ritorna lista
filtrata e paginata.

Filtri (query params):
- `?search=` (ricerca su [colonne])
- `?status=` (uno tra [valori])
- `?from=` `?to=` (range data)

Paginazione:
- `?page=1` `?limit=20` (default)
- Risposta: `{ data: [...], total: number, page: number,
totalPages: number }`

Performance:
- Indici su colonne filtrate
```

```
- Limit max 100 (no abuse)
- Count query separata per `total`

Stack: Drizzle ORM + Postgres.
```

7. Migration database con dati esistenti

```
Voglio modificare la tabella `[nome]` aggiungendo:
- Colonna `[nome_colonna]` di tipo `[tipo]`, nullable: [sì/
no]
- Default value: `[valore]`
- Eventuale index

Considera che ci sono già [N] righe in produzione.

Crea:
1. La migration Drizzle/Prisma con `ALTER TABLE`
2. Eventuale script di backfill per popolare la nuova
colonna sui dati esistenti
3. Comando per applicare in produzione safely

Mostrami anche come fare rollback se qualcosa va storto.
```

8. Test end-to-end del flusso login

```
Aggiungi test E2E con [Playwright / Cypress] per il flusso
di login completo.

Test casi:
- Login con credenziali corrette → redirect a /dashboard,
vedo "Benvenuto [nome]"
- Login con email sbagliata → errore inline "Email non
riconosciuta"
- Login con password sbagliata → errore "Password errata"
- Form vuoto submitato → errori sui campi
- Persistenza sessione: chiudo browser e riapro, sono ancora
loggato
- Logout: redirect a / e session cookie cancellato

Setup: usare un user di test seedato in DB di test, cleanup
dopo ogni test.
```

9. Documentazione automatica per API

```
Genera documentazione OpenAPI per tutte le route in `app/
api/*/route.ts`.

Per ogni endpoint:
- Path, method, description
- Request body schema (Zod)
- Response schema (success + errori 400, 401, 422, 500)
- Esempio request/response

Output: file `openapi.yaml` + endpoint `/api/docs` che serve
Swagger UI per browse interattivo.

Stack: Next.js 14, Zod schema già definiti.
```

10. Performance audit e ottimizzazione

```
Il sito `[URL]` è lento. Lighthouse score: Performance
[X]/100.

Fai un audit performance e proponi fix concreti:

1. Analizza `next build --turbo` output: bundle size, route
che pesano di più
2. Identifica componenti pesanti che dovrebbero essere lazy-
loaded
3. Verifica immagini: sono ottimizzate? Usano `next/image`?
4. Server Components vs Client: ci sono Client unnecessary?
5. Database: query N+1 nascoste? Indici mancanti?
6. CDN: file statici sono cachati correttamente?

Ordina i fix per impatto/effort. Mostra il prima/dopo
Lighthouse atteso.
```

✏️ Pattern generale del prompt informato

Per qualsiasi richiesta, usa questo template a 7 punti:

```
1. STACK: [framework, versione, librerie principali]
2. CONTESTO: [cosa esiste già nel progetto, file rilevanti]
3. OBIETTIVO: [cosa voglio in linguaggio funzionale]
4. LAYOUT: [lessico UI preciso — vedi Glossario 2]
5. COMPORTAMENTO: [stati, edge case, validazioni]
6. SICUREZZA: [auth, autorizzazioni, segreti]
7. NON FARE: [vincoli espliciti — niente OAuth, no
Bootstrap, ecc.]
```

90 secondi di prompt vs 30 minuti di iterazioni. Vale la pena.

Appendice C redatta: 2026-04-25 · Versione 1.0

Appendice D — GDPR per Vibecoder Italiani

ATTENZIONE

2 pagine · Le cose pratiche da fare PRIMA di andare online con un sito che raccoglie dati
⚠️ Non è consulenza legale. È checklist operativa per progetti piccoli. Per progetti grossi, consulta un avvocato.

Cosa è GDPR (in 30 secondi)

GDPR (General Data Protection Regulation) è il regolamento europeo sulla privacy in vigore dal 2018. Vale per **chiunque tratta dati personali di cittadini UE**, inclusi vibecoder italiani con app web, anche piccole.

Non è un'opzione. Le multe partono da 20.000€ per piccole violazioni e arrivano a milioni di euro per quelle gravi.

Buona notizia: per piccoli SaaS / siti / app indie, **rispettarlo costa zero** — basta fare le cose giuste fin dall'inizio.

Le 5 cose da fare assolutamente

1. Privacy Policy

Una pagina sul tuo sito che dichiara:

- **Chi tratta i dati** (tu, nome cognome, email contatto)
- **Quali dati raccogli** (es. email, nome, IP, dati pagamento)
- **Perché** (es. «per gestire l'account utente», «per inviare la newsletter»)
- **Per quanto tempo** li tieni (es. «fino alla cancellazione dell'account»)
- **Con chi li condividi** (es. «Stripe per i pagamenti, Postmark per email»)
- **I diritti dell'utente** (accesso, rettifica, cancellazione, portabilità)
- **Come contattarti** per esercitare i diritti

Generatore gratis online: iubenda.com (free per progetti semplici), oppure termly.io. Costa zero.

2. Cookie Banner (se usi cookie non strettamente necessari)

Se usi:

- **Google Analytics** o equivalente
- **Cookie di marketing** (Facebook Pixel, ads)
- **Profilazione** utenti

→ devi mostrare un **banner cookie** che chiede consenso **prima** di settare i cookie.

Cookie necessari (di sessione, login) **non richiedono consenso**. Ma devi comunque dichiararli.

Strumenti gratis: Cookiebot free, Iubenda free. 30 minuti di setup.

3. Dove vivono i dati

Importante per GDPR: i dati personali devono stare in **server EU** quando possibile. Se li metti su server US, devi avere base legale (es. Standard Contractual Clauses) e dichiararlo nella privacy.

Servizi EU-friendly:

- **Supabase**: scegli regione EU (Frankfurt, Dublin) durante la creazione del progetto. Importante.
- **Neon**: regione EU (Frankfurt) disponibile.
- **Aruba**: server in Italia, perfetto.
- **Vercel**: il piano free serve via CDN globale ma il database può essere EU.

Servizi che richiedono attenzione:

- **Firebase** (US): tecnicamente conforme ma richiede dichiarazioni extra.
- **Auth0** (US): idem.

- **Stripe**: ha entità EU, ok.
- **OpenAI** (US): nel disclaimer dichiara che i dati possono essere processati negli USA.

4. Email transazionali e marketing

Se mandi email all'utente:

Email transazionali (conferma registrazione, password reset, fattura): non richiedono consenso. Sono «necessarie».

Email marketing (newsletter, promo): richiedono **opt-in esplicito**. Niente «iscritto di default». Sempre **link di unsubscribe** funzionante in ogni email.

Strumenti email-marketing GDPR-friendly: **Resend, Postmark, Brevo (ex Sendinblue)**.

5. Diritto all'oblio (delete account)

L'utente deve poter **cancellare il proprio account** con i suoi dati. Implementa:

- Pulsante «Elimina account» nel profilo
- Conferma con password (per sicurezza)
- Cancellazione immediata o entro 30 giorni di tutti i dati personali (con eccezioni per obblighi fiscali — es. fatture devono essere tenute 10 anni)
- Email di conferma cancellazione

Implementare ora costa 1 ora. Implementare dopo (perché un utente lo richiede legalmente) costa una settimana di panico.

Casi specifici per il vibecoder italiano

Sito vetrina senza login

Se hai solo un sito vetrina (no account, no form, no analytics aggressive):

- Privacy policy obbligatoria (anche se semplice)
- Cookie banner non obbligatorio se non usi cookie non-essenziali
- Server EU consigliato

10 minuti di lavoro.

SaaS con utenti EU

Se hai utenti che pagano:

- Privacy policy completa
- Cookie banner se usi analytics
- Dati su server EU (preferito)
- Diritto all'oblio implementato
- Backup conservati con retention dichiarata

1 ora di lavoro.

App con AI che processa contenuti utenti

Se la tua app manda contenuti utente a OpenAI/Anthropic:

- **Dichiara** che i dati passano per server US (questi provider)
- Se possibile, anonimizza i dati prima di inviare
- Per dati molto sensibili, valuta provider EU (Mistral) o on-premise

Checklist finale

Prima del primo utente reale:

- [ ] Privacy Policy generata e linkata in footer
- [ ] Cookie Banner attivo (se applicabile)
- [ ] Server in regione EU configurati
- [ ] Email transazionali con unsubscribe
- [ ] «Elimina account» funzionante
- [ ] Hai dichiarato chi processa cosa nella privacy

Tutto fatto in **mezza giornata**. Vita più tranquilla per anni.

Link utili

- Garante Privacy italiano — testo della legge
- iubenda.com — generatore privacy/cookie
- GDPR Made Easy (in italiano) — guida pratica

SUGGERIMENTO

Disclaimer: questa appendice è una guida operativa per piccoli progetti. Non sostituisce la consulenza di un avvocato. Per progetti con dati sensibili (sanitari, finanziari, minori), parla con un esperto. Costano 100€/ora ma ti risparmiano problemi da 100.000€.

Appendice D redatta: 2026-04-25 · Versione 1.0

`{ vibecoding • serio }`

Fine del libro.

Buon vibecoding.

Sul serio.